

Departamento de Informática

Acceso a Datos: Spring Rest y Mongo en acción

Juan Gualberto Gutiérrez Marín

Enero 2026

AVISO: El texto contiene muchos enlaces embebidos que sólo funcionan si se abre el PDF desde un lector digital. Si se imprime, se perderá mucha información interesante. No obstante, en el apartado “Bibliografía” se han añadido los enlaces para aquellos lectores que deseen una copia impresa y puedan ver la información.

Este documento se encuentra bajo una licencia Creative Commons de Atribución-CompartirIgual (CC BY-SA).

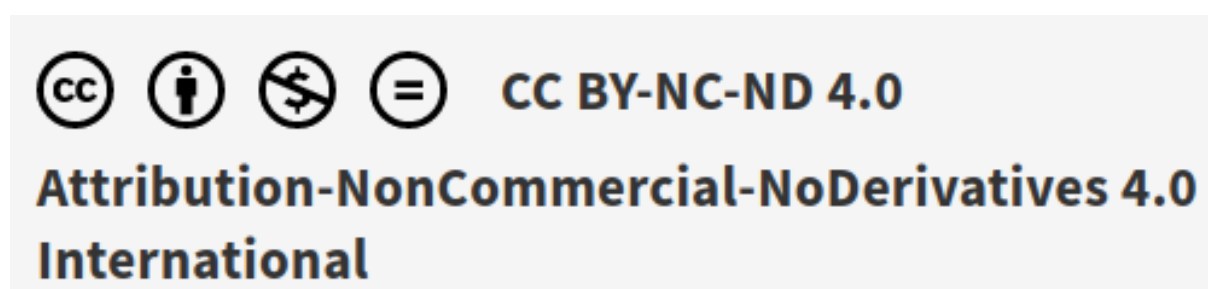


Figura 1: Atribución-CompartirIgual (CC BY-SA)

Esto significa que puedes:

- Compartir: copiar y redistribuir el material en cualquier medio o formato.
- Adaptar: remezclar, transformar y construir sobre el material para cualquier propósito, incluso comercialmente.

Bajo las siguientes condiciones:

- Atribución: debes dar crédito de manera adecuada, proporcionar un enlace a la licencia e indicar si se han realizado cambios. Puedes hacerlo de cualquier manera razonable, pero no de una manera que sugiera que el licenciante te respalda a ti o al uso que hagas del trabajo.
- Compartir igual: si remezclas, transformas o creas a partir del material, debes distribuir tus contribuciones bajo la misma licencia que el original.

Para más detalles, consulta la licencia completa.

Para una versión actualizada de este libro visita esta Web: <https://gitlab.iesvirgendelcarmen.com/juangu>.

Índice

1	Introducción a las bases de datos NoSQL	7
1.1	Introducción a las bases de datos NoSQL	7
1.1.1	Características generales de las bases de datos NoSQL	7
1.1.2	Tipos principales de bases de datos NoSQL	7
1.1.3	Ventajas e inconvenientes de NoSQL frente a bases de datos relacionales	9
1.2	MongoDB	11
1.2.1	Conceptos fundamentales	11
1.2.2	Instalación y configuración de MongoDB	11
1.2.3	Conexión interactiva a MongoDB	14
1.2.4	Operaciones CRUD básicas en MongoDB	16
1.2.5	Colección de ejemplo: estudiantes	17
1.3	Introducción: REST + MongoDB + SPA ligera	19
1.3.1	Objetivo general del capítulo	19
1.3.2	Arquitectura de la aplicación	19
1.3.3	Por qué REST, MongoDB y una SPA ligera	20
1.3.4	Caso práctico: gestión de reservas de pistas deportivas	20
1.4	MongoDB	24
1.4.1	Instalación del Servidor y el Shell	24
1.4.2	Configuración del usuario Root (Primer inicio)	25
1.4.3	Cómo conectar (Pruebas de acceso)	25
1.4.4	Recuperación del Root (Si olvidas la clave)	26
1.5	Modelado y Creación de la Base de Datos: Gestión Académica	27
1.5.1	Estrategias de Relación: Embebido vs. Referencia	27
1.5.2	Implementación Práctica en la Terminal	27
1.5.3	Resumen de diferencias	29
1.6	Manipulación de Datos (Operaciones CRUD)	30
1.7	Consultas Complejas: El Aggregation Framework	31
1.7.1	MongoDB Cheat Sheet: Gestión Académica	32
1.7.2	Documentación Oficial	34
1.8	Ejercicio Práctico: Sistema de Gestión de Escuela de Ingeniería	35
1.8.1	Fase 1: Seguridad y Conexión	35
1.8.2	Fase 2: Modelado e Inserción	35
1.8.3	Fase 3: Manipulación (CRUD)	36
1.8.4	Fase 4: Desafío de Agregación (Reporting)	36
1.9	Preparación del entorno y dependencias	37
1.9.1	Requisitos previos	37

1.9.2	Estructura de carpetas recomendada	37
1.9.3	Creación del archivo .env (variables compartidas)	37
1.9.4	Preparación del Dev Container	38
1.9.5	Creación del proyecto Spring Boot (Maven)	40
1.9.6	Dependencias (starters) necesarias	41
1.9.7	Configuración en YAML (application.yml) usando variables de entorno	41
1.9.8	Verificación del entorno	42
1.9.9	Nota sobre “mongoengine”	43
1.9.10	Archivo .env de ejemplo	43
1.9.11	Archivo .gitignore recomendado	44
1.9.12	Arranque de la aplicación Spring Boot desde el contenedor	44
1.9.13	Cheatsheet para troubleshooting	45
1.10	Spring Data MongoDB: @Document, repositorios y consultas	47
1.10.1	Dependencias necesarias	47
1.10.2	Diseño del modelo y decisiones de persistencia	47
1.10.3	Paquetes recomendados	47
1.10.4	1) Documentos y embebidos	48
1.10.5	2) Repositorios (MongoRepository) y consultas	53
1.10.6	Consideraciones de consulta y modelado	54
1.10.7	Estado del proyecto tras este apartado	55
1.11	Diseño de la API REST: endpoints, DTOs, códigos HTTP	56
1.11.1	Principios de diseño aplicados	56
1.11.2	Recursos y endpoints	56
1.11.3	DTOs (Data Transfer Objects)	59
1.11.4	Códigos HTTP y comportamiento esperado	60
1.11.5	Ejemplos de intercambio (contrato)	61
1.11.6	Preparación para el siguiente apartado	61
1.12	Validación, DTOs y Control de Errores (@ControllerAdvice)	62
1.12.1	1. Entendiendo el patrón DTO (Data Transfer Object)	62
1.12.2	2. Implementación de Validación en DTOs	63
1.12.3	3. Manejo Centralizado de Excepciones	66
1.12.4	4. Resumen de Códigos de Estado HTTP	71
1.12.5	Conclusiones	72
1.13	Validación y control de errores (@ControllerAdvice)	73
1.13.1	1) Validación de entrada en la API	73
1.13.2	2) DTOs con anotaciones de Bean Validation	74
1.13.3	3) Modelo de error unificado	75
1.13.4	4) Excepciones de dominio	75

1.13.5	5) Manejo centralizado con <code>@ControllerAdvice</code>	76
1.13.6	6) Reglas de negocio y códigos HTTP	77
1.13.7	Resultado de este apartado	78
1.14	CORS y configuración por entornos	79
1.14.1	1) Conceptos básicos de CORS	79
1.14.2	2) Estrategias posibles en este proyecto	79
1.14.3	3) Configuración de CORS en Spring Boot	80
1.14.4	4) Configuración por entornos con perfiles	81
1.14.5	5) Variables de entorno necesarias	82
1.14.6	6) Verificación de CORS	82
1.14.7	Resultado de este apartado	83
1.15	Front con Bootstrap: estructura y componentes	84
1.15.1	Estructura de archivos recomendada	84
1.15.2	Diseño de la interfaz	84
1.15.3	<code>index.html</code> (estructura base)	85
1.15.4	<code>css/app.css</code> (opcional)	89
1.15.5	Plantillas de tabla (contrato de renderizado)	90
1.15.6	Objetivo técnico del apartado	90
1.15.7	Consumo de API desde JS (jQuery)	92
1.16	Caso práctico completo (paso a paso)	102
1.17	Paso 1: Arranque del entorno	102
1.17.1	1.1 Abrir el proyecto en Dev Container	102
1.17.2	1.2 Verificar servicios	102
1.18	Paso 2: Arranque del backend Spring Boot	103
1.19	Paso 3: Creación de datos base (API REST)	103
1.19.1	3.1 Crear instalaciones	103
1.19.2	3.2 Crear usuarios	104
1.20	Paso 4: Creación de reservas (regla de negocio)	104
1.20.1	4.1 Crear una reserva válida	104
1.20.2	4.2 Intentar una reserva solapada (conflicto)	105
1.21	Paso 5: Consultas avanzadas	106
1.21.1	5.1 Consultar reservas por instalación y día	106
1.21.2	5.2 Consultar reservas por usuario	106
1.22	Paso 6: Uso del frontend (flujo completo)	106
1.22.1	6.1 Gestión de instalaciones	106
1.22.2	6.2 Gestión de usuarios	107
1.22.3	6.3 Gestión de reservas	107
1.23	Paso 7: Verificación en Mongo Express	107

1.24	Paso 8: Documentación y conclusiones	108
1.25	Resultado final del caso práctico	108
1.26	Spring Security, login y ACLs contra MongoDB	109
1.26.1	1) pom.xml: dependencias nuevas	109
1.26.2	2) Modelo: añadir rol y contraseña al usuario	109
1.26.3	3) Repositorio de usuarios: login por email	111
1.26.4	4) Seguridad: UserDetailsService + PasswordEncoder + configuración HTTP	111
1.26.5	5) Nuevo documento: Horarios disponibles (para que ADMIN gestione horarios)	114
1.26.6	6) Reglas de negocio y endpoints por rol	116
1.26.7	7) Servicios: garantizar reglas “solo sus reservas” y “no pasada”	117
1.26.8	8) Controladores por rol (backend)	118
1.26.9	9) Endpoint útil: “quién soy” (para frontend)	124
1.26.10	Resumen de lo que queda implementado en backend	124
1.26.11	1) DTOs ADMIN	125
1.26.12	2) Repositorio de horarios (si no lo añadiste ya)	127
1.26.13	3) Controlador ADMIN: Usuarios (CRUD completo)	127
1.26.14	4) Controlador ADMIN: Instalaciones (CRUD completo)	129
1.26.15	5) Controlador ADMIN: Horarios disponibles (CRUD completo)	130
1.26.16	6) Recomendación: ocultar passwordHash en respuestas públicas	133
1.26.17	7) Endpoints ADMIN finales (resumen)	133
1.26.18	8) Nota importante para el siguiente paso (frontend)	134
1.26.19	Frontend con login + UI por rol (ADMIN / CONSERJE / CLIENTE)	134

2 Bibliografía

160

1 Introducción a las bases de datos NoSQL

1.1 Introducción a las bases de datos NoSQL

Las bases de datos **NoSQL (Not Only SQL)** surgen como respuesta a las limitaciones de las bases de datos relacionales tradicionales en determinados escenarios, especialmente aquellos relacionados con **gran volumen de datos, alta concurrencia, escalabilidad horizontal** y **modelos de datos cambiantes**.

A diferencia de los sistemas relacionales, NoSQL no impone un modelo único basado en tablas, filas y relaciones estrictas, sino que ofrece distintos enfoques de almacenamiento adaptados a diferentes necesidades.

1.1.1 Características generales de las bases de datos NoSQL

Las bases de datos NoSQL comparten una serie de características comunes:

- **Esquema flexible:** los datos no necesitan ajustarse a una estructura rígida predefinida.
- **Escalabilidad horizontal:** están diseñadas para escalar añadiendo nodos, no aumentando la potencia de uno solo.
- **Alto rendimiento** en operaciones de lectura y escritura.
- **Modelo de consistencia flexible**, habitualmente orientado a consistencia eventual en lugar de transacciones ACID estrictas.
- **Orientación al dominio:** el modelo de datos suele adaptarse a cómo se consume la información.

Estas características las hacen especialmente adecuadas para aplicaciones web modernas, sistemas distribuidos y arquitecturas basadas en microservicios.

1.1.2 Tipos principales de bases de datos NoSQL

Las bases de datos NoSQL se clasifican habitualmente en cuatro grandes categorías:

1.1.2.1 Bases de datos clave-valor Almacenan datos como pares `clave` $\rightarrow$ `valor`.

Ejemplo conceptual:

```
1 "session_1234"  $\rightarrow$  { "usuario": "ana", "rol": "admin" }
```


Uso típico:

- Cachés
- Sesiones de usuario
- Configuración temporal

Ventajas:

- Simplicidad
- Velocidad extrema

Inconvenientes:

- Consultas muy limitadas
- No aptas para datos complejos

1.1.2.2 Bases de datos orientadas a columnas Organizan los datos por columnas en lugar de por filas.

Ejemplo conceptual:

```
1 usuario:1:nombre $-> "Luis"
2 usuario:1:email $-> "luis@email.com"
```

Uso típico:

- Analítica
- Big Data
- Series temporales

Ventajas:

- Excelente rendimiento en grandes volúmenes
- Compresión eficiente

Inconvenientes:

- Curva de aprendizaje
- No pensadas para CRUD clásico

1.1.2.3 Bases de datos orientadas a grafos Modelan los datos como **nodos** y **relaciones**.

Ejemplo conceptual:

```
1 (Usuario)-[RESERVA]->(Pista)
```

Uso típico:

- Redes sociales
- Sistemas de recomendación
- Análisis de relaciones complejas

Ventajas:

- Consultas relacionales muy potentes
- Modelo muy expresivo

Inconvenientes:

- No adecuadas para consultas simples masivas
- Modelo menos intuitivo para CRUD básicos

1.1.2.4 Bases de datos orientadas a documentos Almacenan la información en forma de **documentos estructurados**, normalmente en formato JSON o BSON.

Ejemplo conceptual:

```
1 {  
2   "nombre": "Pista Central",  
3   "ciudad": "Jaén",  
4   "tipo": "Tenis"  
5 }
```

Este tipo de base de datos es el que se utilizará en este capítulo.

1.1.3 Ventajas e inconvenientes de NoSQL frente a bases de datos relacionales

Ventajas

- Mayor flexibilidad del modelo de datos.
- Menor acoplamiento entre aplicación y estructura de almacenamiento.
- Escalabilidad sencilla.
- Mejor adaptación a dominios complejos.

Inconvenientes

- Menor estandarización.
- Ausencia de JOINS clásicos.
- Transacciones limitadas o no disponibles según el motor.
- Mayor responsabilidad en la lógica de aplicación.

Estas diferencias hacen que la elección entre SQL y NoSQL dependa del **caso de uso**, no de una superioridad técnica general.

1.2 MongoDB

MongoDB es un ejemplo de base de datos documental. Las **bases de datos documentales** almacenan la información en forma de documentos completos y autocontenidos. Cada documento representa una entidad del dominio y puede incluir estructuras anidadas, listas y subdocumentos.

Este enfoque se alinea de forma natural con los objetos utilizados en los lenguajes de programación orientados a objetos y facilita el desarrollo de aplicaciones backend modernas.

1.2.1 Conceptos fundamentales

1.2.1.1 Documento Unidad básica de almacenamiento. Equivale conceptualmente a una fila en una base de datos relacional, pero con estructura jerárquica.

Ejemplo:

```
1 {  
2   "_id": "64f8a2...",  
3   "nombre": "Instalación Norte",  
4   "ciudad": "Jaén"  
5 }
```

1.2.1.2 Colección Conjunto de documentos del mismo tipo lógico. Equivale a una tabla.

Ejemplo:

- usuarios
- instalaciones
- reservas

1.2.1.3 Objectid Identificador único generado automáticamente por MongoDB. Incluye información temporal y garantiza unicidad sin coordinación central.

1.2.2 Instalación y configuración de MongoDB

Para crear la infraestructura de servicios necesaria, en lugar de instalar MongoDB directamente en el sistema operativo, y siguiendo la metodología habitual del módulo, se utilizarán **contenedores Docker**. En concreto, se desplegarán dos servicios:

- Un contenedor para el **servidor MongoDB**, que actuará como base de datos documental.

- Un contenedor para **Mongo Express**, una interfaz web que permite inspeccionar y administrar MongoDB de forma visual.

Este enfoque presenta varias ventajas: homogeneiza el entorno de trabajo, reduce el tiempo de configuración inicial y evita dependencias con el sistema anfitrión.

Para ello, se crea una carpeta llamada `stack` y, dentro de ella, el archivo `docker-compose.yml` con el siguiente contenido:

```
1 version: '3.1'
2
3 services:
4
5   mongo:
6     image: mongo
7     restart: 'no'
8     environment:
9       MONGO_INITDB_ROOT_USERNAME: root
10      MONGO_INITDB_ROOT_PASSWORD: 83uddjfp0cmMD
11     ports:
12       - 27017:27017
13
14   mongo-express:
15     image: mongo-express
16     restart: 'no'
17     ports:
18       - 8081:8081
19     environment:
20       ME_CONFIG_BASICAUTH_USERNAME: mongo
21       ME_CONFIG_BASICAUTH_PASSWORD: 83uddjfp0cmMD
22       ME_CONFIG_MONGODB_ADMINUSERNAME: root
23       ME_CONFIG_MONGODB_ADMINPASSWORD: 83uddjfp0cmMD
24       ME_CONFIG_MONGODB_URL: mongodb://root:83uddjfp0cmMD@mongo:27017/
```

1.2.2.1 docker-compose.yml El archivo `docker-compose.yml` permite definir y ejecutar múltiples servicios Docker de forma coordinada. A continuación se analizan sus principales secciones.

```
1 version: '3.1'
```

Esta línea indica la versión del esquema de Docker Compose utilizada. La versión 3.1 es compatible con la mayoría de instalaciones actuales de Docker.

1.2.2.1.1 Servicio mongo

```
1 mongo:
2   image: mongo
3   restart: 'no'
```

```
4 environment:
5   MONGO_INITDB_ROOT_USERNAME: root
6   MONGO_INITDB_ROOT_PASSWORD: 83uddjfp0cmMD
7 ports:
8   - 27017:27017
```

Este servicio define una instancia de MongoDB basada en la imagen oficial:

- **image:** se utiliza la imagen oficial `mongo` desde Docker Hub.
- **restart: 'no':** el contenedor no se reiniciará automáticamente si se detiene.
- **environment:**
 - Se define un usuario administrador (`root`) y su contraseña.
 - Estas credenciales se utilizan para acceder a la base de datos.
- **ports:**
 - Se expone el puerto estándar de MongoDB (27017), permitiendo la conexión desde el host y desde otros contenedores.

1.2.2.2 Servicio mongo-express

```
1 mongo-express:
2   image: mongo-express
3   restart: 'no'
4   ports:
5     - 8081:8081
6   environment:
7     ME_CONFIG_BASICAUTH_USERNAME: mongo
8     ME_CONFIG_BASICAUTH_PASSWORD: 83uddjfp0cmMD
9     ME_CONFIG_MONGODB_ADMINUSERNAME: root
10    ME_CONFIG_MONGODB_ADMINPASSWORD: 83uddjfp0cmMD
11    ME_CONFIG_MONGODB_URL: mongodb://root:83uddjfp0cmMD@mongo:27017/
```

Este servicio despliega Mongo Express, una interfaz web de administración:

- **ports:**
 - Se expone el puerto 8081, que permite acceder a la interfaz desde el navegador.
- **ME_CONFIG_BASICAUTH_USERNAME / PASSWORD:**
 - Definen las credenciales para acceder a la interfaz web.
- **ME_CONFIG_MONGODB_ADMINUSERNAME / PASSWORD:**
 - Son las credenciales con las que Mongo Express se autentica contra MongoDB.

- **ME_CONFIG_MONGODB_URL:**

- Cadena de conexión a MongoDB, donde se especifica el usuario, contraseña y nombre del servicio ([mongo](#)).

Es importante diferenciar claramente entre:

- **MongoDB:** motor de base de datos documental.
- **Mongo Express:** herramienta web de administración.

Aunque este entorno es de desarrollo, **siempre se deben usar contraseñas robustas**, ya que una exposición accidental del servicio podría comprometer los datos almacenados.

1.2.2.3 Puesta en marcha de los servicios Una vez creado el archivo `docker-compose.yml`, y teniendo Docker y Docker Compose instalados, se ejecuta el siguiente comando desde la carpeta `stack`:

```
1 docker-compose up -d
```

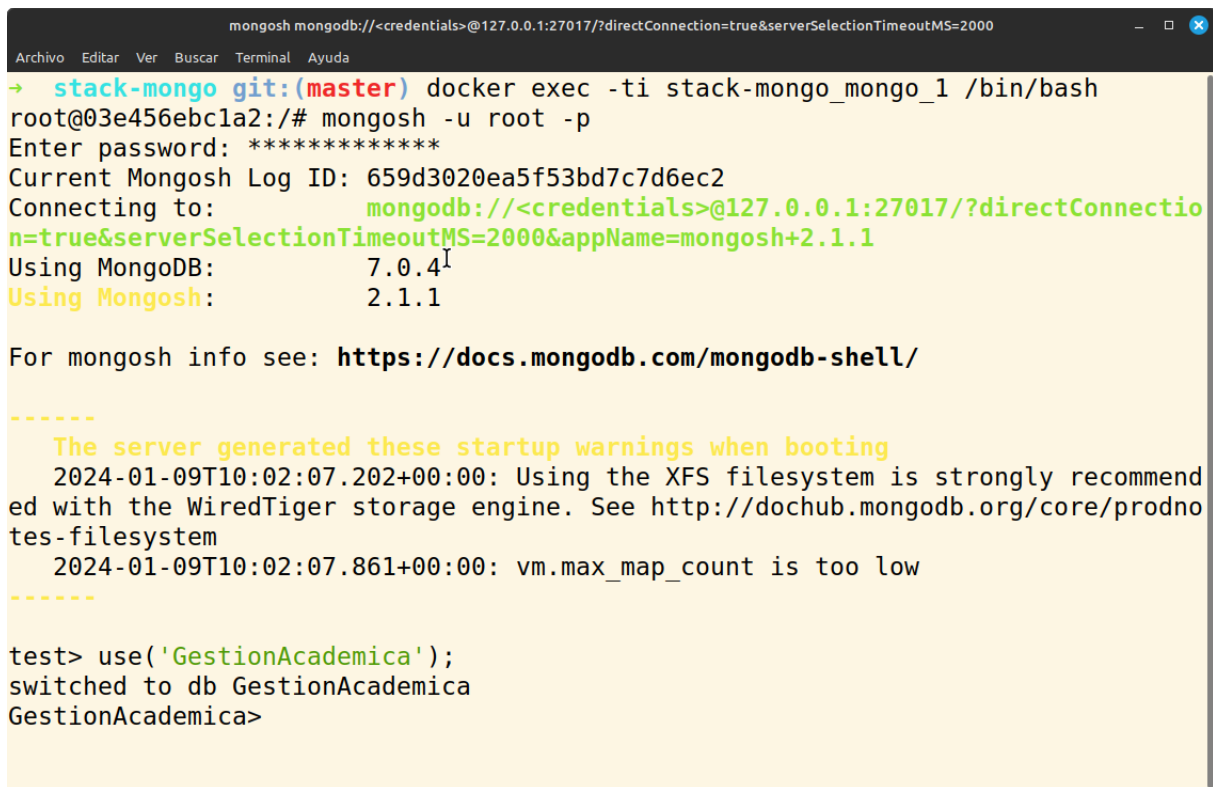
Este comando descarga las imágenes necesarias y lanza los contenedores en segundo plano. Tras unos segundos:

- MongoDB estará disponible en el puerto **27017**.
- Mongo Express será accesible desde el navegador en <http://localhost:8081>.

1.2.3 Conexión interactiva a MongoDB

Aunque en las aplicaciones reales el acceso a MongoDB se realiza desde el backend, resulta muy útil conocer el uso del **shell interactivo** para pruebas, aprendizaje y depuración.

MongoDB utiliza actualmente **mongosh**, que sustituye al antiguo [mongo](#).



```
mongosh mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000
Archivo  Editar  Ver  Buscar  Terminal  Ayuda
→ stack-mongo git:(master) docker exec -ti stack-mongo_mongo_1 /bin/bash
root@03e456ebc1a2:/# mongosh -u root -p
Enter password: *****
Current Mongosh Log ID: 659d3020ea5f53bd7c7d6ec2
Connecting to:      mongodb://<credentials>@127.0.0.1:27017/?directConnectio
n=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.1.1
Using MongoDB:      7.0.4
Using Mongosh:      2.1.1

For mongosh info see: https://docs.mongodb.com/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2024-01-09T10:02:07.202+00:00: Using the XFS filesystem is strongly recommend
ed with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodno
tes-filesystem
  2024-01-09T10:02:07.861+00:00: vm.max_map_count is too low
-----

test> use('GestionAcademica');
switched to db GestionAcademica
GestionAcademica>
```

Figura 2: Ejemplo de conexión en modo interactivo.

1.2.3.1 Acceso al shell de MongoDB

1. Abrir una terminal dentro del contenedor MongoDB:

```
1 docker exec -ti stack-mongo_mongo_1 /bin/bash
```

2. Iniciar el shell interactivo:

```
1 mongosh -u root -p
```

Se solicitará la contraseña configurada en el `docker-compose.yml`.

3. Seleccionar una base de datos:

```
1 use('GestionAcademica')
```

Si la base de datos no existe, MongoDB la creará automáticamente cuando se inserte el primer documento.

1.2.4 Operaciones CRUD básicas en MongoDB

Vamos a suponer que tenemos el siguiente diagrama UML:

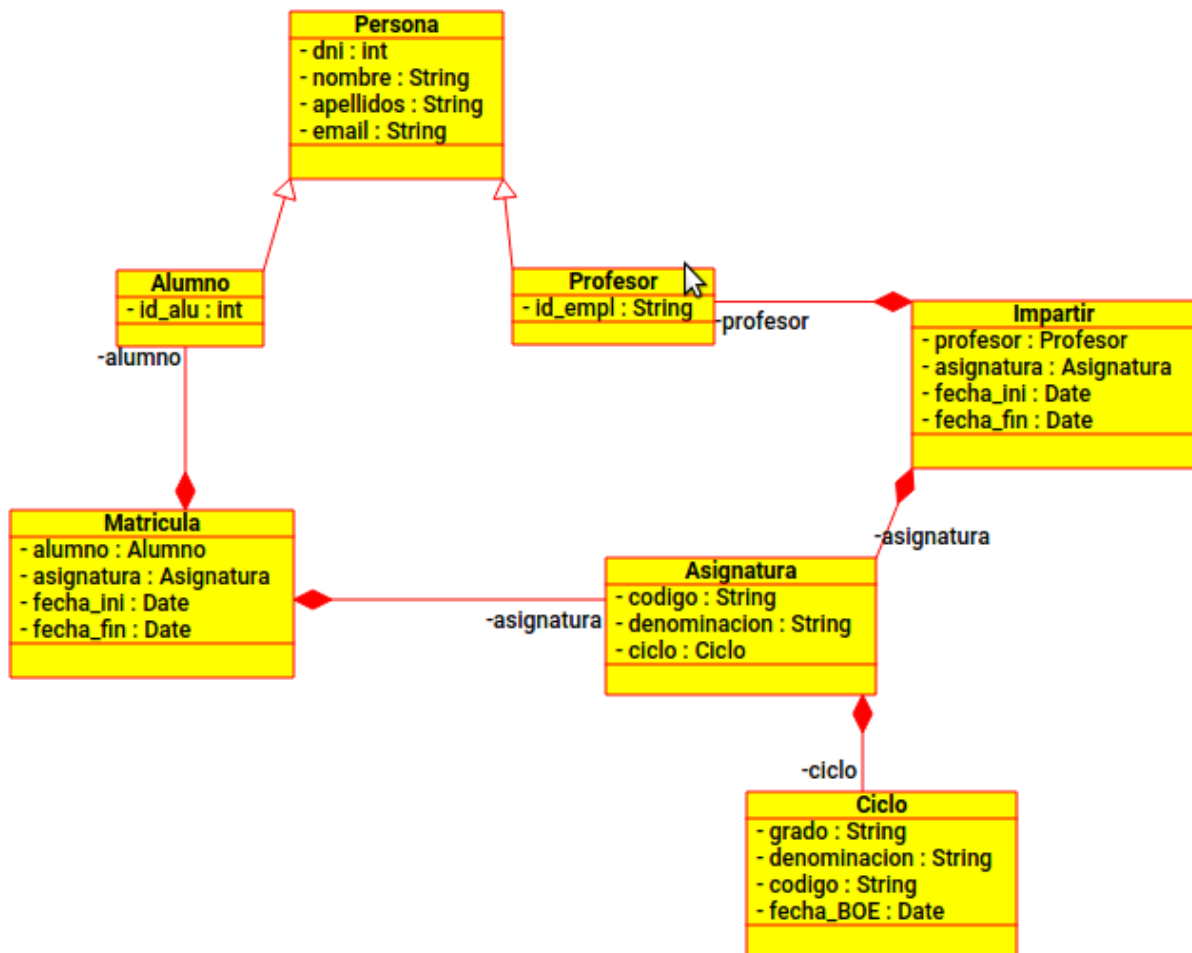


Figura 3: Diagrama UML Gestión Académica

1.2.4.1 Crear un documento (Create) Para insertar un documento en una colección llamada `alumnos`:

```
1 db.alumnos.insertOne({
2   nombre: "Juan Perez",
3   edad: 20,
4   carrera: "Ingeniería Informática"
5 })
```

MongoDB crea automáticamente la colección si no existe y asigna un identificador `_id`.

1.2.4.2 Leer documentos (Read) Para recuperar todos los documentos de la colección:

```
1 db.alumnos.find()
```

1.2.4.3 Actualizar un documento (Update) Para modificar un campo concreto se utiliza el operador `$set`:

```
1 db.alumnos.updateOne(  
2   { nombre: "Juan Perez" },  
3   { $set: { carrera: "Ingeniería Eléctrica" } }  
4 )
```

1.2.4.4 Eliminar un documento (Delete) Para eliminar un documento:

```
1 db.alumnos.deleteOne({ nombre: "Juan Perez" })
```

1.2.4.5 Consultas con condiciones y arrays Buscar profesores por nombre:

```
1 db.profesor.find({ nombre: "Juan" }).pretty()
```

Buscar profesores con asignaturas de más de 4 horas:

```
1 db.profesor.find(  
2   asignaturas: {  
3     $elemMatch: {  
4       horas: { $gt: 4 }  
5     }  
6   }  
7 })
```

1.2.5 Colección de ejemplo: estudiantes

Se utilizará una colección ficticia para practicar consultas:

```
1 db.estudiantes.insertMany([  
2   { nombre: "Ana García", edad: 22, carrera: "Biología", semestre: 5,  
3     promedio: 8.5 },  
4   { nombre: "Carlos López", edad: 21, carrera: "Ingeniería Civil",  
5     semestre: 4, promedio: 7.2 },  
6   { nombre: "María Torres", edad: 20, carrera: "Psicología", semestre:  
7     3, promedio: 9.0 },  
8   { nombre: "Juan Rodríguez", edad: 23, carrera: "Historia", semestre:  
9     6, promedio: 7.8 },  
10  ])
```

```
6   { nombre: "Elena Pérez", edad: 19, carrera: "Matemáticas", semestre:  
7     2, promedio: 9.5 }  
   ])
```

1.2.5.1 Ejemplos de consultas

- Obtener todos los estudiantes:

```
1 db.estudiantes.find({})
```

- Filtrar por carrera:

```
1 db.estudiantes.find({ carrera: "Biología" })
```

- Estudiantes mayores de 21 años:

```
1 db.estudiantes.find({ edad: { $gt: 21 } })
```

- Promedio mayor o igual a 8:

```
1 db.estudiantes.find({ promedio: { $gte: 8.0 } })
```

- Ordenar por promedio descendente:

```
1 db.estudiantes.find().sort({ promedio: -1 })
```

- Limitar resultados:

```
1 db.estudiantes.find().limit(3)
```

MongoDB ofrece una amplia variedad de operadores y opciones para realizar consultas avanzadas. Se recomienda consultar la documentación oficial para profundizar en estos aspectos.

1.3 Introducción: REST + MongoDB + SPA ligera

En este capítulo se aborda el desarrollo de una aplicación web completa basada en una **arquitectura cliente-servidor**, utilizando tecnologías ampliamente empleadas en el desarrollo de aplicaciones modernas. El objetivo es integrar un **backend REST** desarrollado con Spring Boot y MongoDB con un **frontend ligero** construido con HTML5, Bootstrap y JavaScript, siguiendo el modelo de **Single Page Application (SPA) simple**, sin necesidad de herramientas de construcción complejas.

1.3.1 Objetivo general del capítulo

El objetivo principal es que el alumnado sea capaz de **diseñar, implementar, probar y documentar** una aplicación que:

- Exponga una **API REST** para la gestión de datos.
- Utilice una **base de datos orientada a documentos (MongoDB)** para almacenar la información.
- Consuma dicha API desde un cliente web mediante **peticiones HTTP asíncronas**.
- Aplique buenas prácticas básicas de diseño, validación y manejo de errores.

Este enfoque permite evaluar de forma directa y práctica los criterios de evaluación relacionados con el acceso a datos y el desarrollo de aplicaciones multicapa.

1.3.2 Arquitectura de la aplicación

La aplicación se estructura en tres capas claramente diferenciadas:

- **Cliente (frontend)** Implementado con HTML5, Bootstrap y JavaScript (jQuery). Se encarga de la interacción con el usuario y del consumo de la API REST mediante peticiones AJAX.
- **Servidor (backend)** Desarrollado con Spring Boot, expone una API REST que gestiona la lógica de negocio y el acceso a la base de datos. Se apoya en Spring Data MongoDB para la persistencia de la información.
- **Base de datos** MongoDB almacena la información en forma de documentos BSON, permitiendo un modelo flexible que se adapta bien a los requisitos del dominio de reservas.

La comunicación entre cliente y servidor se realiza exclusivamente mediante **JSON sobre HTTP**, lo que desacopla ambas capas y facilita su mantenimiento y evolución.

1.3.3 Por qué REST, MongoDB y una SPA ligera

La combinación de estas tecnologías no es casual y responde a criterios técnicos y pedagógicos:

- **REST** permite definir una interfaz clara, estándar y fácilmente comprobable para el acceso a los datos, favoreciendo la separación de responsabilidades.
- **MongoDB**, como base de datos orientada a documentos, facilita el trabajo con estructuras de datos complejas y jerárquicas, alineándose bien con el modelo de reservas planteado.
- Una **SPA ligera**, sin frameworks pesados ni procesos de compilación, permite centrarse en los conceptos fundamentales del consumo de APIs y la gestión del estado en el cliente, reduciendo la carga cognitiva inicial.

Este enfoque resulta especialmente adecuado para comprender las diferencias entre bases de datos relacionales y documentales, así como para introducir patrones de desarrollo habituales en aplicaciones web actuales.

1.3.4 Caso práctico: gestión de reservas de pistas deportivas

A lo largo del capítulo se desarrollará una aplicación que permite:

- Gestionar **instalaciones deportivas**.
- Definir **horarios** asociados a dichas instalaciones, almacenando un “snapshot” de la información relevante.
- Registrar **usuarios**.
- Crear, consultar, modificar y cancelar **reservas**, combinando información embebida (horario e instalación) con referencias a otros documentos (usuario).

Para entender mejor la aplicación veamos el siguiente caso de uso:

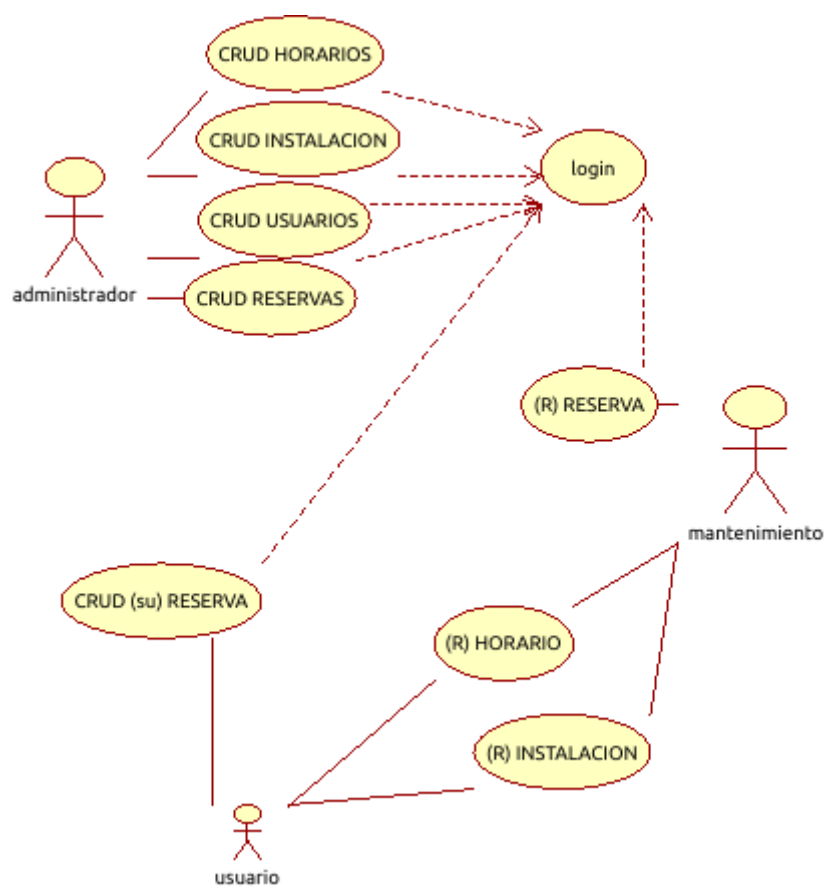


Figura 4: Diagrama de caso de uso de gestión de pistas deportivas

El diagrama UML que seguiremos es este:

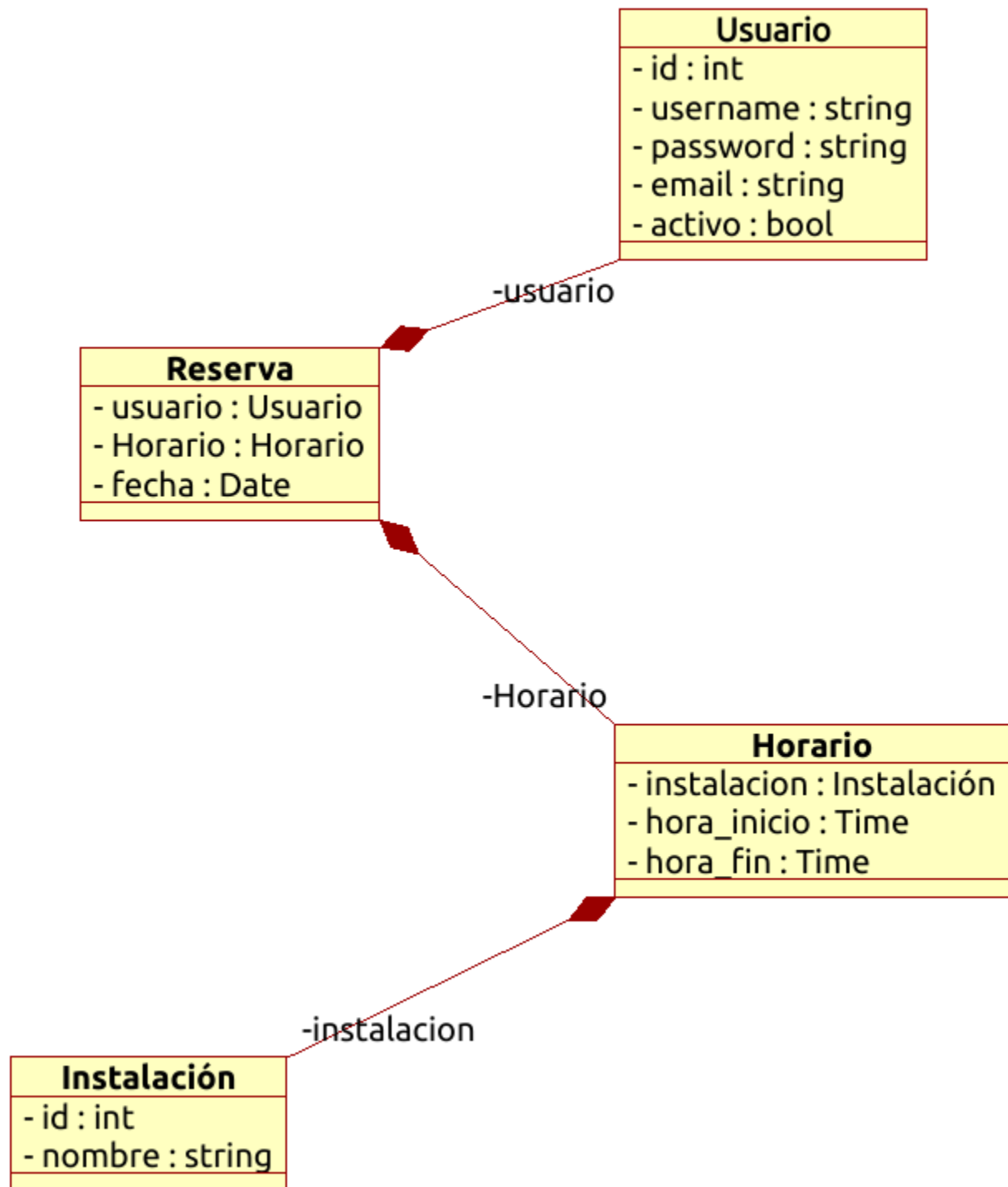


Figura 5: Diagrama UML de Gestión de Pistas Deportivas

Este modelo ha sido elegido porque:

- Presenta **objetos simples y estructurados**, ideales para trabajar la persistencia en MongoDB.
- Requiere **consultas filtradas** (por fecha, usuario o instalación).
- Obliga a gestionar la **consistencia de los datos**, especialmente al evitar solapamientos de reservas, lo que introduce el concepto de transacción o control lógico de integridad.

1.4 MongoDB

Para trabajar con MongoDB, no basta con instalar el servidor; es fundamental asegurar el acceso desde el primer momento y conocer las herramientas de conexión.

1.4.1 Instalación del Servidor y el Shell

Aunque nosotros vamos a trabajar con contenedores Docker para evitar perder el tiempo calentando el plato (ver siguiente apartado), te recordamos que también es posible y muy usada la instalación manual o incluso el uso en línea.

1.4.1.1 Instalación en Windows 11 (vía winget) Utilizaremos la terminal para automatizar la descarga de la versión Community y el cliente de comandos (`mongosh`).

1. Abre **PowerShell** como administrador.
2. Ejecuta:

```
1 winget install MongoDB.Server
2 winget install MongoDB.Shell
```

3. **Prueba de funcionamiento:** Abre una nueva terminal y escribe `mongosh --version`. Debería devolver la versión del shell (ej. 2.x.x).

1.4.1.2 En Ubuntu (Linux)

1. Importa la clave pública y añade el repositorio oficial:

```
1 sudo apt-get install gnupg curl
2 curl -fsSL https://www.mongodb.org/static/pgp/server-7.0.asc | sudo gpg
  -o /usr/share/keyrings/mongodb-server-7.0.gpg --dearmor
3 echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-
  server-7.0.gpg ] https://repo.mongodb.org/apt/ubuntu jammy/mongodb-
  org/7.0 multiverse" | sudo tee /etc/apt/sources.list.d/mongodb-org
  -7.0.list
4 sudo apt-get update
5 sudo apt-get install -y mongodb-org
```

2. Inicia el servicio: `sudo systemctl start mongod`.
3. **Prueba de funcionamiento:** Ejecuta `sudo systemctl status mongod` para verificar que el estado es “active (running)”.

1.4.2 Configuración del usuario Root (Primer inicio)

Por defecto, MongoDB permite la conexión sin contraseña la primera vez. Es crítico crear un administrador antes de habilitar la seguridad.

1. Entra al shell: `mongosh`
2. Cambia a la base de datos de administración:

```
1 use admin
```

3. **Crea el usuario root** (usaremos la clave común para las prácticas):

```
1 db.createUser({
2   user: "root",
3   pwd: "Secreto_123",
4   roles: [ { role: "root", db: "admin" } ]
5 })
```

4. **Activa la autenticación:**

- En **Windows**: Edita el archivo `C:\Program Files\MongoDB\Server\X.X\bin\mongod.cfg`.
- En **Ubuntu**: Edita `/etc/mongod.conf`.
- Busca la sección `security` y añade (ojo con la indentación de dos espacios):

```
1 security:
2   authorization: enabled
```

5. **Reinicia el servicio:**

- Windows: `Restart-Service mongod` (en PowerShell admin).
- Ubuntu: `sudo systemctl restart mongod`.

1.4.3 Cómo conectar (Pruebas de acceso)

Una vez activada la seguridad, si intentas entrar solo con `mongosh`, no tendrás permisos para ver datos. Debes usar la **Connection String**:

- **Por terminal:**

```
1 mongosh "mongodb://root:Secreto_123@localhost:27017/?authSource=admin"
```

- **Por GUI (MongoDB Compass)**: Si prefieres una interfaz visual, descarga **MongoDB Compass**. En la barra de conexión, pega la cadena anterior. Verás que aparecen las bases de datos `admin`, `config` y `local`.

1.4.4 Recuperación del Root (Si olvidas la clave)

Si se pierde el acceso, MongoDB tiene un “modo de emergencia” que permite entrar sin contraseña ignorando las tablas de usuarios:

1. **Detén el servidor:**

- `sudo systemctl stop mongod` (Linux) o detén el servicio en Windows.

2. **Inicia manualmente sin seguridad:** Abre una terminal y lanza el ejecutable con el parámetro `--noauth`:

```
1 mongod --dbpath /var/lib/mongodb --noauth ## (Ajusta la ruta según tu SO)
```

3. **Entra y cambia la clave:** En otra terminal, entra con `mongosh`, ve a `admin` y actualiza la contraseña:

```
1 use admin
2 db.changeUserPassword("root", "Secreto_123")
```

4. **Cierra todo y reinicia normalmente:** Mata el proceso manual y vuelve a iniciar el servicio de forma estándar. La seguridad volverá a estar activa con la nueva clave.

Recordad que `_id` es el identificador único obligatorio. Aunque usemos referencias por `_id`, en MongoDB es habitual duplicar información (como el nombre del alumno en la colección de matrículas) para evitar tener que consultar dos colecciones constantemente.

1.5 Modelado y Creación de la Base de Datos: Gestión Académica

En MongoDB, la unidad básica es el **documento** (un registro en formato BSON). Estos se agrupan en **colecciones**. A diferencia de las bases de datos relacionales, no forzamos una estructura rígida, lo que nos permite evolucionar el sistema fácilmente.

1.5.1 Estrategias de Relación: Embebido vs. Referencia

Esta es la decisión más importante en el diseño de MongoDB.

1.5.1.1 Documentos Embebidos (Denormalización) Consiste en guardar un documento dentro de otro como un subobjeto o un array.

- **Cuándo usarlo:** Cuando los datos se consultan siempre juntos (ej. los datos de contacto de un alumno).
- **Ventaja:** Mayor velocidad de lectura (una sola consulta obtiene todo).
- **Concepto de Snapshot (Foto):** En una matrícula, embebemos el nombre de la asignatura. Si el nombre de la asignatura cambia en el futuro, la matrícula del alumno conserva el nombre que tenía en ese momento. Es una **redundancia intencionada**.

1.5.1.2 Referencias (Normalización) Consiste en guardar el `_id` de un documento en otro campo.

- **Cuándo usarlo:** Cuando el documento referenciado cambia a menudo o es muy grande.
- **Analogía SQL:** Es lo más parecido a una **Foreign Key (Clave Foránea)**, pero MongoDB no impide que borres el destino (no hay integridad referencial automática).

1.5.2 Implementación Práctica en la Terminal

Primero, asegúrate de estar conectado con tu usuario root y selecciona la base de datos (se creará al insertar):

```
1 use gestion_academica
```

1.5.2.1 Paso 1: Creación de Profesores (insertOne) Insertamos un profesor que luego será referenciado. El campo `_id` se genera automáticamente como un **ObjectId** si no lo indicamos.

```
1 db.profesores.insertOne({
2   nombre: "Dra. García",
3   departamento: "LSI",
4   especialidad: "Bases de Datos",
5   fecha_incorporacion: new Date("2015-09-01")
6 })
```

1.5.2.2 Paso 2: Creación de Asignaturas (insertMany) Usamos `insertMany` para cargar el catálogo rápidamente.

```
1 db.asignaturas.insertMany([
2   { _id: "PROG1", nombre: "Programación I", credits: 6 },
3   { _id: "BD1", nombre: "Bases de Datos I", credits: 6 },
4   { _id: "ISO", nombre: "Introducción a los SO", credits: 6 }
5 ])
```

1.5.2.3 Paso 3: El Alumno (Modelo Híbrido: Embebido + Referencia) Este documento es el más complejo. Contiene:

1. **Datos simples:** nombre y expediente.
2. **Documento embebido:** `contacto`.
3. **Array de referencias con Snapshot:** `matriculas`.

```
1 db.alumnos.insertOne({
2   nombre: "Ana López",
3   expediente: "AL2024",
4   contacto: {
5     email: "ana@univ.es",
6     tlf: "600111222",
7     direccion: "Avda. de la Universidad, 4"
8   },
9   matriculas: [
10    {
11      asignatura_id: "BD1",           // REFERENCIA a la asignatura
12      nombre_asig: "Bases de Datos I", // SNAPSHOT (Redundancia)
13      nota: 8.5,
14      fecha: new Date()
15    },
16    {
17      asignatura_id: "PROG1",
18      nombre_asig: "Programación I",
19      nota: 9.0,
20      fecha: new Date()
21    }
22  ]
23 })
```

1.5.3 Resumen de diferencias

Concepto Relacional (SQL)	Concepto MongoDB	Notas de Ingeniería
Fila	Documento	El documento puede ser jerárquico (objetos anidados).
Tabla	Colección	No tienen por qué tener todos los mismos campos.
Clave Primaria	<code>_id</code> (ObjectId)	12 bytes: timestamp + id proceso + contador.
JOIN / Foreign Key	Referencia o Embebido	En MongoDB preferimos duplicar datos (embeber) para evitar JOINS costosos.

Importante: La contraseña de root es **Secreto_123**. No la olvidéis para poder entrar en el shell (`mongosh -u root -p Secreto_123 --authSource admin`).

1.6 Manipulación de Datos (Operaciones CRUD)

En MongoDB, el CRUD no se realiza mediante sentencias de texto como en SQL, sino mediante métodos de JavaScript ejecutados sobre el objeto `db`.

1.6.0.1 Lectura (Read) con Proyecciones y Operadores La lectura no solo es `find()`. Es fundamental aprender a filtrar y a elegir qué campos queremos ver (proyección).

- **Uso de Operadores Lógicos y de Comparación:** No usamos `>`, `<` o `!=`. Usamos operadores BSON: `$gt` (greater than), `$lt` (less than), `$in` (contenido en), `$ne` (not equal).

```
1 // Buscar alumnos con una nota mayor o igual a 9 en alguna asignatura
2 db.alumnos.find({ "matriculas.nota": { $gte: 9 } })
3
4 // Buscar profesores de "LSI" o "Arquitectura" usando $in
5 db.profesores.find({ departamento: { $in: ["LSI", "Arquitectura"] } })
```

- **Proyecciones (Elegir columnas):** El segundo parámetro de `find` indica qué campos queremos (1) y cuáles no (0).

```
1 // Solo queremos el nombre y el expediente (el _id sale por defecto)
2 db.alumnos.find({ "contacto.tlf": { $exists: true } }, { nombre: 1,
   expediente: 1 })
```

1.6.0.2 Actualizaciones Complejas (Update) A diferencia de SQL, donde haces un `SET` de toda la fila, en MongoDB usamos **operadores de actualización** para no machacar el documento entero.

- **\$set:** Modifica un campo o lo crea si no existe.
- **\$inc:** Incrementa un valor numérico (ideal para contadores).
- **\$push:** Añade un elemento a un array (fundamental en nuestra colección de alumnos).

```
1 // Añadir una nueva matrícula al array de un alumno sin borrar las
   anteriores
2 db.alumnos.updateOne(
3   { expediente: "AL2024" },
4   {
5     $push: {
6       matriculas: { asignatura_id: "ISO", nombre_asig: "Sistemas
   Operativos", nota: 7 }
7     }
8   }
9 )
10
11 // Cambiar el nombre del departamento de un profesor
```

```
12 db.profesores.updateMany(  
13   { departamento: "LSI" },  
14   { $set: { departamento: "Lenguajes y Sistemas" } }  
15 )
```

1.6.0.3 Borrado (Delete) El borrado es definitivo. Se recomienda siempre hacer un `find` previo con el mismo filtro para estar seguros de qué vamos a borrar.

```
1 // Borrar alumnos que no tengan ninguna matrícula (array vacío)  
2 db.alumnos.deleteMany({ matriculas: { $size: 0 } })
```

1.7 Consultas Complejas: El Aggregation Framework

Cuando necesitamos realizar cálculos (medias, sumas) o unir colecciones, el `find()` se queda corto. Usamos el **Pipeline de Agregación**, que funciona como una “cadena de montaje”: el resultado de una etapa entra en la siguiente.

1.7.0.1 El operador `$lookup` (El “JOIN” de MongoDB) Como vimos en el tema anterior, MongoDB usa referencias. Si queremos obtener una lista de alumnos que incluya los datos detallados del profesor que tienen asignado como tutor, haríamos lo siguiente:

```
1 db.alumnos.aggregate([  
2   {  
3     $lookup: {  
4       from: "profesores",           // Colección con la que unimos  
5       localField: "tutor_id",       // Campo en 'alumnos' (la  
6                                     referencia)  
7       foreignField: "_id",          // Campo en 'profesores'  
8       as: "info_tutor"              // Nombre del array donde se  
9                                     guardará el resultado  
10    }  
11  }  
12 ])
```

1.7.0.2 Agrupación y Cálculos (`$group` y `$unwind`) Si queremos calcular la **nota media de cada alumno**, tenemos un problema: las notas están dentro de un array. Primero debemos “deshacer” el array.

1. **`$unwind`**: Separa el array. Si un alumno tiene 3 matrículas, genera 3 documentos temporales (uno por matrícula).
2. **`$group`**: Agrupa y calcula.


```
1 db.alumnos.aggregate([
2   { $unwind: "$matriculas" }, // "Explotamos" el array de matrículas
3   {
4     $group: {
5       _id: "$nombre",           // Agrupamos por el nombre del
6       notaMedia: { $avg: "$matriculas.nota" }, // Calculamos la media
7       totalAsignaturas: { $sum: 1 } // Contamos cuántas hay
8     }
9   },
10  { $match: { notaMedia: { $gte: 5 } } } // Filtramos solo los
    aprobados
11 ])
```

1.7.0.3 Resumen de Etapas Clave

Etapas	Función	Analogía SQL
\$match	Filtra documentos	WHERE / HAVING
\$project	Modifica o selecciona campos	SELECT
\$lookup	Une colecciones	LEFT OUTER JOIN
\$group	Agrupar y realiza cálculos	GROUP BY
\$sort	Ordena los resultados	ORDER BY
\$unwind	Descompone arrays para tratarlos individualmente	N/A (Específico de NoSQL)

Aquí tienes una **Cheat Sheet** (Hoja de trucos) diseñada específicamente para que tus alumnos la tengan abierta durante las prácticas de Gestión Académica. Es un resumen compacto de todo lo que hemos visto.

1.7.1 MongoDB Cheat Sheet: Gestión Académica

1.7.1.1 Conexión y Seguridad

- **Conexión básica:** `mongosh "mongodb://root:Secreto_123@localhost:27017/?authSource=admin"`
- **Clave común:** `Secreto_123`
- **Crear usuario (en DB admin):** `db.createUser({ user: "root", pwd: "Secreto_123", roles: ["root"] })`

1.7.1.2 Estructura y Navegación

- **Ver bases de datos:** `show dbs`
- **Usar/Crear base de datos:** `use gestion_academica`
- **Ver colecciones:** `show collections`

1.7.1.3 Inserción (Create)

- **Un documento:** `db.alumnos.insertOne({ nombre: "Ana", expediente: "A1" })`
- **Varios documentos:** `db.profesores.insertMany([ { nombre: "Dr. X"}, { nombre: "Dra. Y"} ])`

1.7.1.4 Consultas (Read)

- **Todo:** `db.alumnos.find()`
- **Filtrar por campo:** `db.alumnos.find({ expediente: "A1" })`
- **Operadores de comparación:**
 - `$gt` / `$gte`: Mayor / Mayor o igual.
 - `$lt` / `$lte`: Menor / Menor o igual.
 - `$in`: Dentro de un array (ej: `{ dep: { $in: ["LSI", "MAT"] } }`).
- **Proyección (elegir campos):** `db.alumnos.find({}, { nombre: 1, _id: 0 })` (1 muestra, 0 oculta).

1.7.1.5 Actualización (Update)

- **Modificar campo:** `db.alumnos.updateOne({ _id: 1 }, { $set: { nombre: "Nuevo" } })`
- **Incrementar valor:** `db.alumnos.updateOne({ _id: 1 }, { $inc: { edad: 1 } })`
- **Añadir a un Array:** `db.alumnos.updateOne({ _id: 1 }, { $push: { matriculas: { asig: "BD" } } })`

1.7.1.6 Agregación (Consultas Avanzadas)

- **`$match`:** Filtrar (como el WHERE).

- **\$group**: Agrupar (como GROUP BY). Usar con acumuladores: { \$avg: "\$nota"}, { \$sum : 1 }.
- **\$unwind**: “Abrir” un array para procesar sus elementos uno a uno.
- **\$lookup**: Unir colecciones (como el JOIN).

1.7.2 Documentación Oficial

Para profundizar o resolver dudas técnicas específicas, la documentación de MongoDB es una de las mejores en el mundo del software:

Manual de MongoDB (Documentación Oficial)

Te recomiendo especialmente estas secciones para los alumnos:

- Operaciones CRUD: Para ejemplos de lectura y escritura.
- Aggregation Framework: Para entender el pipeline de datos.
- Operadores de consulta: Un listado completo de todos los \$gt, \$in, etc.

1.8 Ejercicio Práctico: Sistema de Gestión de Escuela de Ingeniería

Contexto: Vamos a implementar el backend de datos para una escuela. Necesitamos gestionar **Profesores**, **Alumnos** y sus **Matrículas**.

1.8.1 Fase 1: Seguridad y Conexión

1. Inicia el shell de MongoDB (`mongosh`) como administrador utilizando el usuario `root` y la clave `Secreto_123`.
2. Crea una base de datos llamada `escuela_ingenieria`.
3. Crea un usuario específico para esta base de datos llamado `gestor_escuela` con el rol `dbAdmin` y `readWrite`.

1.8.2 Fase 2: Modelado e Inserción

Crea las siguientes colecciones siguiendo las instrucciones de diseño documental:

1.8.2.1 1. Colección profesores Inserta al menos dos profesores.

- **Profesor A:** `nombre: "Miguel Cañas"`, `departamento: "Arquitectura de Computadores"`, `despacho: 102`.
- **Profesor B:** `nombre: "Isabel Ruiz"`, `departamento: "LSI"`, `despacho: 205`.

1.8.2.2 2. Colección alumnos Inserta un alumno con el siguiente modelo híbrido:

- **Datos básicos:** Nombre y expediente.
- **Documento Embebido:** `contacto` (email y ciudad).
- **Array de Referencias con Snapshot:** El campo `inscripciones` debe contener el `id_profesor` (referencia) y el `nombre_departamento` (snapshot/foto en ese momento).

Script de ayuda:

```
1 db.alumnos.insertOne({
2   nombre: "Juan Pérez",
3   expediente: "JP2026",
4   contacto: { email: "juan@correo.es", ciudad: "Jaén" },
5   inscripciones: [
6     {
7       profesor_id: ObjectId("ID_DE_MIGUEL"), // Usa el ID real generado
7       arriba
8       departamento_snapshot: "Arquitectura de Computadores",
```

```
9      fecha: new Date()  
10    }  
11  ]  
12 })
```

1.8.3 Fase 3: Manipulación (CRUD)

Realiza las siguientes operaciones:

1. **Actualización:** Cambia el despacho del profesor “Miguel Cañas” al 108 usando `$set`.
2. **Inserción en Array:** Añade una nueva inscripción al alumno “Juan Pérez” con la profesora “Isabel Ruiz” usando `$push`.
3. **Consulta de Filtro:** Busca todos los alumnos cuya ciudad en el documento embebido `contacto` sea “Jaén”.

1.8.4 Fase 4: Desafío de Agregación (Reporting)

Utiliza el **Aggregation Framework** para generar un informe:

1. **Tarea:** Muestra el nombre del alumno y, mediante un `$lookup`, une la información de sus profesores para mostrar el nombre del profesor y su departamento actual.
2. **Pregunta de reflexión:** Si el profesor Miguel cambia de departamento hoy, ¿qué verás en el campo `departamento_snapshot` de la inscripción del alumno y qué verás en el documento unido mediante `$lookup`?

1.9 Preparación del entorno y dependencias

En este apartado se prepara un entorno reproducible basado en **Dev Containers** para minimizar el tiempo de arranque y evitar problemas de configuración entre equipos. El objetivo es disponer, desde el primer momento, de:

- Un contenedor de desarrollo con **Java + Maven** para el backend.
- Un servicio **MongoDB** como base de datos documental.
- Un servicio **Mongo Express** (interfaz web) para inspección y administración.
- Un archivo **.env en la raíz del proyecto** con variables compartidas tanto por Docker Compose como por Spring Boot.
- Un proyecto **Spring Boot (Maven)** con los *starters* necesarios y configuración en **YAML**.

1.9.1 Requisitos previos

- Docker y Docker Compose instalados en el equipo.
- Visual Studio Code con la extensión **Dev Containers**.
- Git (recomendado) para versionar el proyecto.

1.9.2 Estructura de carpetas recomendada

Se propone la siguiente estructura mínima:

```
1 reservas-pistas/  
2   .devcontainer/  
3     devcontainer.json  
4     docker-compose.yml  
5   src/  
6   pom.xml  
7   .env  
8   application.yml (en src/main/resources)
```

La carpeta **.devcontainer/** contiene toda la configuración necesaria para levantar el entorno completo (IDE + servicios) con un único comando desde VS Code.

1.9.3 Creación del archivo .env (variables compartidas)

En la raíz del proyecto se crea un archivo **.env**. Este archivo centraliza configuración sensible y de entorno (usuario, password, nombre de base de datos, puertos).

Ejemplo recomendado:

```
1 # MongoDB (servicio Docker)
2 MONGO_PORT=27017
3 MONGO_INITDB_ROOT_USERNAME=root
4 MONGO_INITDB_ROOT_PASSWORD=83uddjfp0cmMD
5 MONGO_DB=reservas_db
6
7 # Mongo Express
8 MONGO_EXPRESS_PORT=8081
9 ME_CONFIG_BASICAUTH_USERNAME=mongo
10 ME_CONFIG_BASICAUTH_PASSWORD=83uddjfp0cmMD
```

Buenas prácticas:

- En un entorno real, este archivo no debería subirse al repositorio (usar `.gitignore`).
- En contexto educativo, puede incluirse si se pretende homogeneidad, pero conviene explicitarlo.

1.9.4 Preparación del Dev Container

La idea es que VS Code se conecte a un contenedor de desarrollo (“workspace”) y, a la vez, levante los servicios auxiliares (MongoDB y Mongo Express) mediante Docker Compose.

1.9.4.1 `.devcontainer/docker-compose.yml` Se define el contenedor de desarrollo y los servicios. Este archivo usa variables del `.env`.

```
1 version: "3.8"
2
3 services:
4   app:
5     image: mcr.microsoft.com/devcontainers/java:21
6     volumes:
7       - ../workspace:cached
8     working_dir: /workspace
9     command: sleep infinity
10    depends_on:
11      - mongo
12      - mongo-express
13    environment:
14      # Variables disponibles también dentro del contenedor de
15      # desarrollo
16      MONGO_INITDB_ROOT_USERNAME: ${MONGO_INITDB_ROOT_USERNAME}
17      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_INITDB_ROOT_PASSWORD}
18      MONGO_DB: ${MONGO_DB}
19
20   mongo:
21     image: mongo:7
```

```

21     restart: "no"
22     environment:
23         MONGO_INITDB_ROOT_USERNAME: ${MONGO_INITDB_ROOT_USERNAME}
24         MONGO_INITDB_ROOT_PASSWORD: ${MONGO_INITDB_ROOT_PASSWORD}
25     ports:
26     - "${MONGO_PORT}:27017"
27     volumes:
28     - mongo_data:/data/db
29
30     mongo-express:
31         image: mongo-express:1
32         restart: "no"
33         depends_on:
34         - mongo
35         ports:
36         - "${MONGO_EXPRESS_PORT}:8081"
37         environment:
38             ME_CONFIG_BASICAUTH_USERNAME: ${ME_CONFIG_BASICAUTH_USERNAME}
39             ME_CONFIG_BASICAUTH_PASSWORD: ${ME_CONFIG_BASICAUTH_PASSWORD}
40             ME_CONFIG_MONGODB_ADMINUSERNAME: ${MONGO_INITDB_ROOT_USERNAME}
41             ME_CONFIG_MONGODB_ADMINPASSWORD: ${MONGO_INITDB_ROOT_PASSWORD}
42             ME_CONFIG_MONGODB_URL: mongodb://${MONGO_INITDB_ROOT_USERNAME}:${MONGO_INITDB_ROOT_PASSWORD}@mongo:27017/
43
44     volumes:
45     mongo_data:

```

Notas técnicas:

- Se fija MongoDB 7 por estabilidad y compatibilidad.
- Se añade volumen `mongo_data` para persistencia entre reinicios.
- El contenedor `app` no ejecuta la aplicación; simplemente proporciona el entorno de desarrollo.

1.9.4.2 .devcontainer/devcontainer.json Configura cómo VS Code se conecta al contenedor.

```

1  {
2      "name": "reservas-pistas-devcontainer",
3      "dockerComposeFile": "docker-compose.yml",
4      "service": "app",
5      "workspaceFolder": "/workspace",
6      "shutdownAction": "stopCompose",
7      "customizations": {
8          "vscode": {
9              "extensions": [
10                 "vscjava.vscode-java-pack",
11                 "vmware.vscode-spring-boot",
12                 "redhat.vscode-yaml"
13             ]

```



```
14     }  
15   }  
16 }
```

Con esto, al abrir el proyecto en VS Code:

1. Se elige “Reopen in Container”.
2. Se construye el entorno y se levantan los servicios.
3. MongoDB queda disponible desde el contenedor `app` por hostname `mongo:27017`.

1.9.5 Creación del proyecto Spring Boot (Maven)

El backend se implementará como un proyecto Maven estándar con Spring Boot.

1.9.5.1 Opción recomendada: Spring Initializr

1. Acceder a Spring Initializr (web) o usar integración en IDE.
2. Configurar:
 - Project: **Maven**
 - Language: **Java**
 - Spring Boot: versión estable
 - Group: `com.dam`
 - Artifact: `reservas`
 - Packaging: `jar`
 - Java: 21 (o 17 si el centro lo requiere)
3. Añadir dependencias (*starters*):
 - **Spring Web**
 - **Spring Data MongoDB**
 - **Validation**

Generar, descargar y descomprimir dentro del repositorio `reservas-pistas/`.

1.9.5.2 Alternativa: crear Maven y añadir starters manualmente

Si se parte de un Maven “vacío”:

1. Crear proyecto Maven:

```
1 mvn archetype:generate \  
2   -DgroupId=com.dam \  
3   -DartifactId=reservas \  
4   -DarchetypeArtifactId=maven-archetype-quickstart \  
5   -DinteractiveMode=false
```

2. Sustituir el `pom.xml` por uno basado en Spring Boot, incluyendo:

- Parent `spring-boot-starter-parent`
- Dependencias de `web`, `data-mongodb`, `validation`

En entornos docentes suele ser mejor usar Spring Initializr para evitar ruido.

1.9.6 Dependencias (starters) necesarias

En el `pom.xml` deben estar al menos:

- `spring-boot-starter-web`: API REST (controladores, JSON, HTTP).
- `spring-boot-starter-data-mongodb`: persistencia MongoDB.
- `spring-boot-starter-validation`: validación de DTOs con `@Valid`.

Ejemplo mínimo:

```
1 <dependencies>  
2   <dependency>  
3     <groupId>org.springframework.boot</groupId>  
4     <artifactId>spring-boot-starter-web</artifactId>  
5   </dependency>  
6  
7   <dependency>  
8     <groupId>org.springframework.boot</groupId>  
9     <artifactId>spring-boot-starter-data-mongodb</artifactId>  
10  </dependency>  
11  
12  <dependency>  
13    <groupId>org.springframework.boot</groupId>  
14    <artifactId>spring-boot-starter-validation</artifactId>  
15  </dependency>  
16 </dependencies>
```

1.9.7 Configuración en YAML (`application.yml`) usando variables de entorno

En Spring Boot se usará `application.yml` en `src/main/resources/`.

La idea es **leer las variables del `.env`**. En Docker Compose, el `.env` se carga automáticamente para los servicios. Para Spring, hay dos estrategias:

- Ejecutar Spring desde el contenedor `app` y pasarle variables de entorno (recomendado).
- Cargar `.env` con una librería adicional (opcional, no necesario si se usa Compose).

1.9.7.1 `src/main/resources/application.yml`

```
1 server:
2   port: 8080
3
4 spring:
5   data:
6     mongodb:
7       uri: mongodb://${MONGO_INITDB_ROOT_USERNAME}:${MONGO_INITDB_ROOT_PASSWORD}@mongo:27017/${MONGO_DB}?authSource=admin
8
9 logging:
10  level:
11    org.springframework.web: INFO
```

Notas:

- `mongo` es el hostname del servicio dentro de Docker Compose.
- `authSource=admin` es necesario cuando se autentica con usuario root.
- La base de datos que usará la aplicación será `${MONGO_DB}`.

1.9.8 Verificación del entorno

Una vez abierto el proyecto en Dev Container:

1. Comprobar que Mongo está levantado:

```
1 docker ps
```

2. Acceder a Mongo Express en el host:

- `http://localhost:8081`
- Usuario/contraseña definidos en `.env`

3. Probar conexión desde el contenedor de desarrollo:

```
1 mongosh "mongodb://root:83uddjfp0cmMD@mongo:27017/?authSource=admin"
```

1.9.9 Nota sobre “mongoengine”

MongoEngine es un ODM típico de Python (Flask/Django), no forma parte del stack Java/Spring. En Spring Boot, el equivalente conceptual es:

- **Spring Data MongoDB** (repositorios, mapeo de documentos, consultas derivadas)
- Alternativamente [MongoTemplate](#) para consultas más personalizadas

En este capítulo se utilizará **Spring Data MongoDB** por su alineación con el enfoque REST y por simplificar el CRUD y las consultas.

1.9.10 Archivo .env de ejemplo

El archivo `.env`, situado en la raíz del proyecto, centraliza las variables de entorno utilizadas tanto por Docker Compose como por Spring Boot. De este modo se evita la duplicación de configuración y se facilita la modificación del entorno sin tocar el código.

Ejemplo completo de `.env`:

```
1 # =====
2 # MongoDB
3 # =====
4 MONGO_PORT=27017
5 MONGO_INITDB_ROOT_USERNAME=root
6 MONGO_INITDB_ROOT_PASSWORD=83uddjfp0cmMD
7 MONGO_DB=reservas_db
8
9 # =====
10 # Mongo Express
11 # =====
12 MONGO_EXPRESS_PORT=8081
13 ME_CONFIG_BASICAUTH_USERNAME=mongo
14 ME_CONFIG_BASICAUTH_PASSWORD=83uddjfp0cmMD
```

Notas importantes:

- En un entorno profesional, este archivo **no debe versionarse**.
- En el contexto educativo puede compartirse para facilitar el arranque, pero debe explicarse el motivo y las implicaciones.
- Todas las variables definidas aquí están disponibles para los contenedores definidos en Docker Compose.

1.9.11 Archivo `.gitignore` recomendado

Para evitar subir al repositorio archivos generados automáticamente o que contengan información sensible, se recomienda incluir al menos las siguientes entradas en el archivo `.gitignore`:

```
1 # Variables de entorno
2 .env
3
4 # Directorios generados por Maven
5 target/
6
7 # Archivos de configuración del IDE
8 .idea/
9 *.iml
10 .vscode/
11
12 # Logs
13 *.log
```

Este `.gitignore` evita:

- La publicación accidental de credenciales.
- Conflictos entre distintos entornos de desarrollo.
- La inclusión de binarios o artefactos generados en tiempo de compilación.

1.9.12 Arranque de la aplicación Spring Boot desde el contenedor

Una vez que el proyecto se ha abierto en el **Dev Container**, el arranque de la aplicación se realiza desde el propio contenedor de desarrollo.

Desde el terminal integrado de VS Code:

```
1 ./mvnw spring-boot:run
```

Este comando:

- Utiliza el *Maven Wrapper* incluido en el proyecto.
- Arranca Spring Boot usando la configuración definida en `application.yml`.
- Conecta automáticamente con MongoDB usando las variables de entorno proporcionadas por Docker Compose.

Si todo es correcto, en la salida del log se podrá observar:

- Inicialización del contexto de Spring.
- Conexión correcta a MongoDB.
- Puerto HTTP del servidor web en estado “listening”.

1.9.13 Cheatsheet para troubleshooting

Ante problemas de arranque o conexión, conviene revisar de forma sistemática los siguientes puntos:

1.9.13.1 MongoDB no arranca o no responde

- Verificar que los contenedores están en ejecución:

```
1 docker ps
```

- Comprobar que el puerto definido en `.env` (`MONGO_PORT`) no está siendo usado por otro servicio.
- Revisar los logs del contenedor:

```
1 docker logs <id_contenedor_mongo>
```

1.9.13.2 Error de autenticación en MongoDB

- Confirmar que `MONGO_INITDB_ROOT_USERNAME` y `MONGO_INITDB_ROOT_PASSWORD` coinciden en:

- `.env`
- `docker-compose.yml`
- `application.yml`

- Asegurarse de que la URI de conexión incluye:

```
1 authSource=admin
```

1.9.13.3 Spring Boot no conecta con MongoDB

- Verificar que en `application.yml` se usa el hostname `mongo` y no `localhost`.
- Comprobar que Spring se está ejecutando **dentro del contenedor**, no en el host.
- Probar conexión manual desde el contenedor:

```
1 mongosh "mongodb://root:password@mongo:27017/?authSource=admin"
```

1.9.13.4 Mongo Express no es accesible

- Confirmar que el puerto `MONGO_EXPRESS_PORT` está correctamente expuesto.
- Acceder desde el navegador a:

```
1 http://localhost:8081
```

- Revisar credenciales de acceso web (basic auth).

1.9.13.5 Cambios en `.env` no surten efecto

- Recordar que Docker Compose **no recarga automáticamente** variables de entorno.
- Detener y volver a levantar los servicios:

```
1 docker-compose down
2 docker-compose up -d
```

1.10 Spring Data MongoDB: @Document, repositorios y consultas

En este apartado se prepara la capa de persistencia de la aplicación de **reservas de pistas deportivas** utilizando **Spring Data MongoDB**. El objetivo es dejar definidos:

- Los **documentos** (clases de dominio) mapeados a MongoDB.
- Los **repositorios** (`MongoRepository`) para operaciones CRUD.
- Un conjunto inicial de **consultas** típicas que se utilizarán más adelante en la API REST.

En el siguiente apartado se construirán los **DTOs** y los **endpoints**, por lo que aquí se evita exponer directamente las entidades como contrato de la API.

1.10.1 Dependencias necesarias

Este apartado asume que el proyecto ya incluye el *starter*:

- `spring-boot-starter-data-mongodb`

y que la conexión está configurada en `application.yml` con la URI construida mediante variables de entorno.

1.10.2 Diseño del modelo y decisiones de persistencia

Partimos del diagrama proporcionado, con estas decisiones explícitas:

- `Instalacion` es un **documento independiente** (colección `instalaciones`).
- `Usuario` es un **documento independiente** (colección `usuarios`).
- `Reserva` es un **documento independiente** (colección `reservas`).
- `Reserva` **embebe** `Horario`.
- `Horario` **embebe** un “snapshot” de la instalación (en vez de referenciarla directamente).
- `Reserva` **referencia** a `Usuario` mediante `usuarioId` (String que representa el `_id`).

Esto permite:

- Consultar reservas sin necesidad de joins.
- Mantener en la reserva los datos de instalación tal como estaban en el momento de la reserva.
- Evitar duplicar datos de usuario en cada reserva.

1.10.3 Paquetes recomendados

Para mantener orden y escalabilidad:


```
1 com.dam.reservas
2     model
3     repo
```

1.10.4 1) Documentos y embebidos

1.10.4.1 model/Instalacion.java

```
1 package com.dam.reservas.model;
2
3 import jakarta.validation.constraints.NotBlank;
4 import org.springframework.data.annotation.Id;
5 import org.springframework.data.mongodb.core.mapping.Document;
6
7 @Document(collection = "instalaciones")
8 public class Instalacion {
9
10     @Id
11     private String id;
12
13     @NotBlank(message = "nombre es obligatorio")
14     private String nombre;
15
16     @NotBlank(message = "direccion es obligatoria")
17     private String direccion;
18
19     @NotBlank(message = "ciudad es obligatoria")
20     private String ciudad;
21
22     public Instalacion() {}
23
24     public Instalacion(String nombre, String direccion, String ciudad) {
25         this.nombre = nombre;
26         this.direccion = direccion;
27         this.ciudad = ciudad;
28     }
29
30     public String getId() { return id; }
31     public void setId(String id) { this.id = id; }
32
33     public String getNombre() { return nombre; }
34     public void setNombre(String nombre) { this.nombre = nombre; }
35
36     public String getDireccion() { return direccion; }
37     public void setDireccion(String direccion) { this.direccion =
38         direccion; }
39
40     public String getCiudad() { return ciudad; }
41     public void setCiudad(String ciudad) { this.ciudad = ciudad; }
42 }
```

1.10.4.2 model/Usuario.java

```
1 package com.dam.reservas.model;
2
3 import jakarta.validation.constraints.Email;
4 import jakarta.validation.constraints.NotBlank;
5 import org.springframework.data.annotation.Id;
6 import org.springframework.data.mongodb.core.mapping.Document;
7
8 @Document(collection = "usuarios")
9 public class Usuario {
10
11     @Id
12     private String id;
13
14     @NotBlank(message = "nombre es obligatorio")
15     private String nombre;
16
17     @NotBlank(message = "email es obligatorio")
18     @Email(message = "email no válido")
19     private String email;
20
21     public Usuario() {}
22
23     public Usuario(String nombre, String email) {
24         this.nombre = nombre;
25         this.email = email;
26     }
27
28     public String getId() { return id; }
29     public void setId(String id) { this.id = id; }
30
31     public String getNombre() { return nombre; }
32     public void setNombre(String nombre) { this.nombre = nombre; }
33
34     public String getEmail() { return email; }
35     public void setEmail(String email) { this.email = email; }
36 }
```

1.10.4.3 model/InstalacionSnapshot.java (embebido) Este subdocumento es la “foto” de la instalación incluida dentro de la reserva, cumpliendo la composición del diagrama.

```
1 package com.dam.reservas.model;
2
3 import jakarta.validation.constraints.NotBlank;
4
5 public class InstalacionSnapshot {
6
7     @NotBlank(message = "instalacionId es obligatorio")
8     private String instalacionId;
```

```
9
10 @NotBlank(message = "nombre es obligatorio")
11 private String nombre;
12
13 @NotBlank(message = "direccion es obligatoria")
14 private String direccion;
15
16 @NotBlank(message = "ciudad es obligatoria")
17 private String ciudad;
18
19 public InstalacionSnapshot() {}
20
21 public InstalacionSnapshot(String instalacionId, String nombre,
22     String direccion, String ciudad) {
23     this.instalacionId = instalacionId;
24     this.nombre = nombre;
25     this.direccion = direccion;
26     this.ciudad = ciudad;
27 }
28
29 public String getInstalacionId() { return instalacionId; }
30 public void setInstalacionId(String instalacionId) { this.
31     instalacionId = instalacionId; }
32
33 public String getNombre() { return nombre; }
34 public void setNombre(String nombre) { this.nombre = nombre; }
35
36 public String getDireccion() { return direccion; }
37 public void setDireccion(String direccion) { this.direccion =
38     direccion; }
39
40 public String getCiudad() { return ciudad; }
41 public void setCiudad(String ciudad) { this.ciudad = ciudad; }
42 }
```

1.10.4.4 model/Horario.java (embebido) Se define como objeto embebido, no como documento. Para facilitar consultas y validación, se modela con tipos Java adecuados:

- `LocalDate` para el día.
- `LocalTime` para horas.

```
1 package com.dam.reservas.model;
2
3 import jakarta.validation.Valid;
4 import jakarta.validation.constraints.NotNull;
5
6 import java.time.LocalDate;
7 import java.time.LocalTime;
8
```

```
9 public class Horario {
10
11     @NotNull(message = "dia es obligatorio")
12     private LocalDate dia;
13
14     @NotNull(message = "horaInicio es obligatoria")
15     private LocalTime horaInicio;
16
17     @NotNull(message = "horaFin es obligatoria")
18     private LocalTime horaFin;
19
20     @NotNull(message = "instalacionSnapshot es obligatorio")
21     @Valid
22     private InstalacionSnapshot instalacionSnapshot;
23
24     public Horario() {}
25
26     public Horario(LocalDate dia, LocalTime horaInicio, LocalTime horaFin
27         , InstalacionSnapshot instalacionSnapshot) {
28         this.dia = dia;
29         this.horaInicio = horaInicio;
30         this.horaFin = horaFin;
31         this.instalacionSnapshot = instalacionSnapshot;
32     }
33
34     public LocalDate getDia() { return dia; }
35     public void setDia(LocalDate dia) { this.dia = dia; }
36
37     public LocalTime getHoraInicio() { return horaInicio; }
38     public void setHoraInicio(LocalTime horaInicio) { this.horaInicio =
39         horaInicio; }
40
41     public LocalTime getHoraFin() { return horaFin; }
42     public void setHoraFin(LocalTime horaFin) { this.horaFin = horaFin; }
43
44     public InstalacionSnapshot getInstalacionSnapshot() { return
45         instalacionSnapshot; }
46     public void setInstalacionSnapshot(InstalacionSnapshot
47         instalacionSnapshot) { this.instalacionSnapshot =
48         instalacionSnapshot; }
49
50     public boolean rangoValido() {
51         return horaInicio != null && horaFin != null && horaInicio.isBefore
52             (horaFin);
53     }
54 }
```

1.10.4.5 model/Reserva.java Se define `Reserva` como documento. Incluye:

- `fechaReserva`: marca temporal del momento en que se registró la reserva.

- `horario`: embebido.
- `usuarioId`: referencia al usuario.

```
1 package com.dam.reservas.model;  
2  
3 import jakarta.validation.Valid;  
4 import jakarta.validation.constraints.NotBlank;  
5 import jakarta.validation.constraints.NotNull;  
6 import org.springframework.data.annotation.Id;  
7 import org.springframework.data.mongodb.core.mapping.Document;  
8  
9 import java.time.Instant;  
10  
11 @Document(collection = "reservas")  
12 public class Reserva {  
13  
14     @Id  
15     private String id;  
16  
17     @NotNull(message = "fechaReserva es obligatoria")  
18     private Instant fechaReserva;  
19  
20     @NotNull(message = "horario es obligatorio")  
21     @Valid  
22     private Horario horario;  
23  
24     @NotBlank(message = "usuarioId es obligatorio")  
25     private String usuarioId;  
26  
27     public Reserva() {}  
28  
29     public Reserva(Instant fechaReserva, Horario horario, String  
30         usuarioId) {  
31         this.fechaReserva = fechaReserva;  
32         this.horario = horario;  
33         this.usuarioId = usuarioId;  
34     }  
35  
36     public String getId() { return id; }  
37     public void setId(String id) { this.id = id; }  
38  
39     public Instant getFechaReserva() { return fechaReserva; }  
40     public void setFechaReserva(Instant fechaReserva) { this.fechaReserva  
41         = fechaReserva; }  
42  
43     public Horario getHorario() { return horario; }  
44     public void setHorario(Horario horario) { this.horario = horario; }  
45  
46     public String getUsuarioId() { return usuarioId; }  
47     public void setUsuarioId(String usuarioId) { this.usuarioId =  
48         usuarioId; }
```

```
46 }
```

1.10.5 2) Repositorios (MongoRepository) y consultas

Los repositorios proporcionan CRUD automático y permiten definir consultas por convención de nombres, sin necesidad de escribir SQL.

1.10.5.1 repo/InstalacionRepository.java Consultas típicas:

- Por ciudad
- Búsqueda parcial por nombre

```
1 package com.dam.reservas.repo;
2
3 import com.dam.reservas.model.Instalacion;
4 import org.springframework.data.mongodb.repository.MongoRepository;
5
6 import java.util.List;
7
8 public interface InstalacionRepository extends MongoRepository<
9     Instalacion, String> {
10
11     List<Instalacion> findByCiudadIgnoreCase(String ciudad);
12
13     List<Instalacion> findByNombreContainingIgnoreCase(String q);
14 }
```

1.10.5.2 repo/UsuarioRepository.java Consultas típicas:

- Buscar por email (único lógico)

```
1 package com.dam.reservas.repo;
2
3 import com.dam.reservas.model.Usuario;
4 import org.springframework.data.mongodb.repository.MongoRepository;
5
6 import java.util.Optional;
7
8 public interface UsuarioRepository extends MongoRepository<Usuario,
9     String> {
10
11     Optional<Usuario> findByEmailIgnoreCase(String email);
12 }
```

1.10.5.3 repo/ReservaRepository.java Consultas típicas del dominio:

- Reservas por usuario.
- Reservas por instalación (usando el snapshot embebido).
- Reservas por día.
- Reservas por instalación y día (base para comprobar solapes).

```
1 package com.dam.reservas.repo;
2
3 import com.dam.reservas.model.Reserva;
4 import org.springframework.data.mongodb.repository.MongoRepository;
5
6 import java.time.LocalDate;
7 import java.util.List;
8
9 public interface ReservaRepository extends MongoRepository<Reserva,
10     String> {
11     List<Reserva> findByUsuarioId(String usuarioId);
12
13     List<Reserva> findByHorario_InstalacionSnapshot_InstalacionId(String
14         instalacionId);
15
16     List<Reserva> findByHorario_Dia(LocalDate dia);
17
18     List<Reserva>
19         findByHorario_InstalacionSnapshot_InstalacionIdAndHorario_Dia(
20             String instalacionId, LocalDate dia);
21 }
```

1.10.6 Consideraciones de consulta y modelado

1.10.6.1 Consultar por datos embebidos MongoDB permite consultar campos dentro de documentos anidados. Spring Data lo refleja mediante el uso del guion bajo en el nombre del método:

- `findByHorario_Dia(...)`
- `findByHorario_InstalacionSnapshot_InstalacionId(...)`

Esto evita escribir consultas manuales y favorece la claridad del código.

1.10.6.2 Recomendación sobre índices (preparación para rendimiento) Aunque no es imprescindible en un entorno docente inicial, en un proyecto real se crean índices para acelerar consultas frecuentes. Dado que se consultará con mucha frecuencia por:

- `usuarioId`

- `horario.dia`
- `horario.instalacionSnapshot.instalacionId`

En apartados posteriores se puede introducir `@Indexed` en el documento `Reserva` si se desea.

1.10.7 Estado del proyecto tras este apartado

Al finalizar este apartado, el proyecto queda preparado con:

- Clases de dominio mapeadas a MongoDB con `@Document`.
- Objetos embebidos definidos conforme al diagrama.
- Repositorios con CRUD y consultas base.

En el siguiente apartado comenzaremos:

- **DTOs de entrada y salida**
- **endpoints REST**
- validación a nivel de API y códigos HTTP

De este modo, se evita que las entidades de persistencia se utilicen directamente como contrato de la API, manteniendo una separación clara entre capas.

1.11 Diseño de la API REST: endpoints, DTOs, códigos HTTP

En este apartado se define el **contrato REST** de la aplicación de reservas de pistas deportivas: qué recursos existen, qué endpoints se exponen, qué datos se intercambian mediante **DTOs**, y qué **códigos HTTP** se deben devolver en cada operación. Esta definición es clave para desacoplar el frontend del backend y para garantizar una API coherente, predecible y fácilmente testeable.

El diseño se basa en el modelo ya establecido:

- **Instalacion** (documento independiente)
- **Usuario** (documento independiente)
- **Reserva** (documento principal), que embebe **Horario** y el snapshot de instalación, y referencia a usuario mediante **usuarioId**

1.11.1 Principios de diseño aplicados

1. **Recursos como sustantivos** Las rutas representan recursos (**/reservas**, **/usuarios**, **/instalaciones**) y las operaciones se expresan con métodos HTTP.
2. **JSON como formato de intercambio** Todas las peticiones y respuestas utilizan JSON.
3. **Separación entre entidades y DTOs** Las clases del paquete **model** no se exponen directamente en la API. Se diseñan DTOs específicos de entrada y salida.
4. **Códigos HTTP coherentes** Cada endpoint debe responder con el código más apropiado, evitando respuestas ambiguas.
5. **Validación en el borde (API)** Los DTOs de entrada se validan con **@Valid** y anotaciones de Bean Validation para evitar guardar datos inválidos.

1.11.2 Recursos y endpoints

1.11.2.1 1) Recurso: Instalaciones Las instalaciones representan las pistas o instalaciones deportivas disponibles.

1.11.2.1.1 Endpoints propuestos

Método	Ruta	Descripción	Respuesta
GET	<code>/api/instalaciones</code>	Listar instalaciones (con filtros opcionales)	200 + lista
GET	<code>/api/instalaciones/{id}</code>	Obtener una instalación	200 / 404
POST	<code>/api/instalaciones</code>	Crear instalación	201 / 400
PUT	<code>/api/instalaciones/{id}</code>	Actualizar instalación	200 / 400 / 404
DELETE	<code>/api/instalaciones/{id}</code>	Eliminar instalación	204 / 404

Filtros recomendados:

- `?ciudad=Jaén`
- `?q=pista` (búsqueda por nombre)

1.11.2.2 2) Recurso: Usuarios Los usuarios se almacenan como documentos independientes y se referencian desde la reserva por `usuarioId`.

1.11.2.2.1 Endpoints propuestos

Método	Ruta	Descripción	Respuesta
GET	<code>/api/usuarios</code>	Listar usuarios	200 + lista
GET	<code>/api/usuarios/{id}</code>	Obtener usuario	200 / 404
POST	<code>/api/usuarios</code>	Crear usuario	201 / 400 / 409
PUT	<code>/api/usuarios/{id}</code>	Actualizar usuario	200 / 400 / 404 / 409
DELETE	<code>/api/usuarios/{id}</code>	Eliminar usuario	204 / 404

Se recomienda tratar `email` como identificador lógico único (aunque el índice único se puede introducir más adelante como extensión).

1.11.2.3 3) Recurso: Reservas La reserva es el recurso principal del sistema.

- Embebe `Horario`
- `Horario` embebe `InstalacionSnapshot`
- Referencia a usuario por `usuarioId`

1.11.2.3.1 Endpoints propuestos

Método	Ruta	Descripción	Respuesta
GET	<code>/api/reservas</code>	Listar reservas (con filtros)	200 + lista
GET	<code>/api/reservas/{id}</code>	Obtener una reserva	200 / 404
POST	<code>/api/reservas</code>	Crear reserva	201 / 400 / 404 / 409
PUT	<code>/api/reservas/{id}</code>	Actualizar reserva (reemplazo)	200 / 400 / 404 / 409
DELETE	<code>/api/reservas/{id}</code>	Eliminar reserva	204 / 404

Filtros recomendados para `GET /api/reservas`:

- `?usuarioId=...`
- `?instalacionId=...`
- `?dia=2026-01-18`
- combinación: `?instalacionId=...&dia=...` (base para evitar solapes)

1.11.2.3.2 Conflicto por solape (409) Si se intenta crear/modificar una reserva que se solapa con otra reserva existente de la misma instalación y mismo día, el endpoint debe responder:

- **409 Conflict**
- cuerpo JSON con detalle del motivo

1.11.3 DTOs (Data Transfer Objects)

Se definen DTOs de entrada (request) y salida (response). Los DTOs de entrada se diseñan con los campos mínimos necesarios, evitando que el cliente tenga que construir manualmente objetos embebidos como el snapshot de instalación. El backend se encarga de construir el snapshot consultando la instalación.

1.11.3.1 DTOs para Instalaciones

1.11.3.1.1 InstalacionRequest Campos requeridos para crear/actualizar:

- `nombre`
- `direccion`
- `ciudad`

1.11.3.1.2 InstalacionResponse Incluye además el `id`.

1.11.3.2 DTOs para Usuarios

1.11.3.2.1 UsuarioRequest

- `nombre`
- `email`

1.11.3.2.2 UsuarioResponse Incluye `id`.

1.11.3.3 DTOs para Reservas La reserva necesita:

- Referencia a usuario (`usuarioId`)
- Referencia a instalación (`instalacionId`) para construir el snapshot en el backend
- Día y franja horaria

1.11.3.3.1 ReservaCreateRequest (entrada)

- `usuarioId`
- `instalacionId`
- `dia`

- `horaInicio`
- `horaFin`

1.11.3.3.2 ReservaResponse (salida)

- `id`
- `fechaReserva` (timestamp)
- `usuarioId`
- `horario` con snapshot de instalación embebido

Se recomienda que el response incluya el horario embebido completo, ya que es precisamente el valor añadido del modelo documental.

1.11.4 Códigos HTTP y comportamiento esperado

1.11.4.1 Respuestas típicas

- **200 OK** Operación correcta en GET/PUT.
- **201 Created** Recurso creado correctamente (POST). Recomendación: devolver el recurso creado.
- **204 No Content** Eliminación correcta (DELETE) sin cuerpo de respuesta.
- **400 Bad Request** Datos inválidos (validación), formato incorrecto o parámetros incoherentes.
- **404 Not Found** Recurso inexistente (por ejemplo `usuarioId` inválido al reservar).
- **409 Conflict** Violación de regla de negocio, típicamente:
 - Solape de reserva
 - Email duplicado (si se impone unicidad)

1.11.4.2 Estructura de errores Para mantener uniformidad, se recomienda una estructura JSON única para los errores, gestionada por `@ControllerAdvice`:

```
1 {
2   "timestamp": "2026-01-18T18:45:00Z",
3   "status": 400,
4   "error": "VALIDATION",
5   "message": "Datos no válidos",
6   "details": [
7     "día: es obligatorio",
8     "horaFin: debe ser posterior a horaInicio"
9   ]
10 }
```

1.11.5 Ejemplos de intercambio (contrato)

1.11.5.1 Crear reserva (POST /api/reservas) Request:

```
1 {  
2   "usuarioId": "65a199f3e0b1c2d3e4f56789",  
3   "instalacionId": "65a18888aa11223344556677",  
4   "dia": "2026-01-18",  
5   "horaInicio": "18:00",  
6   "horaFin": "19:00"  
7 }
```

Respuestas:

- 201 si se crea
- 404 si el usuario o la instalación no existen
- 409 si la franja se solapa

1.11.6 Preparación para el siguiente apartado

Tras definir el contrato REST, el siguiente paso es implementarlo:

1. Crear los DTOs de entrada/salida.
2. Mapear entidades ↔ DTOs.
3. Implementar controladores y servicios aplicando:
 - Validación (`@Valid`)
 - Códigos HTTP correctos
 - Gestión uniforme de errores (`ControllerAdvice`)
 - Regla de negocio de solape (409 Conflict)

En el siguiente apartado se desarrollarán los **DTOs concretos y su validación**, dejando los controladores listos para implementar los endpoints de la API.

1.12 Validación, DTOs y Control de Errores (@ControllerAdvice)

En el desarrollo de una API REST profesional, no basta con que el código “funcione”. Debe ser robusto, seguro y fácil de mantener. Para ello, nos apoyamos en dos pilares fundamentales: el patrón **DTO** para la transferencia de datos y un **Manejador Global de Excepciones**.

En una API REST, la validación de datos y la gestión homogénea de errores son elementos esenciales para garantizar que el sistema:

- Rechaza entradas inválidas antes de persistirlas.
- Responde con **códigos HTTP correctos** y mensajes consistentes.
- Facilita el consumo desde el frontend (manejo de errores predecible).
- Simplifica el mantenimiento: los controladores no se llenan de **try/catch**.

1.12.1 1. Entendiendo el patrón DTO (Data Transfer Object)

Antes de implementar el código, es crucial entender qué es un DTO y por qué no debemos usar nuestras Entidades (@Entity) directamente en los controladores.

¿Qué es un DTO? Un DTO es un objeto simple (POJO o Java Record) que transporta datos de un proceso a otro. No tiene lógica de negocio ni acceso a base de datos; es puramente un contenedor de datos.

¿Por qué lo usamos en lugar de la Entidad?

1. **Desacoplamiento (Seguridad):** Tu base de datos no debe exponerse directamente al cliente. Una entidad puede tener campos sensibles (como `password`, `fechaCreacion`, `flagBorrado`) que el cliente no debe ver ni manipular. El DTO actúa como un filtro.
2. **Validación Específica:** Las reglas de validación de la API (ej. “el email debe tener formato válido”) suelen ser diferentes a las de la base de datos (ej. “el email no puede ser nulo”). Usar DTOs permite poner anotaciones `@NotBlank` o `@Email` sin ensuciar la entidad JPA.
3. **Independencia del Modelo:** Si mañana cambias la estructura de tu base de datos (divides una tabla en dos), no rompes la API que consumen los clientes, porque el DTO se mantiene estable y tú te encargas de transformar los datos internamente.
4. **Optimización:** Puedes crear diferentes DTOs para la misma entidad. Por ejemplo, `UsuarioResumenDTO` (solo nombre y foto) para un listado, y `UsuarioDetalleDTO` (todos los datos) para la vista de perfil.

El Flujo de Trabajo:

1. **Request:** El cliente envía un JSON Spring lo convierte a **DTO**.
2. **Mapping:** El controlador o servicio convierte ese **DTO** a una **Entidad**.

3. **Persistencia:** La **Entidad** se guarda en la BBDD.

1.12.2 2. Implementación de Validación en DTOs

La validación se realiza en los **DTOs de entrada** (request). Esto permite rechazar peticiones mal formadas antes de que lleguen siquiera a procesarse.

Este diagrama de secuencia muestra cómo un dato “crudo” (JSON) se transforma en un DTO validado y, finalmente, en una Entidad persistente. Ilustra la separación de responsabilidades:

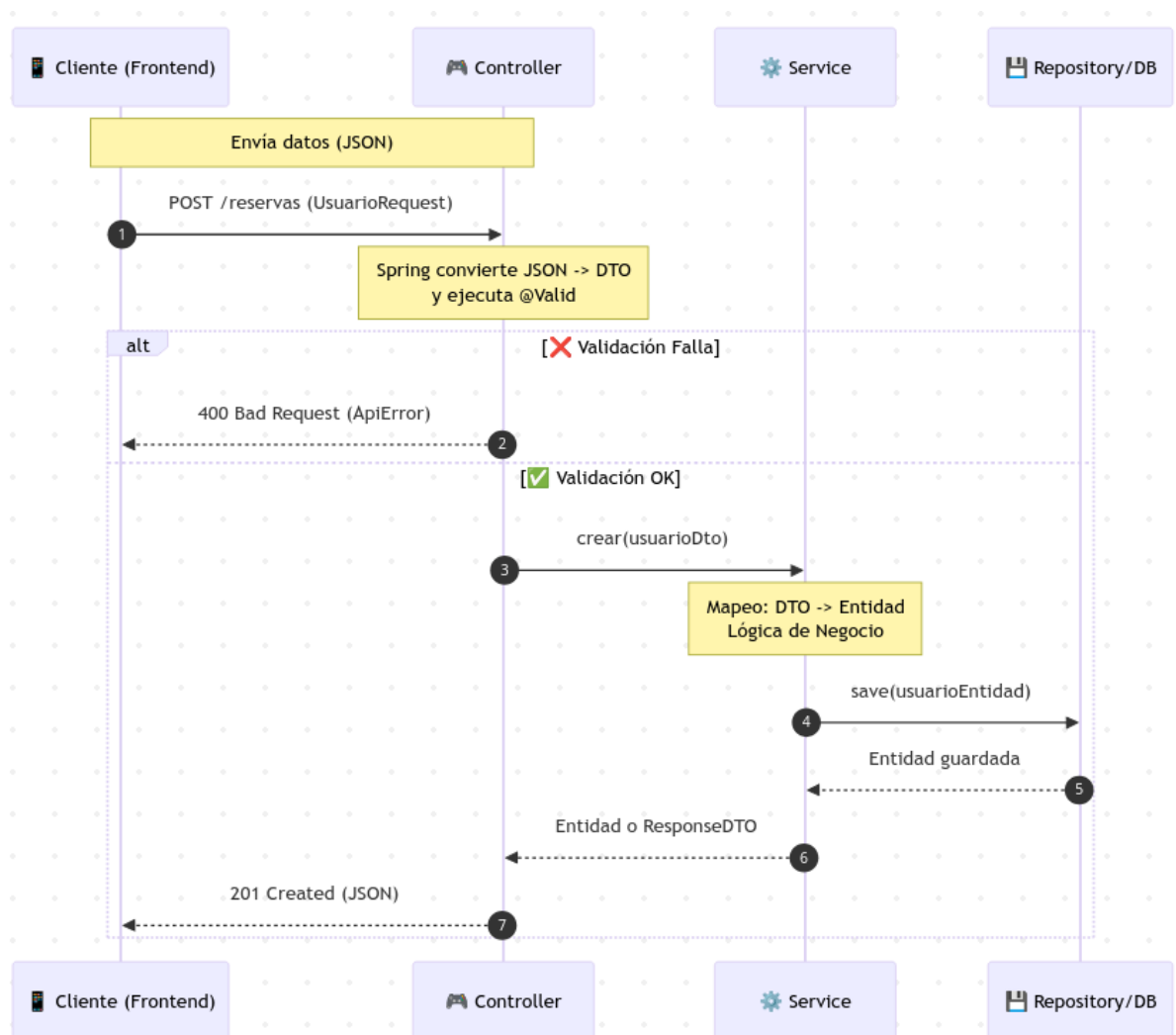


Figura 6: Diagrama secuencia conversión JSON-Entidad

1 sequenceDiagram


```

2    autonumber
3    participant Cliente as Cliente (Frontend)
4    participant Ctrl as Controller
5    participant Srv as Service
6    participant Repo as Repository/DB
7
8    Note over Cliente, Ctrl: Envía datos (JSON)
9    Cliente->>Ctrl: POST /reservas (UsuarioRequest)
10
11    Note over Ctrl: Spring convierte JSON -> DTO<br/>y ejecuta @Valid
12
13    alt Validación Falla
14        Ctrl-->>Cliente: 400 Bad Request (ApiError)
15    else Validación OK
16        Ctrl->>Srv: crear(usuarioDto)
17
18        Note over Srv: Mapeo: DTO -> Entidad<br/>Lógica de Negocio
19
20        Srv->>Repo: save(usuarioEntidad)
21        Repo-->>Srv: Entidad guardada
22
23        Srv-->>Ctrl: Entidad o ResponseDTO
24        Ctrl-->>Cliente: 201 Created (JSON)
25    end

```

1.12.2.1 Dependencia necesaria Asegúrate de tener en `pom.xml`:

- `spring-boot-starter-validation`

1.12.2.2 Activación en el Controlador Usamos dos anotaciones clave:

- `@RequestBody`: Le dice a Spring “toma el JSON del cuerpo de la petición y conviértelo a este objeto Java”.
- `@Valid`: Le dice a Spring “antes de entrar al método, revisa que este objeto cumpla todas las restricciones definidas en su clase”.

```

1  @PostMapping
2  @ResponseStatus(HttpStatus.CREATED)
3  // Si 'req' no cumple las validaciones, el método NO se ejecuta
4  // y salta una excepción automática.
5  public ReservaResponse crear(@Valid @RequestBody ReservaCreateRequest
    req) { ... }

```

1.12.2.3 Definición de DTOs con `jakarta.validation` A continuación, los DTOs usando Java Records (inmutables y concisos) y anotaciones de validación.

dto/InstalacionRequest.java

```
1 package com.dam.reservas.dto;
2
3 import jakarta.validation.constraints.NotBlank;
4
5 public record InstalacionRequest(
6     @NotBlank(message = "nombre es obligatorio") String nombre,
7     @NotBlank(message = "direccion es obligatoria") String direccion,
8     @NotBlank(message = "ciudad es obligatoria") String ciudad
9 ) {}
```

dto/UsuarioRequest.java

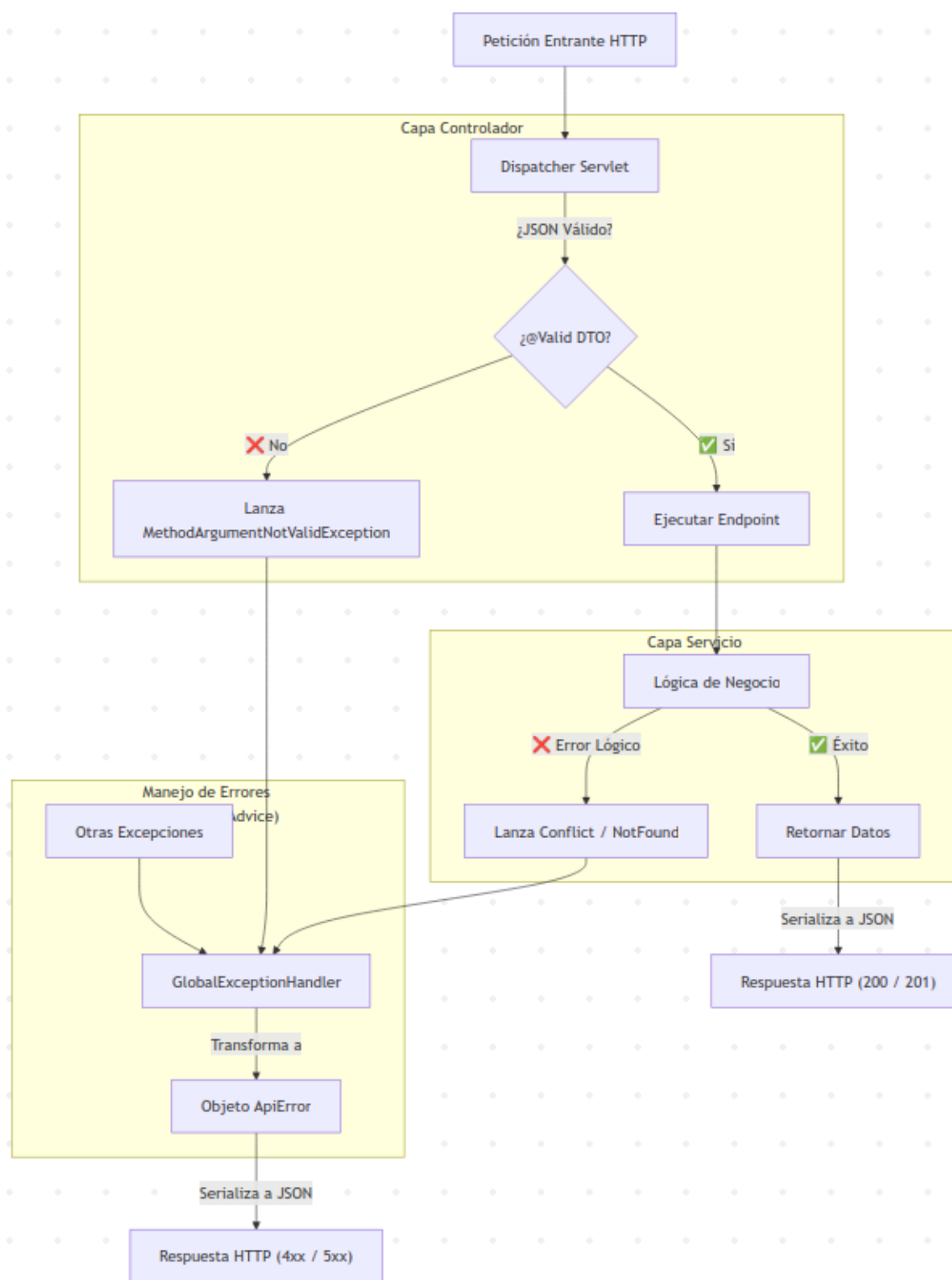
```
1 package com.dam.reservas.dto;
2
3 import jakarta.validation.constraints.Email;
4 import jakarta.validation.constraints.NotBlank;
5
6 public record UsuarioRequest(
7     @NotBlank(message = "nombre es obligatorio") String nombre,
8     @NotBlank(message = "email es obligatorio") @Email(message = "email
9         no válido") String email
10 ) {}
```

dto/ReservaCreateRequest.java *Nota:* Aquí validamos formato y presencia. Las reglas de negocio complejas (ej. “hora fin mayor que hora inicio”) se validan en el Servicio.

```
1 package com.dam.reservas.dto;
2
3 import jakarta.validation.constraints.NotBlank;
4 import jakarta.validation.constraints.NotNull;
5 import java.time.LocalDate;
6 import java.time.LocalDateTime;
7
8 public record ReservaCreateRequest(
9     @NotBlank(message = "usuarioId es obligatorio") String usuarioId,
10    @NotBlank(message = "instalacionId es obligatorio") String
11        instalacionId,
12    @NotNull(message = "dia es obligatorio") LocalDate dia,
13    @NotNull(message = "horaInicio es obligatoria") LocalDateTime
14        horaInicio,
15    @NotNull(message = "horaFin es obligatoria") LocalDateTime horaFin
16 ) {}
```

1.12.3 3. Manejo Centralizado de Excepciones

En una aplicación tradicional, tendríamos bloques **try-catch** repetidos en cada controlador, lo cual ensucia el código y hace difícil mantener una respuesta de error coherente.

**Figura 7:** Diagrama de flujo para el manejo de excepciones

```

1 flowchart TD
2     req["Petición Entrante HTTP"] --> dispatcher["Dispatcher Servlet"]
3
4     subgraph Capa_Controlador ["Capa Controlador"]
5         dispatcher -->|?`JSON Válido?| valid{"?`@Valid DTO?"}
6         valid -->| No| exValid["Lanza MethodArgumentNotValidException"]
7         valid -->| Sí| ctrl["Ejecutar Endpoint"]
8     end
9
10    subgraph Capa_Servicio ["Capa Servicio"]
11        ctrl --> service["Lógica de Negocio"]
12        service -->| Error Lógico| exBiz["Lanza Conflict / NotFound"]
13        service -->| Éxito| success["Retornar Datos"]
14    end
15
16    subgraph Manejo_Errores ["Manejo de Errores (@ControllerAdvice)"]
17        exValid --> handler["GlobalExceptionHandler"]
18        exBiz --> handler
19        exGenerica["Otras Excepciones"] --> handler
20
21        handler -->|Transforma a| apiError["Objeto ApiError"]
22    end
23
24    apiError -->|Serializa a JSON| response["Respuesta HTTP (4xx / 5xx)"]
25    success -->|Serializa a JSON| response2["Respuesta HTTP (200 / 201)"]

```

Spring Boot ofrece una solución elegante: **@ControllerAdvice**.

¿Cómo funciona? Funciona mediante Programación Orientada a Aspectos (AOP). Es un componente que “escucha” a todos los controladores. Si un controlador lanza una excepción (ej. `NotFoundException`), el flujo normal se interrumpe y el `@ControllerAdvice` captura esa excepción, permitiéndote generar una respuesta JSON personalizada.

Beneficios:

- **Código Limpio:** Los controladores solo se ocupan del “camino feliz” (cuando todo va bien).
- **Consistencia:** Todos los errores (404, 400, 500) tienen exactamente el mismo formato JSON, facilitando la vida al desarrollador del Frontend.

1.12.3.1 3.1. Modelo de error unificado (ApiError) Definimos una “plantilla” de cómo se verán todos nuestros errores.

```

1 package com.dam.reservas.web;
2
3 import java.time.Instant;

```

```
4 import java.util.List;
5
6 public record ApiError(
7     Instant timestamp, // Cuándo ocurrió
8     int status,         // Código HTTP (400, 404, etc.)
9     String error,       // Nombre corto del error
10    String message,      // Mensaje legible para humanos
11    List<String> details // Detalles técnicos o lista de campos
                        // fallidos
12 ) {}
```

1.12.3.2 3.2. Excepciones de Dominio Creamos nuestras propias excepciones semánticas que heredan de `RuntimeException`. Esto nos permite lanzar errores que “significan algo” en nuestro negocio.

web/NotFoundException.java (Para recursos no encontrados - 404)

```
1 package com.dam.reservas.web;
2 public class NotFoundException extends RuntimeException {
3     public NotFoundException(String message) { super(message); }
4 }
```

web/BadRequestException.java (Para peticiones mal formadas - 400)

```
1 package com.dam.reservas.web;
2 public class BadRequestException extends RuntimeException {
3     public BadRequestException(String message) { super(message); }
4 }
```

web/ConflictException.java (Para reglas de negocio rotas, ej. solapamiento - 409)

```
1 package com.dam.reservas.web;
2 public class ConflictException extends RuntimeException {
3     public ConflictException(String message) { super(message); }
4 }
```

1.12.3.3 3.3. Implementación del GlobalExceptionHandler Aquí es donde ocurre la magia. Mapeamos cada excepción Java a una respuesta HTTP concreta.

web/GlobalExceptionHandler.java

```
1 package com.dam.reservas.web;
2
3 import org.springframework.http.HttpStatus;
4 import org.springframework.http.ResponseEntity;
5 import org.springframework.validation.FieldError;
6 import org.springframework.web.bind.MethodArgumentNotValidException;
```

```
7 import org.springframework.web.bind.annotation.*;
8
9 import java.time.Instant;
10 import java.util.List;
11
12 @ControllerAdvice
13 public class GlobalExceptionHandler {
14
15     // 1. Manejo de Recurso No Encontrado (404)
16     @ExceptionHandler(NotFoundException.class)
17     public ResponseEntity<ApiError> handleNotFound(NotFoundException ex)
18     {
19         return ResponseEntity.status(HttpStatus.NOT_FOUND).body(
20             new ApiError(Instant.now(), 404, "NOT_FOUND", ex.getMessage(),
21                 List.of())
22         );
23     }
24
25     // 2. Manejo de Reglas de Negocio Generales (400)
26     @ExceptionHandler(BadRequestException.class)
27     public ResponseEntity<ApiError> handleBadRequest(BadRequestException
28     ex) {
29         return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
30             new ApiError(Instant.now(), 400, "BAD_REQUEST", ex.getMessage(),
31                 List.of())
32         );
33     }
34
35     // 3. Manejo de Conflictos (409) - Ej: Reserva duplicada
36     @ExceptionHandler(ConflictException.class)
37     public ResponseEntity<ApiError> handleConflict(ConflictException ex)
38     {
39         return ResponseEntity.status(HttpStatus.CONFLICT).body(
40             new ApiError(Instant.now(), 409, "CONFLICT", ex.getMessage(),
41                 List.of())
42         );
43     }
44
45     // 4. Manejo de Validación de DTOs (@Valid falla)
46     // Spring lanza MethodArgumentNotValidException cuando @Valid detecta
47     // errores.
48     @ExceptionHandler(MethodArgumentNotValidException.class)
49     public ResponseEntity<ApiError> handleValidation(
50         MethodArgumentNotValidException ex) {
51
52         // Extraemos los errores campo por campo para informar al frontend
53         List<String> details = ex.getBindingResult().getAllErrors().stream
54             ()
55             .map(err -> {
56                 if (err instanceof FieldError fe) {
57                     return fe.getField() + ": " + fe.getDefaultMessage();
58                 }
59             })
60             .collect(Collectors.toList());
61
62         return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
63             new ApiError(Instant.now(), 400, "VALIDATION_ERROR", details,
64                 List.of())
65         );
66     }
67 }
```

```

49         }
50         return err.getDefaultMessage();
51     })
52     .toList();
53
54     return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
55         new ApiError(Instant.now(), 400, "VALIDATION", "Datos de
           entrada inválidos", details)
56     );
57 }
58
59 // 5. Manejo de Errores Inesperados (500)
60 // Captura cualquier otra excepción no controlada (NullPointerException, SQL
   error, etc.)
61 @ExceptionHandler(Exception.class)
62 public ResponseEntity<ApiError> handleGeneric(Exception ex) {
63     // IMPORTANTE: En producción, no mostrar ex.getMessage() si
       contiene info sensible.
64     return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(
65         (
66             new ApiError(Instant.now(), 500, "INTERNAL_ERROR", "Error
               interno del servidor", List.of(ex.getMessage()))
67         );
68 }

```

1.12.4 4. Resumen de Códigos de Estado HTTP

Al implementar estas validaciones y excepciones, tu API responderá semánticamente:

Código HTTP	Significado	Cuándo ocurre
201 Created	Creado	Reserva creada exitosamente.
400 Bad Request	Petición Incorrecta	Falla <code>@Valid</code> (campo vacío) o regla lógica simple (hora fin < hora inicio).
404 Not Found	No Encontrado	ID de usuario o instalación no existen en BBDD.
409 Conflict	Conflicto	Intento de reservar una pista que ya está ocupada.

Código HTTP	Significado	Cuándo ocurre
500 Internal Error	Error Servidor	Fallo de conexión a BBDD o error de programación no controlado.

1.12.5 Conclusiones

Al integrar este código, tu aplicación:

1. Rechaza automáticamente JSONs inválidos gracias a los **DTOs validados**.
2. Captura errores de lógica de negocio lanzados desde los Servicios.
3. Devuelve siempre una estructura JSON uniforme (**ApiError**) que el Frontend puede parsear fácilmente para mostrar alertas al usuario.

En el siguiente apartado se implementará **CORS y configuración por entornos**.

1.13 Validación y control de errores (@ControllerAdvice)

En una API REST, la validación de datos y la gestión homogénea de errores son elementos esenciales para garantizar que el sistema:

- Rechaza entradas inválidas antes de persistirlas.
- Responde con **códigos HTTP correctos** y mensajes consistentes.
- Facilita el consumo desde el frontend (manejo de errores predecible).
- Simplifica el mantenimiento: los controladores no se llenan de **try/catch**.

En este apartado se implementan dos bloques:

1. **Validación de DTOs** mediante Bean Validation (`jakarta.validation`) y `@Valid`.
2. **Manejo centralizado de excepciones** mediante `@ControllerAdvice`.

1.13.1 1) Validación de entrada en la API

La validación se realiza en los **DTOs de entrada** (request), no en las entidades del modelo persistente. Esto permite:

- Separar contrato de API y persistencia.
- Validar solo lo que el cliente debe enviar.
- Mantener reglas coherentes aunque la persistencia evolucione.

1.13.1.1 Dependencia necesaria Debe estar incluida la dependencia:

- `spring-boot-starter-validation`

1.13.1.2 Activación de validación en controladores En cada endpoint que reciba datos, se debe usar:

- `@Valid` en el parámetro del DTO.
- `@RequestBody` para deserializar JSON.

Ejemplo conceptual:

```
1 @PostMapping
2 @ResponseStatus(HttpStatus.CREATED)
3 public ReservaResponse crear(@Valid @RequestBody ReservaCreateRequest
    req) { ... }
```

Si el DTO no cumple las restricciones, Spring lanzará automáticamente una `MethodArgumentNotValidException`.

1.13.2 2) DTOs con anotaciones de Bean Validation

A continuación se incluyen los DTOs mínimos para el dominio, con validación.

1.13.2.1 dto/InstalacionRequest.java

```
1 package com.dam.reservas.dto;  
2  
3 import jakarta.validation.constraints.NotBlank;  
4  
5 public record InstalacionRequest(  
6     @NotBlank(message = "nombre es obligatorio") String nombre,  
7     @NotBlank(message = "direccion es obligatoria") String direccion,  
8     @NotBlank(message = "ciudad es obligatoria") String ciudad  
9 ) {}
```

1.13.2.2 dto/UsuarioRequest.java

```
1 package com.dam.reservas.dto;  
2  
3 import jakarta.validation.constraints.Email;  
4 import jakarta.validation.constraints.NotBlank;  
5  
6 public record UsuarioRequest(  
7     @NotBlank(message = "nombre es obligatorio") String nombre,  
8     @NotBlank(message = "email es obligatorio") @Email(message = "email  
    no válido") String email  
9 ) {}
```

1.13.2.3 dto/ReservaCreateRequest.java Se valida que los campos existan. La relación `horaInicio < horaFin` es una regla de negocio; se validará en la capa de servicio para poder responder con 400/409 según corresponda.

```
1 package com.dam.reservas.dto;  
2  
3 import jakarta.validation.constraints.NotBlank;  
4 import jakarta.validation.constraints.NotNull;  
5  
6 import java.time.LocalDate;  
7 import java.time.LocalTime;  
8  
9 public record ReservaCreateRequest(  
10     @NotBlank(message = "usuarioId es obligatorio") String usuarioId,  
11     @NotBlank(message = "instalacionId es obligatorio") String  
        instalacionId,  
12     @NotNull(message = "dia es obligatorio") LocalDate dia,  
13     @NotNull(message = "horaInicio es obligatoria") LocalTime  
        horaInicio,  
14     @NotNull(message = "horaFin es obligatoria") LocalTime horaFin
```

```
15 ) {}
```

1.13.3 3) Modelo de error unificado

Se define un formato de respuesta común para los errores. Esto permite al frontend:

- Mostrar mensajes coherentes.
- Leer detalles campo a campo cuando procede.
- Diferenciar tipos de error (`VALIDATION`, `NOT_FOUND`, etc.).

1.13.3.1 web/ApiError.java

```
1 package com.dam.reservas.web;  
2  
3 import java.time.Instant;  
4 import java.util.List;  
5  
6 public record ApiError(  
7     Instant timestamp,  
8     int status,  
9     String error,  
10    String message,  
11    List<String> details  
12 ) {}
```

1.13.4 4) Excepciones de dominio

Se definen excepciones específicas que se lanzarán desde la capa de servicio:

- `NotFoundException` → 404
- `BadRequestException` → 400
- `ConflictException` → 409 (por solapes u otras reglas)

1.13.4.1 web/NotFoundException.java

```
1 package com.dam.reservas.web;  
2  
3 public class NotFoundException extends RuntimeException {  
4     public NotFoundException(String message) { super(message); }  
5 }
```

1.13.4.2 web/BadRequestException.java

```
1 package com.dam.reservas.web;  
2  
3 public class BadRequestException extends RuntimeException {
```

```
4 public BadRequestException(String message) { super(message); }
5 }
```

1.13.4.3 web/ConflictException.java

```
1 package com.dam.reservas.web;
2
3 public class ConflictException extends RuntimeException {
4     public ConflictException(String message) { super(message); }
5 }
```

1.13.5 5) Manejo centralizado con @ControllerAdvice

Con `@ControllerAdvice` se interceptan excepciones lanzadas en cualquier controlador y se transforman en respuestas HTTP con el formato establecido.

1.13.5.1 web/GlobalExceptionHandler.java

```
1 package com.dam.reservas.web;
2
3 import org.springframework.http.HttpStatus;
4 import org.springframework.http.ResponseEntity;
5 import org.springframework.validation.FieldError;
6 import org.springframework.web.bind.MethodArgumentNotValidException;
7 import org.springframework.web.bind.annotation.*;
8
9 import java.time.Instant;
10 import java.util.List;
11
12 @ControllerAdvice
13 public class GlobalExceptionHandler {
14
15     @ExceptionHandler(NotFoundException.class)
16     public ResponseEntity<ApiError> handleNotFound(NotFoundException ex)
17     {
18         return ResponseEntity.status(HttpStatus.NOT_FOUND).body(
19             new ApiError(Instant.now(), 404, "NOT_FOUND", ex.getMessage(),
20                 List.of())
21         );
22     }
23
24     @ExceptionHandler(BadRequestException.class)
25     public ResponseEntity<ApiError> handleBadRequest(BadRequestException
26     ex) {
27         return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
28             new ApiError(Instant.now(), 400, "BAD_REQUEST", ex.getMessage(),
29                 List.of())
30         );
31     }
32 }
```

```
28
29 @ExceptionHandler(ConflictException.class)
30 public ResponseEntity<ApiError> handleConflict(ConflictException ex)
31 {
32     return ResponseEntity.status(HttpStatus.CONFLICT).body(
33         new ApiError(Instant.now(), 409, "CONFLICT", ex.getMessage(),
34             List.of())
35     );
36 }
37
38 @ExceptionHandler(MethodArgumentNotValidException.class)
39 public ResponseEntity<ApiError> handleValidation(
40     MethodArgumentNotValidException ex) {
41
42     List<String> details = ex.getBindingResult().getAllErrors().stream
43     ()
44     .map(err -> {
45         if (err instanceof FieldError fe) {
46             return fe.getField() + ": " + fe.getDefaultMessage();
47         }
48         return err.getDefaultMessage();
49     })
50     .toList();
51
52     return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
53         new ApiError(Instant.now(), 400, "VALIDATION", "Datos no vá
54         lidos", details)
55     );
56 }
57
58 @ExceptionHandler(Exception.class)
59 public ResponseEntity<ApiError> handleGeneric(Exception ex) {
60     // En un entorno docente puede resultar útil devolver el mensaje.
61     // En producción se recomienda no exponer detalles internos.
62     return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(
63         (
64             new ApiError(Instant.now(), 500, "INTERNAL_ERROR", "Error
65             interno del servidor", List.of(ex.getMessage()))
66         )
67     );
68 }
69 }
```

1.13.6 6) Reglas de negocio y códigos HTTP

No toda validación es “campo requerido”. En una aplicación real aparecen reglas como:

- `horaInicio` debe ser anterior a `horaFin` → **400 Bad Request**
- No se puede reservar una pista si ya hay una reserva solapada → **409 Conflict**

- No se puede reservar para un usuario inexistente → **404 Not Found**

Estas reglas se implementarán en la capa de servicio y lanzarán las excepciones definidas, para que el `@ControllerAdvice` las convierta en respuestas coherentes.

1.13.7 Resultado de este apartado

Al finalizar este apartado se dispone de:

- DTOs de entrada con validación.
- Un formato de error uniforme (`ApiError`).
- Excepciones de dominio.
- Un `@ControllerAdvice` que convierte errores en respuestas HTTP coherentes.

En el siguiente apartado se implementará **CORS y configuración por entornos**, garantizando que el frontend pueda consumir la API de forma segura y controlada en desarrollo y despliegue.

1.14 CORS y configuración por entornos

En una arquitectura cliente–servidor donde el frontend (HTML/JS) consume una API REST, es habitual que el navegador aplique la política de seguridad **Same-Origin Policy**. Esta política bloquea, por defecto, peticiones AJAX a un origen distinto (dominio, puerto o protocolo diferentes). Para permitir el acceso controlado a la API se utiliza **CORS (Cross-Origin Resource Sharing)**.

Además, en proyectos reales se requiere distinguir configuraciones según el entorno:

- **Desarrollo** (Dev Container, puertos locales, logs más verbosos).
- **Producción** (orígenes permitidos restringidos, credenciales seguras, logs moderados).

En este apartado se configura:

1. CORS para permitir el consumo desde el frontend.
2. Configuración por entornos con perfiles (**dev**, **prod**) manteniendo el uso de **.env**.

1.14.1 1) Conceptos básicos de CORS

CORS es un mecanismo basado en cabeceras HTTP que permite a un servidor indicar qué orígenes están autorizados a realizar solicitudes.

Elementos habituales:

- **Access-Control-Allow-Origin**: origen permitido.
- **Access-Control-Allow-Methods**: métodos permitidos (GET, POST, etc.).
- **Access-Control-Allow-Headers**: cabeceras permitidas.
- Petición **preflight** (OPTIONS): el navegador la envía antes de algunas solicitudes para comprobar permisos.

En desarrollo, si el frontend se abre, por ejemplo, desde:

- **http://localhost:5500** (Live Server) y la API está en:
- **http://localhost:8080**

son orígenes distintos por el puerto y el navegador bloqueará la llamada si no hay CORS configurado.

1.14.2 2) Estrategias posibles en este proyecto

En este capítulo se trabajará con dos escenarios comunes:

- **Escenario A (más simple):** el frontend se sirve desde Spring Boot (`src/main/resources/static`) y el origen es el mismo. En este caso, CORS no es estrictamente necesario, pero conviene conocerlo porque no siempre será así.
- **Escenario B (habitual en prácticas):** el frontend se sirve desde un servidor distinto (Live Server, Nginx, etc.). Aquí sí es necesario habilitar CORS.

Se implementará una configuración que permita **activar/desactivar** el origen permitido mediante variables de entorno.

1.14.3 3) Configuración de CORS en Spring Boot

Se define una clase de configuración implementando `WebMvcConfigurer` para aplicar CORS de forma global a `/api/**`.

1.14.3.1 config/CorsConfig.java

```
1 package com.dam.reservas.config;
2
3 import org.springframework.beans.factory.annotation.Value;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.web.servlet.config.annotation.*;
6
7 @Configuration
8 public class CorsConfig implements WebMvcConfigurer {
9
10     @Value("${app.cors.allowed-origins:*}")
11     private String allowedOrigins;
12
13     @Override
14     public void addCorsMappings(CorsRegistry registry) {
15         registry.addMapping("/api/**")
16             .allowedOrigins(allowedOrigins.split(","))
17             .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS")
18             .allowedHeaders("*")
19             .allowCredentials(false)
20             .maxAge(3600);
21     }
22 }
```

Notas:

- `allowedOrigins` se puede definir como una lista separada por comas.
- En desarrollo puede ser `*` (no recomendado en producción).
- `allowCredentials(false)` evita combinaciones inseguras con `*`.

1.14.4 4) Configuración por entornos con perfiles

Spring Boot permite definir distintos ficheros YAML por perfil:

- `application.yml` (común)
- `application-dev.yml`
- `application-prod.yml`

La idea es mantener en `application.yml` lo esencial y delegar ajustes por perfil (CORS, logs, etc.) a los YAML específicos.

1.14.4.1 4.1 `application.yml` (base) Ubicación: `src/main/resources/application.yml`

```
1 server:
2   port: 8080
3
4 spring:
5   profiles:
6     active: ${SPRING_PROFILES_ACTIVE:dev}
7
8   data:
9     mongodb:
10      uri: mongodb://${MONGO_INITDB_ROOT_USERNAME}:${MONGO_INITDB_ROOT_PASSWORD}@mongo:27017/${MONGO_DB}?authSource=admin
11
12 app:
13   cors:
14     allowed-origins: ${APP_CORS_ALLOWED_ORIGINS:*}
```

- Se activa por defecto el perfil `dev` si no se define `SPRING_PROFILES_ACTIVE`.
- La URI de MongoDB se construye con variables del `.env` y el hostname `mongo` (servicio Compose).

1.14.4.2 4.2 `application-dev.yml` (desarrollo)

```
1 logging:
2   level:
3     org.springframework.web: INFO
4     org.springframework.data.mongodb: INFO
5
6 app:
7   cors:
8     allowed-origins: ${APP_CORS_ALLOWED_ORIGINS:*}
```

En desarrollo se suelen permitir orígenes más amplios y logs más verbosos.

1.14.4.3 4.3 application-prod.yml (producción)

```
1 logging:
2   level:
3     org.springframework.web: WARN
4     org.springframework.data.mongodb: WARN
5
6 app:
7   cors:
8     allowed-origins: ${APP_CORS_ALLOWED_ORIGINS:https://mi-dominio-
      ejemplo}
```

En producción:

- Se restringen orígenes permitidos a dominios concretos.
- Se moderan los logs.

1.14.5 5) Variables de entorno necesarias

En el `.env` se pueden definir variables adicionales para controlar CORS y perfil:

```
1 SPRING_PROFILES_ACTIVE=dev
2 APP_CORS_ALLOWED_ORIGINS=http://localhost:5500,http://localhost:8080
```

En desarrollo, si se usa Live Server, se añade su origen:

- `http://localhost:5500` (puede variar según configuración)

Si el frontend se sirve desde Spring (/static), bastaría con `http://localhost:8080`.

1.14.6 6) Verificación de CORS

Para comprobar CORS, se puede:

- Abrir la consola del navegador y verificar que no hay errores del tipo:
 - “blocked by CORS policy”
- Probar con `curl` simulando un origen:

```
1 curl -i http://localhost:8080/api/instalaciones \
2   -H "Origin: http://localhost:5500"
```

Debe observarse en la respuesta una cabecera `Access-Control-Allow-Origin` adecuada.

1.14.7 Resultado de este apartado

Tras este apartado el proyecto queda con:

- CORS configurado globalmente para `/api/**`.
- Control del origen permitido mediante variables de entorno (`APP_CORS_ALLOWED_ORIGINS`).
- Separación por entornos con perfiles (`dev / prod`) y YAML específico.
- Preparación para servir frontend desde un origen distinto sin problemas de navegador.

En el siguiente apartado se abordará: **“Front con Bootstrap: estructura y componentes”**, donde se definirá la estructura del cliente web y la interfaz para gestionar instalaciones, usuarios y reservas.

1.15 Front con Bootstrap: estructura y componentes

En este apartado se diseña el frontend de la aplicación como una interfaz web ligera basada en **HTML5 + Bootstrap 5**. El objetivo no es construir una SPA compleja, sino una interfaz clara y funcional que permita:

- Consultar instalaciones, usuarios y reservas.
- Crear y eliminar registros.
- Crear reservas indicando usuario, instalación y franja horaria.
- Mostrar mensajes de estado (éxito, error, validación).
- Preparar la estructura para que, en el siguiente apartado, se conecte la interfaz a la API mediante jQuery (AJAX).

Para mantener el proyecto simple y alineado con el entorno, se servirá el frontend desde Spring Boot en:

`src/main/resources/static/`

1.15.1 Estructura de archivos recomendada

Se propone separar HTML, CSS y JavaScript:

```
1 src/main/resources/static/  
2   index.html  
3   css/  
4     app.css  
5   js/  
6     app.js
```

- `index.html` contendrá el marcado y los contenedores de la interfaz.
- `app.css` incluirá pequeños ajustes de estilo (si son necesarios).
- `app.js` contendrá la lógica de interfaz y, posteriormente, las llamadas AJAX.

1.15.2 Diseño de la interfaz

La aplicación tendrá tres áreas funcionales:

1. **Instalaciones:** listado y alta rápida.
2. **Usuarios:** listado y alta rápida.
3. **Reservas:** listado con filtros y formulario de alta.

Se implementará con componentes Bootstrap estándar:

- `navbar` para navegación.
- `card` para secciones.
- `table` para listados.
- `form` para altas y filtros.
- `modal` (opcional) para edición o confirmación.
- `alert` para mensajes de éxito/error.

1.15.3 `index.html` (estructura base)

A continuación se define una versión inicial del `index.html` con Bootstrap y contenedores listos para enlazar con JavaScript. Esta versión aún no realiza peticiones a la API; únicamente prepara la estructura.

```
1 <!doctype html>
2 <html lang="es">
3 <head>
4   <meta charset="utf-8" />
5   <meta name="viewport" content="width=device-width, initial-scale=1" /
6   <title>Reservas de Pistas</title>
7
8   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/
9     bootstrap.min.css" rel="stylesheet">
10  <link rel="stylesheet" href="css/app.css">
11 </head>
12 <body class="bg-light">
13
14 <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
15   <div class="container">
16     <a class="navbar-brand" href="#">Reservas de Pistas</a>
17     <button class="navbar-toggler" type="button" data-bs-toggle="
18       collapse" data-bs-target="#nav">
19       <span class="navbar-toggler-icon"></span>
20     </button>
21
22     <div class="collapse navbar-collapse" id="nav">
23       <ul class="navbar-nav ms-auto">
24         <li class="nav-item"><a class="nav-link" href="#instalaciones">
25           Instalaciones</a></li>
26         <li class="nav-item"><a class="nav-link" href="#usuarios">
27           Usuarios</a></li>
28         <li class="nav-item"><a class="nav-link" href="#reservas">
29           Reservas</a></li>
30       </ul>
31     </div>
32   </div>
33 </nav>
```

```

29 </nav>
30
31 <main class="container py-4">
32
33   <!-- Área de alertas -->
34   <div id="alerta" class="alert d-none" role="alert"></div>
35
36   <!-- Instalaciones -->
37   <section id="instalaciones" class="mb-4">
38     <div class="card shadow-sm">
39       <div class="card-header fw-semibold">Instalaciones</div>
40       <div class="card-body">
41
42         <form id="formInstalacion" class="row g-2 align-items-end">
43           <div class="col-md-4">
44             <label class="form-label">Nombre</label>
45             <input id="instNombre" class="form-control" required>
46           </div>
47           <div class="col-md-4">
48             <label class="form-label">Dirección</label>
49             <input id="instDireccion" class="form-control" required>
50           </div>
51           <div class="col-md-3">
52             <label class="form-label">Ciudad</label>
53             <input id="instCiudad" class="form-control" required>
54           </div>
55           <div class="col-md-1 d-grid">
56             <button class="btn btn-primary" type="submit">Añadir</button>
57           </div>
58         </form>
59
60         <hr>
61
62         <div class="table-responsive">
63           <table class="table table-hover align-middle">
64             <thead>
65               <tr>
66                 <th>Nombre</th>
67                 <th>Dirección</th>
68                 <th>Ciudad</th>
69                 <th class="text-end">Acciones</th>
70               </tr>
71             </thead>
72             <tbody id="tablaInstalaciones"></tbody>
73           </table>
74         </div>
75
76       </div>
77     </div>
78   </section>

```

```

79
80 <!-- Usuarios -->
81 <section id="usuarios" class="mb-4">
82   <div class="card shadow-sm">
83     <div class="card-header fw-semibold">Usuarios</div>
84     <div class="card-body">
85
86       <form id="formUsuario" class="row g-2 align-items-end">
87         <div class="col-md-5">
88           <label class="form-label">Nombre</label>
89           <input id="userNombre" class="form-control" required>
90         </div>
91         <div class="col-md-5">
92           <label class="form-label">Email</label>
93           <input id="userEmail" type="email" class="form-control"
94             required>
95         </div>
96         <div class="col-md-2 d-grid">
97           <button class="btn btn-primary" type="submit">Añadir</
98             button>
99         </div>
100       </form>
101
102       <hr>
103
104       <div class="table-responsive">
105         <table class="table table-hover align-middle">
106           <thead>
107             <tr>
108               <th>Nombre</th>
109               <th>Email</th>
110               <th class="text-end">Acciones</th>
111             </tr>
112           </thead>
113           <tbody id="tablaUsuarios"></tbody>
114         </table>
115       </div>
116     </div>
117   </section>
118
119 <!-- Reservas -->
120 <section id="reservas">
121   <div class="card shadow-sm">
122     <div class="card-header fw-semibold">Reservas</div>
123     <div class="card-body">
124
125       <!-- Filtros -->
126       <form id="formFiltroReservas" class="row g-2 align-items-end">
127         <div class="col-md-4">

```



```

128         <label class="form-label">Usuario</label>
129         <select id="filtroUsuario" class="form-select">
130             <option value="">Todos</option>
131         </select>
132     </div>
133     <div class="col-md-4">
134         <label class="form-label">Instalación</label>
135         <select id="filtroInstalacion" class="form-select">
136             <option value="">Todas</option>
137         </select>
138     </div>
139     <div class="col-md-3">
140         <label class="form-label">Día</label>
141         <input id="filtroDia" type="date" class="form-control">
142     </div>
143     <div class="col-md-1 d-grid">
144         <button class="btn btn-outline-secondary" type="submit">
145             Filtrar</button>
146     </div>
147 </form>
148
149 <hr>
150
151 <!-- Alta de reserva -->
152 <form id="formReserva" class="row g-2 align-items-end">
153     <div class="col-md-4">
154         <label class="form-label">Usuario</label>
155         <select id="resUsuario" class="form-select" required></select>
156     </div>
157     <div class="col-md-4">
158         <label class="form-label">Instalación</label>
159         <select id="resInstalacion" class="form-select" required></select>
160     </div>
161     <div class="col-md-2">
162         <label class="form-label">Día</label>
163         <input id="resDia" type="date" class="form-control"
164             required>
165     </div>
166     <div class="col-md-1">
167         <label class="form-label">Inicio</label>
168         <input id="resHoraInicio" type="time" class="form-control"
169             required>
170     </div>
171     <div class="col-md-1">
172         <label class="form-label">Fin</label>
173         <input id="resHoraFin" type="time" class="form-control"
174             required>
175     </div>
176 </form>
177
178 <div class="col-12 d-grid mt-2">

```

```
173         <button class="btn btn-success" type="submit">Crear reserva
174         </button>
175     </div>
176 </form>
177
178 <hr>
179
180 <div class="table-responsive">
181     <table class="table table-hover align-middle">
182         <thead>
183             <tr>
184                 <th>Día</th>
185                 <th>Horario</th>
186                 <th>Instalación</th>
187                 <th>Usuario</th>
188                 <th class="text-end">Acciones</th>
189             </tr>
190         </thead>
191         <tbody id="tablaReservas"></tbody>
192     </table>
193 </div>
194
195 </div>
196 </section>
197
198 </main>
199
200 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/
201     bootstrap.bundle.min.js"></script>
202 <script src="js/app.js"></script>
203 </body>
204 </html>
```

1.15.4 css/app.css (opcional)

En general, Bootstrap es suficiente. Se recomienda evitar estilos excesivos y limitarse a pequeños ajustes:

```
1  /* app.css */
2  body {
3      min-height: 100vh;
4  }
5
6  #alerta {
7      position: sticky;
8      top: 1rem;
9      z-index: 1020;
10 }
```

1.15.5 Plantillas de tabla (contrato de renderizado)

Aunque el renderizado se implementará en el siguiente apartado con jQuery, conviene fijar desde ya qué se mostrará:

1.15.5.1 Tabla de instalaciones

- Nombre
- Dirección
- Ciudad
- Botón eliminar

1.15.5.2 Tabla de usuarios

- Nombre
- Email
- Botón eliminar

1.15.5.3 Tabla de reservas

- Día (`horario.dia`)
- Franja (`horaInicio` - `horaFin`)
- Instalación (desde snapshot embebido: `horario.instalacionSnapshot.nombre`)
- Usuario (inicialmente se mostrará `usuarioId`; se podrá resolver a nombre en el frontend si se carga la lista de usuarios)
- Botón eliminar

1.15.6 Objetivo técnico del apartado

Al finalizar este apartado se dispone de:

- Una interfaz web estática con Bootstrap.
- Formularios y tablas con identificadores claros (`id`) para conectar con JavaScript.
- Separación correcta de recursos (instalaciones, usuarios, reservas).
- Preparación para implementar el consumo real de la API mediante AJAX en el siguiente apartado.

En el siguiente apartado se implementará “**Consumo de API desde JS (jQuery)**”, que añadirá:

- Carga inicial de datos (GET).

- Altas (POST) desde formularios.
- Eliminación (DELETE).
- Filtrado de reservas usando parámetros de query.
- Gestión de errores utilizando el formato `ApiError` definido en el backend.

1.15.7 Consumo de API desde JS (jQuery)

En este apartado se implementa la lógica del cliente para consumir la API REST mediante **jQuery** y peticiones AJAX. El objetivo es que el frontend:

- Cargue los datos iniciales (instalaciones, usuarios, reservas).
- Permita crear instalaciones, usuarios y reservas desde formularios.
- Permita eliminar registros.
- Permita filtrar reservas mediante parámetros de consulta.
- Gestione correctamente errores de validación y conflictos (por ejemplo, solapes de reservas) utilizando el formato `ApiError` devuelto por la API.

Se asume la estructura creada en el apartado anterior:

- `src/main/resources/static/index.html`
- `src/main/resources/static/js/app.js`
- `src/main/resources/static/css/app.css`

Este apartado implementa `js/app.js`.

1.15.7.1 Convenciones y endpoints usados Se utilizarán los endpoints definidos previamente:

- Instalaciones: `/api/instalaciones`
- Usuarios: `/api/usuarios`
- Reservas: `/api/reservas`

Las respuestas de error seguirán el modelo:

```
1 {
2   "timestamp": "...",
3   "status": 400,
4   "error": "VALIDATION",
5   "message": "Datos no válidos",
6   "details": ["campo: mensaje", "..."]
7 }
```

1.15.7.2 Implementación del cliente: `static/js/app.js`

```
1 /* global $ */
2
3 const API = {
4   instalaciones: "/api/instalaciones",
5   usuarios: "/api/usuarios",
6   reservas: "/api/reservas"
7 };
8
```

```
9 $(document).ready(function () {
10     wireEvents();
11     cargarTodo();
12 });
13
14 /* =====
15     Eventos de formularios
16     ===== */
17
18 function wireEvents() {
19     $("#formInstalacion").on("submit", function (e) {
20         e.preventDefault();
21         crearInstalacion();
22     });
23
24     $("#formUsuario").on("submit", function (e) {
25         e.preventDefault();
26         crearUsuario();
27     });
28
29     $("#formReserva").on("submit", function (e) {
30         e.preventDefault();
31         crearReserva();
32     });
33
34     $("#formFiltroReservas").on("submit", function (e) {
35         e.preventDefault();
36         cargarReservasConFiltros();
37     });
38 }
39
40 /* =====
41     Alertas y utilidades
42     ===== */
43
44 function showAlert(type, msg) {
45     $("#alerta")
46         .removeClass("d-none alert-success alert-danger alert-warning alert
47             -info")
48         .addClass("alert-" + type)
49         .text(msg);
50     setTimeout(() => $("#alerta").addClass("d-none"), 3000);
51 }
52
53 function parseApiError(xhr, fallbackMsg) {
54     const r = xhr.responseJSON;
55     if (!r) return fallbackMsg;
56
57     if (Array.isArray(r.details) && r.details.length > 0) {
58         return `${r.message}: ${r.details.join(" | ")}`;
```

```
59     }
60     return r.message || fallbackMsg;
61 }
62
63 function escapeHtml(s) {
64     return String(s)
65         .replaceAll("&", "&amp;")
66         .replaceAll("<", "&lt;")
67         .replaceAll(">", "&gt;");
68 }
69
70 /* =====
71     Carga inicial
72     ===== */
73
74 function cargarTodo() {
75     $.when(cargarInstalaciones(), cargarUsuarios())
76         .done(function () {
77             cargarReservas();
78         })
79         .fail(function () {
80             showAlert("danger", "Error cargando datos iniciales");
81         });
82 }
83
84 /* =====
85     Instalaciones
86     ===== */
87
88 function cargarInstalaciones() {
89     return $.getJSON(API.instalaciones)
90         .done(function (data) {
91             renderInstalaciones(data);
92             rellenarSelectInstalaciones(data);
93         })
94         .fail(function (xhr) {
95             showAlert("danger", parseApiError(xhr, "Error cargando
96                 instalaciones"));
97         });
98 }
99
100 function renderInstalaciones(instalaciones) {
101     const rows = (instalaciones || []).map(function (i) {
102         return `
103             <tr>
104                 <td>${escapeHtml(i.nombre)}</td>
105                 <td>${escapeHtml(i.direccion)}</td>
106                 <td>${escapeHtml(i.ciudad)}</td>
107                 <td class="text-end">
108                     <button class="btn btn-sm btn-outline-danger" data-action="
109                         del-inst" data-id="${i.id}">
```

```

108         Eliminar
109     </button>
110 </td>
111 </tr>
112 `;
113 }).join("");
114
115 $("#tablaInstalaciones").html(rows || `<tr><td colspan="4" class="
    text-center text-muted">Sin datos</td></tr>`);
116
117 // Delegación de eventos para botones generados dinámicamente
118 $("#tablaInstalaciones button[data-action='del-inst']").off("click").
    on("click", function () {
119     const id = $(this).data("id");
120     eliminarInstalacion(id);
121 });
122 }
123
124 function rellenarSelectInstalaciones(instalaciones) {
125     const opts = (instalaciones || []).map(i =>
126         `<option value="${i.id}">${escapeHtml(i.nombre)} (${escapeHtml(i.
            ciudad)})</option>`
127     ).join("");
128
129     // Selects: filtros y alta
130     $("#filtroInstalacion").html(`<option value="">Todas</option>${opts}
        `);
131     $("#resInstalacion").html(`<option value="" disabled selected>
        Seleccione...</option>${opts}`);
132 }
133
134 function crearInstalacion() {
135     const payload = {
136         nombre: $("#instNombre").val().trim(),
137         direccion: $("#instDireccion").val().trim(),
138         ciudad: $("#instCiudad").val().trim()
139     };
140
141     $.ajax({
142         url: API.instalaciones,
143         method: "POST",
144         contentType: "application/json",
145         data: JSON.stringify(payload)
146     })
147     .done(function () {
148         showAlert("success", "Instalación creada");
149         $("#formInstalacion")[0].reset();
150         cargarInstalaciones().done(cargarReservasConFiltros);
151     })
152     .fail(function (xhr) {

```



```
153     showAlert("danger", parseApiError(xhr, "Error creando instalación
154         "));
155 }
156
157 function eliminarInstalacion(id) {
158     if (!confirm("?`Eliminar la instalación?")) return;
159
160     $.ajax({
161         url: `${API.instalaciones}/${id}`,
162         method: "DELETE"
163     })
164     .done(function () {
165         showAlert("success", "Instalación eliminada");
166         cargarInstalaciones().done(cargarReservasConFiltros);
167     })
168     .fail(function (xhr) {
169         showAlert("danger", parseApiError(xhr, "Error eliminando
170             instalación"));
171     });
172 }
173
174 /* =====
175     Usuarios
176     ===== */
177
178 function cargarUsuarios() {
179     return $.getJSON(API.usuarios)
180     .done(function (data) {
181         renderUsuarios(data);
182         rellenarSelectUsuarios(data);
183     })
184     .fail(function (xhr) {
185         showAlert("danger", parseApiError(xhr, "Error cargando usuarios")
186     );
187     });
188 }
189
190 function renderUsuarios(usuarios) {
191     const rows = (usuarios || []).map(function (u) {
192         return `
193         <tr>
194             <td>${escapeHtml(u.nombre)}</td>
195             <td>${escapeHtml(u.email)}</td>
196             <td class="text-end">
197                 <button class="btn btn-sm btn-outline-danger" data-action="
198                     del-user" data-id="${u.id}">
199                     Eliminar
200                 </button>
201             </td>
202         </tr>`
203     });
204     return rows.join("");
205 }
```

```
200     `;
201   }).join("");
202
203   $("#tablaUsuarios").html(rows || `<tr><td colspan="3" class="text-center text-muted">Sin datos</td></tr>`);
204
205   $("#tablaUsuarios button[data-action='del-user']").off("click").on("click", function () {
206     const id = $(this).data("id");
207     eliminarUsuario(id);
208   });
209 }
210
211 function rellenarSelectUsuarios(usuarios) {
212   const opts = (usuarios || []).map(u =>
213     `<option value="${u.id}">${escapeHtml(u.nombre)} (${escapeHtml(u.email)})</option>`
214   ).join("");
215
216   $("#filtroUsuario").html(`<option value="">Todos</option>${opts}`);
217   $("#resUsuario").html(`<option value="" disabled selected>Seleccione ...</option>${opts}`);
218 }
219
220 function crearUsuario() {
221   const payload = {
222     nombre: $("#userNombre").val().trim(),
223     email: $("#userEmail").val().trim()
224   };
225
226   $.ajax({
227     url: API.usuarios,
228     method: "POST",
229     contentType: "application/json",
230     data: JSON.stringify(payload)
231   })
232     .done(function () {
233       showAlert("success", "Usuario creado");
234       $("#formUsuario")[0].reset();
235       cargarUsuarios().done(cargarReservasConFiltros);
236     })
237     .fail(function (xhr) {
238       showAlert("danger", parseApiError(xhr, "Error creando usuario"));
239     });
240 }
241
242 function eliminarUsuario(id) {
243   if (!confirm(`¿Eliminar el usuario?`)) return;
244
245   $.ajax({
246     url: `${API.usuarios}/${id}`,
```

```
247     method: "DELETE"
248   })
249   .done(function () {
250     showAlert("success", "Usuario eliminado");
251     cargarUsuarios().done(cargarReservasConFiltros);
252   })
253   .fail(function (xhr) {
254     showAlert("danger", parseApiError(xhr, "Error eliminando usuario"));
255   });
256 }
257
258 /* =====
259    Reservas
260    ===== */
261
262 function cargarReservas() {
263   return $.getJSON(API.reservas)
264     .done(function (data) {
265       renderReservas(data);
266     })
267     .fail(function (xhr) {
268       showAlert("danger", parseApiError(xhr, "Error cargando reservas"));
269     });
270 }
271
272 function cargarReservasConFiltros() {
273   const usuarioId = $("#filtroUsuario").val();
274   const instalacionId = $("#filtroInstalacion").val();
275   const dia = $("#filtroDia").val();
276
277   const params = {};
278   if (usuarioId) params.usuarioId = usuarioId;
279   if (instalacionId) params.instalacionId = instalacionId;
280   if (dia) params.dia = dia;
281
282   const query = $.param(params);
283   const url = query ? `${API.reservas}?${query}` : API.reservas;
284
285   return $.getJSON(url)
286     .done(function (data) {
287       renderReservas(data);
288     })
289     .fail(function (xhr) {
290       showAlert("danger", parseApiError(xhr, "Error filtrando reservas"));
291     });
292 }
293
294 function crearReserva() {
```

```
295     const payload = {
296       usuarioId: $("#resUsuario").val(),
297       instalacionId: $("#resInstalacion").val(),
298       dia: $("#resDia").val(),
299       horaInicio: $("#resHoraInicio").val(),
300       horaFin: $("#resHoraFin").val()
301     };
302
303     $.ajax({
304       url: API.reservas,
305       method: "POST",
306       contentType: "application/json",
307       data: JSON.stringify(payload)
308     })
309     .done(function () {
310       showAlert("success", "Reserva creada");
311       $("#formReserva")[0].reset();
312       cargarReservasConFiltros();
313     })
314     .fail(function (xhr) {
315       // 409 típico por solape, 400 por validación, 404 por usuario/
316       // instalación inexistente
317       showAlert("danger", parseApiError(xhr, "Error creando reserva"));
318     });
319
320     function eliminarReserva(id) {
321       if (!confirm("? Eliminar la reserva?")) return;
322
323       $.ajax({
324         url: `${API.reservas}/${id}`,
325         method: "DELETE"
326       })
327       .done(function () {
328         showAlert("success", "Reserva eliminada");
329         cargarReservasConFiltros();
330       })
331       .fail(function (xhr) {
332         showAlert("danger", parseApiError(xhr, "Error eliminando reserva"));
333       });
334     }
335
336     function renderReservas(reservas) {
337       const usuariosMap = construirUsuariosMap();
338       const rows = (reservas || []).map(function (r) {
339         const h = r.horario || {};
340         const snap = h.instalacionSnapshot || {};
341         const usuarioNombre = usuariosMap.get(r.usuarioId) || r.usuarioId;
342
343         const dia = h.dia || "";
```

```

344     const tramo = `${h.horaInicio || ""} - ${h.horaFin || ""}`;
345     const instalacion = snap.nombre ? `${snap.nombre} (${snap.ciudad || ""})` : (snap.instalacionId || "");
346
347     return `
348         <tr>
349             <td>${escapeHtml(dia)}</td>
350             <td>${escapeHtml(tramo)}</td>
351             <td>${escapeHtml(instalacion)}</td>
352             <td>${escapeHtml(usuarioNombre)}</td>
353             <td class="text-end">
354                 <button class="btn btn-sm btn-outline-danger" data-action="
355                     del-res" data-id="${r.id}">
356                     Eliminar
357                 </button>
358             </td>
359         </tr>
360     `;
361     }).join("");
362     $("#tablaReservas").html(rows || `<tr><td colspan="5" class="text-center text-muted">Sin datos</td></tr>`);
363
364     $("#tablaReservas button[data-action='del-res']").off("click").on("
365         click", function () {
366         const id = $(this).data("id");
367         eliminarReserva(id);
368     });
369
370     /* =====
371     Mapas auxiliares
372     ===== */
373
374     function construirUsuariosMap() {
375         // Construye un mapa id -> nombre leyendo la tabla ya cargada.
376         // Alternativa: guardar el último listado en una variable global.
377         const map = new Map();
378
379         // Se intenta construir desde el select de reservas (que contiene
380         // nombre + email).
381         $("#resUsuario option").each(function () {
382             const val = $(this).attr("value");
383             const txt = $(this).text();
384             if (val) map.set(val, txt);
385         });
386
387         return map;

```

1.15.7.2.1 Explicación técnica de la implementación Carga inicial (flujo)

1. Se cargan instalaciones y usuarios (necesarios para los `select`).
2. Una vez cargados, se cargan reservas.

Esto evita que el formulario de alta de reservas aparezca sin opciones.

Gestión de errores con `ApiError`

La función `parseApiError` intenta extraer:

- `message`
- `details[]`

Así, los errores de validación se muestran con detalle, y los conflictos (409) se reportan de forma clara.

Consultas con filtros

El filtrado se realiza construyendo la query string con `$.param()` para:

- `usuarioId`
- `instalacionId`
- `dia`

1.15.7.2.2 Resultado de este apartado Al finalizar este apartado el frontend queda plenamente funcional:

- CRUD básico de instalaciones (crear/listar/eliminar).
- CRUD básico de usuarios (crear/listar/eliminar).
- CRUD básico de reservas (crear/listar/eliminar).
- Filtros sobre reservas.
- Mensajes de error consistentes.

En el siguiente apartado se desarrollará **“Caso práctico completo (paso a paso)”**, donde se integrará todo el flujo en orden: creación de datos base, creación de reservas, pruebas con Postman, verificación desde la interfaz web y comprobación de las reglas de negocio (incluyendo el conflicto por solape).

1.16 Caso práctico completo (paso a paso)

En este apartado se recorre **todo el flujo funcional de la aplicación**, desde el arranque del entorno hasta la verificación final del sistema, siguiendo un orden lógico y reproducible. El objetivo es que el alumnado pueda **comprobar que todos los criterios de evaluación se cumplen** mediante un caso real y completo.

El caso práctico se desarrolla en **tres niveles**:

1. Verificación técnica del entorno.
 2. Uso de la API REST (backend).
 3. Uso de la interfaz web (frontend).
-

1.17 Paso 1: Arranque del entorno

1.17.1 1.1 Abrir el proyecto en Dev Container

1. Abrir la carpeta del proyecto en VS Code.
2. Seleccionar **“Reopen in Container”**.
3. Esperar a que:
 - Se levanten los contenedores (`app`, `mongo`, `mongo-express`).
 - El contenedor de desarrollo esté operativo.

1.17.2 1.2 Verificar servicios

Desde el terminal del contenedor:

```
1 docker ps
```

Debe aparecer:

- Un contenedor MongoDB en ejecución.
- Un contenedor Mongo Express en ejecución.

Accesos:

- Mongo Express: <http://localhost:8081>
- API (cuando arranque Spring): <http://localhost:8080>

1.18 Paso 2: Arranque del backend Spring Boot

Desde el terminal integrado del contenedor:

```
1 ./mvnw spring-boot:run
```

Verificaciones en el log:

- Spring Boot arranca sin errores.
- Conexión correcta a MongoDB.
- API escuchando en el puerto 8080.

Este paso valida:

- **b)** Establecimiento de conexiones.
 - **h)** Correcto despliegue de la aplicación.
-

1.19 Paso 3: Creación de datos base (API REST)

Antes de usar el frontend, se crean datos base mediante la API. Esto permite comprobar el funcionamiento independiente del backend.

1.19.1 3.1 Crear instalaciones

POST `/api/instalaciones`

```
1 {
2   "nombre": "Pista Central",
3   "direccion": "C/ Mayor 1",
4   "ciudad": "Jaén"
5 }
```

Crear al menos dos instalaciones (por ejemplo, una pista cubierta y una al aire libre).

Verificar:

- Código HTTP **201 Created**.
- Documento almacenado en la colección `instalaciones`.

Criterios evaluados:

- **c)** Persistencia de objetos simples.
 - **e)** Consultas básicas (GET).
-

1.19.2 3.2 Crear usuarios

POST `/api/usuarios`

```
1 {  
2   "nombre": "Ana Pérez",  
3   "email": "ana@ejemplo.com"  
4 }
```

Crear al menos dos usuarios distintos.

Verificar:

- Código HTTP **201 Created**.
- Documentos en la colección `usuarios`.

Criterios evaluados:

- **c)** Persistencia de objetos simples.
 - **e)** Consultas.
-

1.20 Paso 4: Creación de reservas (regla de negocio)

1.20.1 4.1 Crear una reserva válida

POST `/api/reservas`

```
1 {  
2   "usuarioId": "ID_USUARIO",  
3   "instalacionId": "ID_INSTALACION",  
4   "dia": "2026-01-20",  
5   "horaInicio": "18:00",  
6   "horaFin": "19:00"  
7 }
```

Verificar:

- Código HTTP **201 Created**.
- Documento en `reservas` con:
 - `horario` embebido.
 - `instalacionSnapshot` embebido.
 - `usuarioId` como referencia.

Criterios evaluados:

- **d)** Persistencia de objetos estructurados.
 - **e)** Consultas sobre datos embebidos.
-

1.20.2 4.2 Intentar una reserva solapada (conflicto)

POST `/api/reservas`

```
1 {  
2   "usuarioId": "OTRO_USUARIO",  
3   "instalacionId": "MISMA_INSTALACION",  
4   "dia": "2026-01-20",  
5   "horaInicio": "18:30",  
6   "horaFin": "19:30"  
7 }
```

Resultado esperado:

- Código HTTP **409 Conflict**.
- Respuesta `ApiError` indicando solape de reservas.

Este paso valida:

- **f)** Modificación/gestión de objetos almacenados.
 - **g)** Gestión de transacciones a nivel lógico (control de integridad).
-

1.21 Paso 5: Consultas avanzadas

1.21.1 5.1 Consultar reservas por instalación y día

GET `/api/reservas?instalacionId=...&dia=2026-01-20`

Verificar:

- Se devuelven solo las reservas correspondientes.
- No aparecen reservas de otras instalaciones o días.

1.21.2 5.2 Consultar reservas por usuario

GET `/api/reservas?usuarioId=...`

Verificar:

- Se devuelven solo las reservas del usuario indicado.

Criterios evaluados:

- **e)** Desarrollo de consultas.
- Uso de filtros y parámetros de consulta.

1.22 Paso 6: Uso del frontend (flujo completo)

Acceder desde el navegador a:

```
1 http://localhost:8080
```

1.22.1 6.1 Gestión de instalaciones

- Crear una nueva instalación desde el formulario.
- Verla reflejada en la tabla.
- Eliminarla y comprobar que desaparece.

1.22.2 6.2 Gestión de usuarios

- Crear un nuevo usuario.
- Verlo en la tabla y en los desplegados de reservas.
- Eliminarlo (si no tiene reservas asociadas).

1.22.3 6.3 Gestión de reservas

- Crear una reserva desde la interfaz.
- Comprobar que aparece en la tabla.
- Intentar crear una reserva solapada y observar el mensaje de error.
- Filtrar reservas por:
 - Usuario
 - Instalación
 - Día

Este paso valida:

- Integración frontend-backend.
- Gestión de errores desde el cliente.
- Consumo correcto de la API REST.

1.23 Paso 7: Verificación en Mongo Express

Acceder a <http://localhost:8081> y comprobar:

- Colecciones creadas automáticamente:
 - `instalaciones`
 - `usuarios`
 - `reservas`
- Estructura de los documentos:
 - Uso de `_id` (ObjectId).
 - Documentos embebidos (`horario`, `instalacionSnapshot`).
 - Referencias (`usuarioId`).

Este paso refuerza:

- **a)** Comprensión del modelo documental.
 - Diferencias frente a bases de datos relacionales.
-

1.24 Paso 8: Documentación y conclusiones

Para cerrar el caso práctico, el alumnado debe documentar:

- Estructura del proyecto.
- Decisiones de modelado (embebido vs referencia).
- Endpoints implementados.
- Ejemplo de error 409 por solape.
- Capturas de:
 - API funcionando.
 - Interfaz web.
 - Documentos en MongoDB.

Criterio evaluado:

- **h)** Pruebas y documentación de la aplicación desarrollada.

1.25 Resultado final del caso práctico

Al finalizar este apartado se ha demostrado, de forma práctica y verificable, que la aplicación:

- Usa MongoDB como base de datos documental.
- Gestiona objetos simples y estructurados.
- Implementa reglas de negocio reales.
- Expone una API REST coherente.
- Integra frontend y backend correctamente.
- Está correctamente probada y documentada.

En el siguiente y último apartado se presentarán **extensiones y mejoras** (paginación, búsqueda avanzada, logs, Docker, seguridad), que pueden utilizarse como ampliación, reto o trabajo voluntario.

1.26 Spring Security, login y ACLs contra MongoDB

En este apartado vamos a añadir Spring Security, login con usuarios en MongoDB y autorización por roles a nuestro proyecto.

En esta mejora se incorpora **Spring Security** para que **todas las operaciones** del sistema requieran identificación y se aplica **control de acceso** según el tipo de usuario:

- **ADMIN**

- CRUD de **usuarios, instalaciones y horarios disponibles**.

- **CONSERJE**

- Solo puede ver **las pistas reservadas del día actual**, sin conocer el usuario que reserva.

- **CLIENTE**

- Solo puede ver **sus reservas**.
- Puede **modificar o borrar** una reserva **si aún no ha pasado** (fecha/hora futura).
- Puede consultar **pistas disponibles** para una fecha y franja horaria para poder reservar.

La autenticación se realizará con **usuarios almacenados en MongoDB** (colección **usuarios**) usando como “username” el **email**, y contraseñas guardadas como **hash BCrypt**.

A continuación detallamos **las modificaciones del backend**: **pom.xml**, modelo, repositorios, seguridad, DTOs mínimos de seguridad y controladores/servicios necesarios para cumplir las reglas.

1.26.1 1) pom.xml: dependencias nuevas

Añadir Spring Security:

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-security</artifactId>
4 </dependency>
```

(¡Ojo! se mantiene Web, Data MongoDB y Validation.)

1.26.2 2) Modelo: añadir rol y contraseña al usuario

1.26.2.1 2.1 Enum de rol Para implementar los tipos de usuarios podemos hacerlo mediante una tabla y dejarlo abierto (más complicado para el backend) o bien mediante un enum de Java y dejarlo más atado. En nuestro caso usaremos **enum**, así creamos el archivo **model/RolUsuario.java**:

```
1 package com.dam.reservas.model;  
2  
3 public enum RolUsuario {  
4     ADMIN, CONSERJE, CLIENTE  
5 }
```

1.26.2.2 2.2 Modificar Usuario Cambios:

- añadir passwordHash (BCrypt)
- añadir rol

model/Usuario.java

```
1 package com.dam.reservas.model;  
2  
3 import jakarta.validation.constraints.Email;  
4 import jakarta.validation.constraints.NotBlank;  
5 import jakarta.validation.constraints.NotNull;  
6 import org.springframework.data.annotation.Id;  
7 import org.springframework.data.mongodb.core.mapping.Document;  
8  
9 @Document(collection = "usuarios")  
10 public class Usuario {  
11  
12     @Id  
13     private String id;  
14  
15     @NotBlank(message = "nombre es obligatorio")  
16     private String nombre;  
17  
18     @NotBlank(message = "email es obligatorio")  
19     @Email(message = "email no válido")  
20     private String email;  
21  
22     // Guardar SIEMPRE hash BCrypt, nunca texto plano  
23     @NotBlank(message = "passwordHash es obligatorio")  
24     private String passwordHash;  
25  
26     @NotNull(message = "rol es obligatorio")  
27     private RolUsuario rol;  
28  
29     public Usuario() {}  
30  
31     public Usuario(String nombre, String email, String passwordHash,  
32         RolUsuario rol) {  
33         this.nombre = nombre;  
34         this.email = email;  
35         this.passwordHash = passwordHash;  
36         this.rol = rol;  
37     }  
38 }
```

```
36     }
37
38     public String getId() { return id; }
39     public void setId(String id) { this.id = id; }
40
41     public String getNombre() { return nombre; }
42     public void setNombre(String nombre) { this.nombre = nombre; }
43
44     public String getEmail() { return email; }
45     public void setEmail(String email) { this.email = email; }
46
47     public String getPasswordHash() { return passwordHash; }
48     public void setPasswordHash(String passwordHash) { this.passwordHash
        = passwordHash; }
49
50     public RolUsuario getRol() { return rol; }
51     public void setRol(RolUsuario rol) { this.rol = rol; }
52 }
```

1.26.3 3) Repositorio de usuarios: login por email

repo/UsuarioRepository.java

```
1 package com.dam.reservas.repo;
2
3 import com.dam.reservas.model.Usuario;
4 import org.springframework.data.mongodb.repository.MongoRepository;
5
6 import java.util.Optional;
7
8 public interface UsuarioRepository extends MongoRepository<Usuario,
    String> {
9     Optional<Usuario> findByEmailIgnoreCase(String email);
10    boolean existsByEmailIgnoreCase(String email);
11 }
```

1.26.4 4) Seguridad: UserDetailsService + PasswordEncoder + configuración HTTP

1.26.4.1 4.1 Principal propio (incluye id y rol) security/AuthUser.java

```
1 package com.dam.reservas.security;
2
3 import com.dam.reservas.model.RolUsuario;
4 import org.springframework.security.core.GrantedAuthority;
5 import org.springframework.security.core.authority.
    SimpleGrantedAuthority;
6 import org.springframework.security.core.userdetails.UserDetails;
```



```

7
8 import java.util.Collection;
9 import java.util.List;
10
11 public class AuthUser implements UserDetails {
12
13     private final String id;
14     private final String email;
15     private final String passwordHash;
16     private final RolUsuario rol;
17
18     public AuthUser(String id, String email, String passwordHash,
19         RolUsuario rol) {
20         this.id = id;
21         this.email = email;
22         this.passwordHash = passwordHash;
23         this.rol = rol;
24     }
25
26     public String getId() { return id; }
27     public RolUsuario getRol() { return rol; }
28
29     @Override
30     public Collection<? extends GrantedAuthority> getAuthorities() {
31         // Spring Security usa convención ROLE_*
32         return List.of(new SimpleGrantedAuthority("ROLE_" + rol.name()));
33     }
34
35     @Override
36     public String getPassword() { return passwordHash; }
37
38     @Override
39     public String getUsername() { return email; }
40
41     @Override public boolean isAccountNonExpired() { return true; }
42     @Override public boolean isAccountNonLocked() { return true; }
43     @Override public boolean isCredentialsNonExpired() { return true; }
44     @Override public boolean isEnabled() { return true; }
45 }

```

1.26.4.2 4.2 UserDetailsService cargando desde MongoDB `security/MongoUserDetailsService.java`

```

1 package com.dam.reservas.security;
2
3 import com.dam.reservas.model.Usuario;
4 import com.dam.reservas.repo.UsuarioRepository;
5 import org.springframework.security.core.userdetails.*;
6 import org.springframework.stereotype.Service;

```

```

7
8 @Service
9 public class MongoUserDetailsService implements UserDetailsService {
10
11     private final UsuarioRepository usuarioRepo;
12
13     public MongoUserDetailsService(UsuarioRepository usuarioRepo) {
14         this.usuarioRepo = usuarioRepo;
15     }
16
17     @Override
18     public UserDetails loadUserByUsername(String username) throws
        UsernameNotFoundException {
19         Usuario u = usuarioRepo.findByEmailIgnoreCase(username)
20             .orElseThrow(() -> new UsernameNotFoundException("Usuario no
                encontrado"));
21         return new AuthUser(u.getId(), u.getEmail(), u.getPasswordHash(), u
            .getRol());
22     }
23 }

```

1.26.4.3 4.3 Configuración de Spring Security Se propone un enfoque didáctico y práctico:

- Permitir recursos estáticos (/, /index.html, /js/**, /css/**) sin login.
- Proteger toda la API (/api/**) exigiendo autenticación.
- Usar `formLogin()` (login por sesión/cookie).
- Desactivar CSRF para simplificar POST/PUT/DELETE desde jQuery (en producción se haría de otra forma).

security/SecurityConfig.java

```

1 package com.dam.reservas.security;
2
3 import org.springframework.context.annotation.*;
4 import org.springframework.security.config.annotation.method.
    configuration.EnableMethodSecurity;
5 import org.springframework.security.config.annotation.web.builders.
    HttpSecurity;
6 import org.springframework.security.crypto.bcrypt.*;
7 import org.springframework.security.crypto.password.PasswordEncoder;
8 import org.springframework.security.web.SecurityFilterChain;
9
10 @Configuration
11 @EnableMethodSecurity // habilita @PreAuthorize
12 public class SecurityConfig {
13
14     @Bean
15     public PasswordEncoder passwordEncoder() {

```

```

16     return new BCryptPasswordEncoder();
17 }
18
19 @Bean
20 public SecurityFilterChain filterChain(HttpSecurity http) throws
    Exception {
21     http
22         .csrf(csrf -> csrf.disable())
23         .authorizeHttpRequests(auth -> auth
24             .requestMatchers(
25                 "/", "/index.html",
26                 "/css/**", "/js/**",
27                 "/favicon.ico"
28             ).permitAll()
29             .requestMatchers("/api/**").authenticated()
30             .anyRequest().permitAll()
31         )
32         .formLogin(form -> form.permitAll())
33         .logout(logout -> logout.permitAll());
34
35     return http.build();
36 }
37 }

```

1.26.5 5) Nuevo documento: Horarios disponibles (para que ADMIN gestione horarios)

Como en el modelo original el `Horario` estaba embebido en `Reserva`, para poder cumplir “CRUD de horarios” se introduce un documento explícito de disponibilidad.

1.26.5.1 5.1 Modelo `HorarioDisponible` `model/HorarioDisponible.java`

```

1 package com.dam.reservas.model;
2
3 import jakarta.validation.constraints.NotNull;
4 import org.springframework.data.annotation.Id;
5 import org.springframework.data.mongodb.core.mapping.Document;
6
7 import java.time.LocalDate;
8 import java.time.LocalDateTime;
9
10 @Document(collection = "horarios")
11 public class HorarioDisponible {
12
13     @Id
14     private String id;
15
16     @NotNull
17     private String instalacionId;

```

```
18
19     @NotNull
20     private LocalDate dia;
21
22     @NotNull
23     private LocalTime horaInicio;
24
25     @NotNull
26     private LocalTime horaFin;
27
28     public HorarioDisponible() {}
29
30     public HorarioDisponible(String instalacionId, LocalDate dia,
31                               LocalTime horaInicio, LocalTime horaFin) {
32         this.instalacionId = instalacionId;
33         this.dia = dia;
34         this.horaInicio = horaInicio;
35         this.horaFin = horaFin;
36     }
37
38     public String getId() { return id; }
39     public void setId(String id) { this.id = id; }
40
41     public String getInstalacionId() { return instalacionId; }
42     public void setInstalacionId(String instalacionId) { this.
43         instalacionId = instalacionId; }
44
45     public LocalDate getDia() { return dia; }
46     public void setDia(LocalDate dia) { this.dia = dia; }
47
48     public LocalTime getHoraInicio() { return horaInicio; }
49     public void setHoraInicio(LocalTime horaInicio) { this.horaInicio =
50         horaInicio; }
51
52     public LocalTime getHoraFin() { return horaFin; }
53     public void setHoraFin(LocalTime horaFin) { this.horaFin = horaFin; }
54 }
```

1.26.5.2 5.2 Repositorio repo/HorarioDisponibleRepository.java

```
1 package com.dam.reservas.repo;
2
3 import com.dam.reservas.model.HorarioDisponible;
4 import org.springframework.data.mongodb.repository.MongoRepository;
5
6 import java.time.LocalDate;
7 import java.util.List;
8
9 public interface HorarioDisponibleRepository extends MongoRepository<
10     HorarioDisponible, String> {
```

```
10 List<HorarioDisponible> findByInstalacionIdAndDia(String  
    instalacionId, LocalDate dia);  
11 }
```

1.26.6 6) Reglas de negocio y endpoints por rol

Para mantener claro el control de acceso se definen rutas por “área”:

- `/api/admin/**` → ADMIN
- `/api/conserje/**` → CONSERJE
- `/api/cliente/**` → CLIENTE

Y se aplican permisos con `@PreAuthorize`.

1.26.6.1 6.1 DTOs mínimos necesarios

1.26.6.1.1 Crear usuario (ADMIN) con contraseña en claro (solo en request) El backend la convierte a `passwordHash`.

`dto/AdminUsuarioCreateRequest.java`

```
1 package com.dam.reservas.dto;  
2  
3 import com.dam.reservas.model.RolUsuario;  
4 import jakarta.validation.constraints.*;  
5  
6 public record AdminUsuarioCreateRequest(  
7     @NotBlank String nombre,  
8     @NotBlank @Email String email,  
9     @NotBlank @Size(min = 6, message = "password mínimo 6 caracteres")  
10    String password,  
11    @NotNull RolUsuario rol  
12 ) {}
```

`dto/AdminUsuarioResponse.java`

```
1 package com.dam.reservas.dto;  
2  
3 import com.dam.reservas.model.RolUsuario;  
4  
5 public record AdminUsuarioResponse(  
6     String id,  
7     String nombre,  
8     String email,  
9     RolUsuario rol  
10 ) {}
```

1.26.6.1.2 Reserva (CLIENTE): el cliente NO envía usuarioid `dto/ClienteReservaCreateRequest.java`

```
1 package com.dam.reservas.dto;
2
3 import jakarta.validation.constraints.NotBlank;
4 import jakarta.validation.constraints.NotNull;
5
6 import java.time.LocalDate;
7 import java.time.LocalTime;
8
9 public record ClienteReservaCreateRequest(
10     @NotBlank String instalacionId,
11     @NotNull LocalDate dia,
12     @NotNull LocalTime horaInicio,
13     @NotNull LocalTime horaFin
14 ) {}
```

1.26.6.1.3 Disponibilidad (CLIENTE): instalaciones libres para una franja `dto/DisponibilidadRequest.java`

```
1 package com.dam.reservas.dto;
2
3 import jakarta.validation.constraints.NotNull;
4 import java.time.LocalDate;
5 import java.time.LocalTime;
6
7 public record DisponibilidadRequest(
8     @NotNull LocalDate dia,
9     @NotNull LocalTime horaInicio,
10    @NotNull LocalTime horaFin
11 ) {}
```

1.26.7 7) Servicios: garantizar reglas “solo sus reservas” y “no pasada”

1.26.7.1 7.1 Utilidad para “reserva pasada” Regla: una reserva se considera “pasada” si el día es anterior a hoy, o si es hoy y la horaFin ya ha pasado.

Se aplica en update/delete del cliente.

1.26.7.2 7.2 ReservaRepository: consultas necesarias Asegúrate de tener estas consultas (o equivalentes):

`repo/ReservaRepository.java` (añadir)

```
1 List<Reserva> findByUsuarioIdAndHorario_Dia(String usuarioId, java.time
    .LocalDate dia);
2 List<Reserva> findByHorario_Dia(java.time.LocalDate dia);
```

Ya tenías otras por instalación y día, útiles para solapes.

1.26.8 8) Controladores por rol (backend)

1.26.8.1 8.1 Admin: CRUD de usuarios, instalaciones y horarios

1.26.8.1.1 AdminUsuariosController web/AdminUsuariosController.java

```
1 package com.dam.reservas.web;
2
3 import com.dam.reservas.dto.*;
4 import com.dam.reservas.model.Usuario;
5 import com.dam.reservas.repo.UsuarioRepository;
6 import jakarta.validation.Valid;
7 import org.springframework.security.access.prepost.PreAuthorize;
8 import org.springframework.security.crypto.password.PasswordEncoder;
9 import org.springframework.web.bind.annotation.*;
10
11 import java.util.List;
12
13 @RestController
14 @RequestMapping("/api/admin/usuarios")
15 @PreAuthorize("hasRole('ADMIN')")
16 public class AdminUsuariosController {
17
18     private final UsuarioRepository repo;
19     private final PasswordEncoder encoder;
20
21     public AdminUsuariosController(UsuarioRepository repo,
22         PasswordEncoder encoder) {
23         this.repo = repo;
24         this.encoder = encoder;
25     }
26
27     @GetMapping
28     public List<AdminUsuarioResponse> listar() {
29         return repo.findAll().stream()
30             .map(u -> new AdminUsuarioResponse(u.getId(), u.getNombre(), u.
31                 getEmail(), u.getRol()))
32             .toList();
33
34     @PostMapping
```

```

34 public AdminUsuarioResponse crear(@Valid @RequestBody
    AdminUsuarioCreateRequest req) {
35     if (repo.existsByEmailIgnoreCase(req.email())) {
36         throw new ConflictException("Ya existe un usuario con ese email")
            ;
37     }
38     Usuario u = new Usuario(req.nombre(), req.email(), encoder.encode(
        req.password()), req.rol());
39     u = repo.save(u);
40     return new AdminUsuarioResponse(u.getId(), u.getNombre(), u.
        getEmail(), u.getRol());
41 }
42
43 @DeleteMapping("/{id}")
44 public void borrar(@PathVariable String id) {
45     if (!repo.existsById(id)) throw new NotFoundException("Usuario no
        encontrado: " + id);
46     repo.deleteById(id);
47 }
48 }

```

Para instalaciones y horarios disponibles, el patrón es el mismo (`@PreAuthorize("hasRole('ADMIN')")`) reutilizando repositorios.

1.26.8.2 8.2 Conserje: ver reservas del día (sin usuario) Endpoint:

- GET /api/conserje/reservas-hoy

Devuelve: día, tramo, instalación (snapshot) — sin `usuarioId`.

dto/ConserjeReservaHoyResponse.java

```

1 package com.dam.reservas.dto;
2
3 public record ConserjeReservaHoyResponse(
4     String instalacionNombre,
5     String instalacionCiudad,
6     String horaInicio,
7     String horaFin
8 ) {}

```

web/ConserjeController.java

```

1 package com.dam.reservas.web;
2
3 import com.dam.reservas.dto.ConserjeReservaHoyResponse;
4 import com.dam.reservas.repo.ReservaRepository;
5 import org.springframework.security.access.prepost.PreAuthorize;
6 import org.springframework.web.bind.annotation.*;

```



```

7
8 import java.time.LocalDate;
9 import java.util.List;
10
11 @RestController
12 @RequestMapping("/api/conserje")
13 @PreAuthorize("hasRole('CONSERJE')")
14 public class ConserjeController {
15
16     private final ReservaRepository reservaRepo;
17
18     public ConserjeController(ReservaRepository reservaRepo) {
19         this.reservaRepo = reservaRepo;
20     }
21
22     @GetMapping("/reservas-hoy")
23     public List<ConserjeReservaHoyResponse> reservasHoy() {
24         LocalDate hoy = LocalDate.now();
25         return reservaRepo.findByHorario_Dia(hoy).stream()
26             .map(r -> new ConserjeReservaHoyResponse(
27                 r.getHorario().getInstalacionSnapshot().getNombre(),
28                 r.getHorario().getInstalacionSnapshot().getCiudad(),
29                 r.getHorario().getHoraInicio().toString(),
30                 r.getHorario().getHoraFin().toString()
31             ))
32             .toList();
33     }
34 }

```

1.26.8.3 8.3 Cliente: solo sus reservas + disponibilidad + modificar/borrar si no ha pasado Para obtener el usuario autenticado con su `id`, se usa `@AuthenticationPrincipal AuthUser`.

`web/ClienteReservasController.java`

```

1 package com.dam.reservas.web;
2
3 import com.dam.reservas.dto.ClienteReservaCreateRequest;
4 import com.dam.reservas.dto.DisponibilidadRequest;
5 import com.dam.reservas.model.*;
6 import com.dam.reservas.repo.*;
7 import com.dam.reservas.security.AuthUser;
8 import jakarta.validation.Valid;
9 import org.springframework.security.access.prepost.PreAuthorize;
10 import org.springframework.security.core.annotation.
    AuthenticationPrincipal;
11 import org.springframework.web.bind.annotation.*;
12
13 import java.time.*;
14 import java.util.List;
15

```

```
16 @RestController
17 @RequestMapping("/api/cliente")
18 @PreAuthorize("hasRole('CLIENTE')")
19 public class ClienteReservasController {
20
21     private final ReservaRepository reservaRepo;
22     private final InstalacionRepository instalacionRepo;
23
24     public ClienteReservasController(ReservaRepository reservaRepo,
25         InstalacionRepository instalacionRepo) {
26         this.reservaRepo = reservaRepo;
27         this.instalacionRepo = instalacionRepo;
28     }
29
30     @GetMapping("/reservas")
31     public List<Reserva> misReservas(@AuthenticationPrincipal AuthUser
32         auth) {
33         return reservaRepo.findByUsuarioId(auth.getId());
34     }
35
36     @PostMapping("/reservas")
37     public Reserva crearReserva(@AuthenticationPrincipal AuthUser auth,
38         @Valid @RequestBody
39         ClienteReservaCreateRequest req) {
40
41         if (!req.horaInicio().isBefore(req.horaFin())) {
42             throw new BadRequestException("El rango horario no es válido (
43                 horaInicio debe ser anterior a horaFin)");
44         }
45
46         Instalacion inst = instalacionRepo.findById(req.instalacionId())
47             .orElseThrow(() -> new NotFoundException("Instalación no
48                 encontrada: " + req.instalacionId()));
49
50         // Comprobar solape en esa instalación/día
51         List<Reserva> delDia = reservaRepo
52             .findByHorario_InstalacionSnapshot_InstalacionIdAndHorario_Dia(
53                 inst.getId(), req.dia());
54
55         boolean solapa = delDia.stream().anyMatch(r ->
56             r.getHorario().getHoraInicio().isBefore(req.horaFin())
57             && req.horaInicio().isBefore(r.getHorario().getHoraFin())
58         );
59         if (solapa) throw new ConflictException("Existe una reserva que
60             solapa en esa instalación y franja horaria");
61
62         InstalacionSnapshot snap = new InstalacionSnapshot(inst.getId(),
63             inst.getNombre(), inst.getDireccion(), inst.getCiudad());
64         Horario horario = new Horario(req.dia(), req.horaInicio(), req.
65             horaFin(), snap);
66     }
67 }
```

```
58     Reserva reserva = new Reserva(Instant.now(), horario, auth.getId())
59     ;
60     return reservaRepo.save(reserva);
61 }
62 @DeleteMapping("/reservas/{id}")
63 public void borrarReserva(@AuthenticationPrincipal AuthUser auth,
64     @PathVariable String id) {
65     Reserva r = reservaRepo.findById(id).orElseThrow(() -> new
66         NotFoundException("Reserva no encontrada: " + id));
67     if (!r.getUsuarioId().equals(auth.getId())) throw new
68         NotFoundException("Reserva no encontrada: " + id);
69     if (reservaYaPasada(r)) throw new ConflictException("No se puede
70         borrar una reserva pasada");
71     reservaRepo.deleteById(id);
72 }
73
74 @PutMapping("/reservas/{id}")
75 public Reserva modificarReserva(@AuthenticationPrincipal AuthUser
76     auth,
77     @PathVariable String id,
78     @Valid @RequestBody
79         ClienteReservaCreateRequest req) {
80     Reserva r = reservaRepo.findById(id).orElseThrow(() -> new
81         NotFoundException("Reserva no encontrada: " + id));
82     if (!r.getUsuarioId().equals(auth.getId())) throw new
83         NotFoundException("Reserva no encontrada: " + id);
84     if (reservaYaPasada(r)) throw new ConflictException("No se puede
85         modificar una reserva pasada");
86     if (!req.horaInicio().isBefore(req.horaFin())) {
87         throw new BadRequestException("El rango horario no es válido (
88             horaInicio debe ser anterior a horaFin)");
89     }
90
91     Instalacion inst = instalacionRepo.findById(req.instalacionId())
92         .orElseThrow(() -> new NotFoundException("Instalación no
93             encontrada: " + req.instalacionId()));
94
95     // Solape excluyendo la reserva actual
96     List<Reserva> delDia = reservaRepo
97         .findByHorario_InstalacionSnapshot_InstalacionIdAndHorario_Dia(
98             inst.getId(), req.dia());
99
100     boolean solapa = delDia.stream()
101         .filter(x -> !x.getId().equals(r.getId()))
102         .anyMatch(x -> x.getHorario().getHoraInicio().isBefore(req.
103             horaFin())
104             && req.horaInicio().isBefore(x.getHorario().getHoraFin()));
105
106     if (solapa) throw new ConflictException("Existe una reserva que
107         solapa en esa instalación y franja horaria");
```

```
94
95     InstalacionSnapshot snap = new InstalacionSnapshot(inst.getId(),
96         inst.getNombre(), inst.getDireccion(), inst.getCiudad());
97     r.setHorario(new Horario(req.dia(), req.horaInicio(), req.horaFin()
98         , snap));
99     return reservaRepo.save(r);
100 }
101
102 // Instalaciones disponibles para una fecha y franja dada
103 @PostMapping("/disponibilidad")
104 public List<Instalacion> instalacionesDisponibles(@Valid @RequestBody
105     DisponibilidadRequest req) {
106     if (!req.horaInicio().isBefore(req.horaFin())) {
107         throw new BadRequestException("El rango horario no es válido (
108             horaInicio debe ser anterior a horaFin)");
109     }
110
111     List<Instalacion> instalaciones = instalacionRepo.findAll();
112     List<Reserva> reservasDelDia = reservaRepo.findByHorario_Dia(req.
113         dia());
114
115     return instalaciones.stream().filter(inst -> {
116         List<Reserva> deInst = reservasDelDia.stream()
117             .filter(r -> inst.getId().equals(r.getHorario().
118                 getInstalacionSnapshot().getInstalacionId()))
119             .toList();
120
121         boolean solapa = deInst.stream().anyMatch(r ->
122             r.getHorario().getHoraInicio().isBefore(req.horaFin())
123             && req.horaInicio().isBefore(r.getHorario().getHoraFin())
124         );
125
126         return !solapa;
127     }).toList();
128 }
129
130 private boolean reservaYaPasada(Reserva r) {
131     LocalDate dia = r.getHorario().getDia();
132     LocalTime fin = r.getHorario().getHoraFin();
133
134     LocalDate hoy = LocalDate.now();
135     LocalTime ahora = LocalTime.now();
136
137     if (dia.isBefore(hoy)) return true;
138     if (dia.isAfter(hoy)) return false;
139     return !fin.isAfter(ahora); // hoy: pasada si fin <= ahora
140 }
141 }
```

1.26.9 9) Endpoint útil: “quién soy” (para frontend)

Para que el frontend pueda saber el rol y adaptar menús/pantallas:

`web/AuthController.java`

```
1 package com.dam.reservas.web;
2
3 import com.dam.reservas.security.AuthUser;
4 import org.springframework.security.core.annotation.
    AuthenticationPrincipal;
5 import org.springframework.web.bind.annotation.*;
6
7 @RestController
8 @RequestMapping("/api/auth")
9 public class AuthController {
10
11     @GetMapping("/me")
12     public Object me(@AuthenticationPrincipal AuthUser auth) {
13         return new Object() {
14             public final String id = auth.getId();
15             public final String email = auth.getUsername();
16             public final String rol = auth.getRol().name();
17         };
18     }
19 }
```

1.26.10 Resumen de lo que queda implementado en backend

- Autenticación con **usuarios de MongoDB** (email + BCrypt).
- Autorización por roles con `@PreAuthorize`:
 - ADMIN: `/api/admin/**`
 - CONSERJE: `/api/conserje/**`
 - CLIENTE: `/api/cliente/**`
- Cliente:
 - solo ve sus reservas
 - modifica/borra solo si no está pasada
 - consulta instalaciones disponibles para un día/franja
- Conserje:
 - ve reservas del día sin datos de usuario

- Admin:
 - CRUD de usuarios (y preparado para extender a instalaciones y horarios)

1.26.11 1) DTOs ADMIN

A continuación completamos los endpoints y lógica para **el rol ADMIN** con CRUD completo de:

- **Usuarios** (con rol y contraseña, guardando BCrypt)
- **Instalaciones**
- **Horarios disponibles** (documento [horarios](#))

Incluiremos **DTOs**, **controladores** y la **lógica mínima** de validación/reglas (email único, rango horario válido). Usamos el patrón `/api/admin/**` protegido con `@PreAuthorize("hasRole('ADMIN'))`.

1.26.11.1 1.1 Usuarios (ADMIN) Ya tenías `AdminUsuarioCreateRequest` y `AdminUsuarioResponse`. Añadimos DTO de actualización (permite cambiar nombre, email, rol y, opcionalmente, password).

`dto/AdminUsuarioUpdateRequest.java`

```
1 package com.dam.reservas.dto;
2
3 import com.dam.reservas.model.RolUsuario;
4 import jakarta.validation.constraints.Email;
5 import jakarta.validation.constraints.NotBlank;
6 import jakarta.validation.constraints.NotNull;
7 import jakarta.validation.constraints.Size;
8
9 public record AdminUsuarioUpdateRequest(
10     @NotBlank String nombre,
11     @NotBlank @Email String email,
12     @NotNull RolUsuario rol,
13     @Size(min = 6, message = "password mínimo 6 caracteres") String
14         password // opcional (puede ser null o "")
15 ) {}
```

1.26.11.2 1.2 Instalaciones (ADMIN) `dto/AdminInstalacionRequest.java`

```
1 package com.dam.reservas.dto;
2
3 import jakarta.validation.constraints.NotBlank;
4
```

```
5 public record AdminInstalacionRequest(  
6     @NotBlank String nombre,  
7     @NotBlank String direccion,  
8     @NotBlank String ciudad  
9 ) {}
```

dto/AdminInstalacionResponse.java

```
1 package com.dam.reservas.dto;  
2  
3 public record AdminInstalacionResponse(  
4     String id,  
5     String nombre,  
6     String direccion,  
7     String ciudad  
8 ) {}
```

1.26.11.3 1.3 Horarios disponibles (ADMIN) dto/AdminHorarioRequest.java

```
1 package com.dam.reservas.dto;  
2  
3 import jakarta.validation.constraints.NotBlank;  
4 import jakarta.validation.constraints.NotNull;  
5  
6 import java.time.LocalDate;  
7 import java.time.LocalTime;  
8  
9 public record AdminHorarioRequest(  
10     @NotBlank String instalacionId,  
11     @NotNull LocalDate dia,  
12     @NotNull LocalTime horaInicio,  
13     @NotNull LocalTime horaFin  
14 ) {}
```

dto/AdminHorarioResponse.java

```
1 package com.dam.reservas.dto;  
2  
3 import java.time.LocalDate;  
4 import java.time.LocalTime;  
5  
6 public record AdminHorarioResponse(  
7     String id,  
8     String instalacionId,  
9     LocalDate dia,  
10    LocalTime horaInicio,  
11    LocalTime horaFin  
12 ) {}
```

1.26.12 2) Repositorio de horarios (si no lo añadiste ya)

repo/HorarioDisponibleRepository.java

```
1 package com.dam.reservas.repo;
2
3 import com.dam.reservas.model.HorarioDisponible;
4 import org.springframework.data.mongodb.repository.MongoRepository;
5
6 import java.time.LocalDate;
7 import java.util.List;
8
9 public interface HorarioDisponibleRepository extends MongoRepository<
    HorarioDisponible, String> {
10     List<HorarioDisponible> findByInstalacionIdAndDia(String
        instalacionId, LocalDate dia);
11     List<HorarioDisponible> findByDia(LocalDate dia);
12 }
```

1.26.13 3) Controlador ADMIN: Usuarios (CRUD completo)

web/AdminUsuariosController.java

```
1 package com.dam.reservas.web;
2
3 import com.dam.reservas.dto.*;
4 import com.dam.reservas.model.Usuario;
5 import com.dam.reservas.repo.UsuarioRepository;
6 import jakarta.validation.Valid;
7 import org.springframework.security.access.prepost.PreAuthorize;
8 import org.springframework.security.crypto.password.PasswordEncoder;
9 import org.springframework.web.bind.annotation.*;
10
11 import java.util.List;
12
13 @RestController
14 @RequestMapping("/api/admin/usuarios")
15 @PreAuthorize("hasRole('ADMIN')")
16 public class AdminUsuariosController {
17
18     private final UsuarioRepository repo;
19     private final PasswordEncoder encoder;
20
21     public AdminUsuariosController(UsuarioRepository repo,
        PasswordEncoder encoder) {
22         this.repo = repo;
23         this.encoder = encoder;
24     }
25 }
```



```
26 @GetMapping
27 public List<AdminUsuarioResponse> listar() {
28     return repo.findAll().stream()
29         .map(u -> new AdminUsuarioResponse(u.getId(), u.getNombre(), u.
30             getEmail(), u.getRol()))
31         .toList();
32 }
33 @GetMapping("/{id}")
34 public AdminUsuarioResponse obtener(@PathVariable String id) {
35     Usuario u = repo.findById(id).orElseThrow(() -> new
36         NotFoundException("Usuario no encontrado: " + id));
37     return new AdminUsuarioResponse(u.getId(), u.getNombre(), u.
38         getEmail(), u.getRol());
39 }
40 @PostMapping
41 public AdminUsuarioResponse crear(@Valid @RequestBody
42     AdminUsuarioCreateRequest req) {
43     if (repo.existsByEmailIgnoreCase(req.email())) {
44         throw new ConflictException("Ya existe un usuario con ese email")
45         ;
46     }
47     Usuario u = new Usuario(req.nombre(), req.email(), encoder.encode(
48         req.password()), req.rol());
49     u = repo.save(u);
50     return new AdminUsuarioResponse(u.getId(), u.getNombre(), u.
51         getEmail(), u.getRol());
52 }
53 @PutMapping("/{id}")
54 public AdminUsuarioResponse actualizar(@PathVariable String id,
55     @Valid @RequestBody AdminUsuarioUpdateRequest req) {
56     Usuario u = repo.findById(id).orElseThrow(() -> new
57         NotFoundException("Usuario no encontrado: " + id));
58     // email único (si cambia)
59     String emailNuevo = req.email();
60     if (!u.getEmail().equalsIgnoreCase(emailNuevo) && repo.
61         existsByEmailIgnoreCase(emailNuevo)) {
62         throw new ConflictException("Ya existe un usuario con ese email")
63         ;
64     }
65     u.setNombre(req.nombre());
66     u.setEmail(emailNuevo);
67     u.setRol(req.rol());
68     if (req.password() != null && !req.password().isBlank()) {
69         u.setPasswordHash(encoder.encode(req.password()));
70     }
71 }
```

```
66
67     u = repo.save(u);
68     return new AdminUsuarioResponse(u.getId(), u.getNombre(), u.
        getEmail(), u.getRol());
69 }
70
71 @DeleteMapping("/{id}")
72 public void borrar(@PathVariable String id) {
73     if (!repo.existsById(id)) throw new NotFoundException("Usuario no
        encontrado: " + id);
74     repo.deleteById(id);
75 }
76 }
```

1.26.14 4) Controlador ADMIN: Instalaciones (CRUD completo)

web/AdminInstalacionesController.java

```
1  package com.dam.reservas.web;
2
3  import com.dam.reservas.dto.AdminInstalacionRequest;
4  import com.dam.reservas.dto.AdminInstalacionResponse;
5  import com.dam.reservas.model.Instalacion;
6  import com.dam.reservas.repo.InstalacionRepository;
7  import jakarta.validation.Valid;
8  import org.springframework.security.access.prepost.PreAuthorize;
9  import org.springframework.web.bind.annotation.*;
10
11 import java.util.List;
12
13 @RestController
14 @RequestMapping("/api/admin/instalaciones")
15 @PreAuthorize("hasRole('ADMIN')")
16 public class AdminInstalacionesController {
17
18     private final InstalacionRepository repo;
19
20     public AdminInstalacionesController(InstalacionRepository repo) {
21         this.repo = repo;
22     }
23
24     @GetMapping
25     public List<AdminInstalacionResponse> listar() {
26         return repo.findAll().stream()
27             .map(i -> new AdminInstalacionResponse(i.getId(), i.getNombre(),
                i.getDireccion(), i.getCiudad()))
28             .toList();
29     }
30 }
```

```
31 @GetMapping("/{id}")
32 public AdminInstalacionResponse obtener(@PathVariable String id) {
33     Instalacion i = repo.findById(id).orElseThrow(() -> new
34         NotFoundException("Instalación no encontrada: " + id));
35     return new AdminInstalacionResponse(i.getId(), i.getNombre(), i.
36         getDireccion(), i.getCiudad());
37 }
38 @PostMapping
39 public AdminInstalacionResponse crear(@Valid @RequestBody
40     AdminInstalacionRequest req) {
41     Instalacion i = new Instalacion(req.nombre(), req.direccion(), req.
42         ciudad());
43     i = repo.save(i);
44     return new AdminInstalacionResponse(i.getId(), i.getNombre(), i.
45         getDireccion(), i.getCiudad());
46 }
47 @PutMapping("/{id}")
48 public AdminInstalacionResponse actualizar(@PathVariable String id,
49     @Valid @RequestBody AdminInstalacionRequest req) {
50     Instalacion i = repo.findById(id).orElseThrow(() -> new
51         NotFoundException("Instalación no encontrada: " + id));
52     i.setNombre(req.nombre());
53     i.setDireccion(req.direccion());
54     i.setCiudad(req.ciudad());
55     i = repo.save(i);
56     return new AdminInstalacionResponse(i.getId(), i.getNombre(), i.
57         getDireccion(), i.getCiudad());
58 }
59 @DeleteMapping("/{id}")
60 public void borrar(@PathVariable String id) {
61     if (!repo.existsById(id)) throw new NotFoundException("Instalación
62         no encontrada: " + id);
63     repo.deleteById(id);
64 }
```

1.26.15 5) Controlador ADMIN: Horarios disponibles (CRUD completo)

Reglas mínimas:

- `horaInicio` < `horaFin` → 400
- `instalacionId` debe existir → 404
- (Opcional) evitar duplicados/solapes de horarios disponibles. Aquí lo dejamos como extensión; se puede añadir igual que con reservas.

web/AdminHorariosController.java

```
1 package com.dam.reservas.web;
2
3 import com.dam.reservas.dto.AdminHorarioRequest;
4 import com.dam.reservas.dto.AdminHorarioResponse;
5 import com.dam.reservas.model.HorarioDisponible;
6 import com.dam.reservas.repo.HorarioDisponibleRepository;
7 import com.dam.reservas.repo.InstalacionRepository;
8 import jakarta.validation.Valid;
9 import org.springframework.security.access.prepost.PreAuthorize;
10 import org.springframework.web.bind.annotation.*;
11
12 import java.util.List;
13
14 @RestController
15 @RequestMapping("/api/admin/horarios")
16 @PreAuthorize("hasRole('ADMIN')")
17 public class AdminHorariosController {
18
19     private final HorarioDisponibleRepository repo;
20     private final InstalacionRepository instalacionRepo;
21
22     public AdminHorariosController(HorarioDisponibleRepository repo,
23     InstalacionRepository instalacionRepo) {
24         this.repo = repo;
25         this.instalacionRepo = instalacionRepo;
26     }
27
28     @GetMapping
29     public List<AdminHorarioResponse> listar(@RequestParam(required =
30     false) String instalacionId,
31     @RequestParam(required =
32     false) java.time.LocalDate
33     dia) {
34         if (instalacionId != null && !instalacionId.isBlank() && dia !=
35         null) {
36             return repo.findByInstalacionIdAndDia(instalacionId, dia).stream
37             ()
38             .map(h -> new AdminHorarioResponse(h.getId(), h.
39             getInstalacionId(), h.getDia(), h.getHoraInicio(), h.
40             getHoraFin()))
41             .toList();
42         }
43         if (dia != null) {
44             return repo.findByDia(dia).stream()
45             .map(h -> new AdminHorarioResponse(h.getId(), h.
46             getInstalacionId(), h.getDia(), h.getHoraInicio(), h.
47             getHoraFin()))
48             .toList();
49         }
50     }
51 }
```

```
40     return repo.findAll().stream()
41         .map(h -> new AdminHorarioResponse(h.getId(), h.getInstalacionId(), h.getDia(), h.getHoraInicio(), h.getHoraFin()))
42         .toList();
43 }
44
45 @GetMapping("/{id}")
46 public AdminHorarioResponse obtener(@PathVariable String id) {
47     HorarioDisponible h = repo.findById(id).orElseThrow(() -> new
48         NotFoundException("Horario no encontrado: " + id));
49     return new AdminHorarioResponse(h.getId(), h.getInstalacionId(), h.
50         getDia(), h.getHoraInicio(), h.getHoraFin());
51 }
52
53 @PostMapping
54 public AdminHorarioResponse crear(@Valid @RequestBody
55     AdminHorarioRequest req) {
56     validarRango(req);
57     if (!instalacionRepo.existsById(req.instalacionId())) {
58         throw new NotFoundException("Instalación no encontrada: " + req.
59             instalacionId());
60     }
61     HorarioDisponible h = new HorarioDisponible(req.instalacionId(),
62         req.dia(), req.horaInicio(), req.horaFin());
63     h = repo.save(h);
64     return new AdminHorarioResponse(h.getId(), h.getInstalacionId(), h.
65         getDia(), h.getHoraInicio(), h.getHoraFin());
66 }
67
68 @PutMapping("/{id}")
69 public AdminHorarioResponse actualizar(@PathVariable String id,
70     @Valid @RequestBody AdminHorarioRequest req) {
71     validarRango(req);
72     HorarioDisponible h = repo.findById(id).orElseThrow(() -> new
73         NotFoundException("Horario no encontrado: " + id));
74
75     if (!instalacionRepo.existsById(req.instalacionId())) {
76         throw new NotFoundException("Instalación no encontrada: " + req.
77             instalacionId());
78     }
79
80     h.setInstalacionId(req.instalacionId());
81     h.setDia(req.dia());
82     h.setHoraInicio(req.horaInicio());
83     h.setHoraFin(req.horaFin());
84
85     h = repo.save(h);
86     return new AdminHorarioResponse(h.getId(), h.getInstalacionId(), h.
87         getDia(), h.getHoraInicio(), h.getHoraFin());
88 }
```

```
80 @DeleteMapping("/{id}")
81 public void borrar(@PathVariable String id) {
82     if (!repo.existsById(id)) throw new NotFoundException("Horario no
      encontrado: " + id);
83     repo.deleteById(id);
84 }
85
86 private void validarRango(AdminHorarioRequest req) {
87     if (!req.horaInicio().isBefore(req.horaFin())) {
88         throw new BadRequestException("El rango horario no es válido (
      horaInicio debe ser anterior a horaFin)");
89     }
90 }
91 }
```

1.26.16 6) Recomendación: ocultar passwordHash en respuestas públicas

Con ADMIN ya no devolvemos entidad `Usuario` directamente, sino `AdminUsuarioResponse` sin `passwordHash`. Aun así, conviene **no usar jamás `Usuario` como response** en endpoints públicos o de cliente.

1.26.17 7) Endpoints ADMIN finales (resumen)

Todos requieren sesión iniciada y rol ADMIN:

- Usuarios:
 - GET `/api/admin/usuarios`
 - GET `/api/admin/usuarios/{id}`
 - POST `/api/admin/usuarios`
 - PUT `/api/admin/usuarios/{id}`
 - DELETE `/api/admin/usuarios/{id}`
- Instalaciones:
 - GET `/api/admin/instalaciones`
 - GET `/api/admin/instalaciones/{id}`
 - POST `/api/admin/instalaciones`
 - PUT `/api/admin/instalaciones/{id}`
 - DELETE `/api/admin/instalaciones/{id}`
- Horarios disponibles:

- GET /api/admin/horarios?instalacionId=...&dia=YYYY-MM-DD
- GET /api/admin/horarios/{id}
- POST /api/admin/horarios
- PUT /api/admin/horarios/{id}
- DELETE /api/admin/horarios/{id}

1.26.18 8) Nota importante para el siguiente paso (frontend)

Como la autenticación es `formLogin()`, cuando el usuario hace login:

- el navegador guarda una **cookie de sesión**
- y jQuery podrá llamar a `/api/**` sin añadir tokens manualmente (mientras sea mismo origen)

1.26.19 Frontend con login + UI por rol (ADMIN / CONSERJE / CLIENTE)

En este paso dejamos el **frontend funcional con autenticación por sesión (Spring Security form-Login)** y una interfaz que:

- Muestra **pantalla de login**
- Tras login, consulta `/api/auth/me` y **renderiza menús/zonas según rol**
- Implementa:
 - **ADMIN**: CRUD de usuarios, instalaciones y horarios disponibles
 - **CONSERJE**: ver reservas de hoy (sin usuario)
 - **CLIENTE**: ver sus reservas, crear/modificar/borrar si no pasadas, consultar disponibilidad

Incluyo los **cambios mínimos de backend** necesarios para usar una página de login propia (solo configuración), y luego los **archivos estáticos**.

1.26.19.1 1) Backend: ajustar Spring Security para login.html (mínimo) Modifica tu `SecurityConfig` para:

- Permitir `/login.html`
- Usar login page propia
- Mantener sesión/cookies

`security/SecurityConfig.java` (cambia solo el bloque `formLogin` y matchers)

```

1 @Bean
2 public SecurityFilterChain filterChain(HttpSecurity http) throws
   Exception {
3     http
4         .csrf(csrf -> csrf.disable())
5         .authorizeHttpRequests(auth -> auth
6             .requestMatchers(
7                 "/", "/index.html", "/login.html",
8                 "/css/**", "/js/**",
9                 "/favicon.ico"
10            ).permitAll()
11            .requestMatchers("/api/**").authenticated()
12            .anyRequest().permitAll()
13        )
14        .formLogin(form -> form
15            .loginPage("/login.html")
16            .loginProcessingUrl("/login")
17            .defaultSuccessUrl("/", true)
18            .permitAll()
19        )
20        .logout(logout -> logout
21            .logoutUrl("/logout")
22            .logoutSuccessUrl("/login.html")
23            .permitAll()
24        );
25
26     return http.build();
27 }

```

Importante:

- Spring Security espera por defecto parámetros `username` y `password` en el POST `/login`.

1.26.19.2 2) Frontend: archivos Ubicación: `src/main/resources/static/`

2.1 login.html

Página simple con Bootstrap. Envía POST a `/login` con `username` (email) y `password`.

```

1 <!doctype html>
2 <html lang="es">
3 <head>
4     <meta charset="utf-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1" /
6     <title>Login - Reservas</title>

```



```
7   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/
    bootstrap.min.css" rel="stylesheet">
8 </head>
9
10 <body class="bg-light">
11   <main class="container py-5" style="max-width: 520px;">
12     <div class="card shadow-sm">
13       <div class="card-header fw-semibold">Acceso al sistema</div>
14       <div class="card-body">
15
16         <div id="loginError" class="alert alert-danger d-none" role="
            alert"></div>
17
18         <form method="post" action="/login" id="loginForm">
19           <div class="mb-3">
20             <label class="form-label">Email</label>
21             <input class="form-control" name="username" type="email"
                required autocomplete="username">
22           </div>
23           <div class="mb-3">
24             <label class="form-label">Contraseña</label>
25             <input class="form-control" name="password" type="password"
                required autocomplete="current-password">
26           </div>
27           <button class="btn btn-primary w-100" type="submit">Entrar</
            button>
28         </form>
29
30         <hr class="my-4">
31
32         <p class="text-muted mb-0">
33           El acceso está restringido a usuarios registrados (ADMIN,
            CONSERJE, CLIENTE).
34         </p>
35       </div>
36     </div>
37   </main>
38
39   <script>
40     // Si Spring Security redirige a /login.html?error, mostramos
        mensaje.
41     const params = new URLSearchParams(window.location.search);
42     if (params.has("error")) {
43       const el = document.getElementById("loginError");
44       el.textContent = "Credenciales no válidas.";
45       el.classList.remove("d-none");
46     }
47     if (params.has("logout")) {
48       const el = document.getElementById("loginError");
49       el.textContent = "Sesión cerrada.";
50       el.classList.remove("d-none");
```

```

51     el.classList.remove("alert-danger");
52     el.classList.add("alert-success");
53 }
54 </script>
55 </body>
56 </html>

```

2.2 index.html (dashboard por rol)

Este `index.html` carga jQuery, consulta `/api/auth/me` y muestra secciones según rol.

```

1  <!doctype html>
2  <html lang="es">
3  <head>
4    <meta charset="utf-8" />
5    <meta name="viewport" content="width=device-width, initial-scale=1" /
6    <title>Reservas de Pistas</title>
7    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/
8      bootstrap.min.css" rel="stylesheet">
9  </head>
10 <body class="bg-light">
11
12 <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
13   <div class="container">
14     <a class="navbar-brand" href="/">Reservas</a>
15
16     <div class="ms-auto d-flex gap-2 align-items-center">
17       <span id="who" class="navbar-text text-white-50 small"></span>
18       <form method="post" action="/logout" class="mb-0">
19         <button class="btn btn-sm btn-outline-light" type="submit">
20           Salir</button>
21       </form>
22     </div>
23   </div>
24 </nav>
25 <main class="container py-4">
26
27   <div id="alerta" class="alert d-none" role="alert"></div>
28
29   <!-- ADMIN -->
30   <section id="secAdmin" class="d-none">
31     <div class="row g-3">
32
33       <!-- Usuarios -->
34       <div class="col-12">
35         <div class="card shadow-sm">
36           <div class="card-header fw-semibold">ADMIN · Usuarios</div>
37           <div class="card-body">

```

```

38         <form id="formAdminUsuario" class="row g-2 align-items-end"
39             >
40             <div class="col-md-3">
41                 <label class="form-label">Nombre</label>
42                 <input id="auNombre" class="form-control" required>
43             </div>
44             <div class="col-md-3">
45                 <label class="form-label">Email</label>
46                 <input id="auEmail" type="email" class="form-control"
47                     required>
48             </div>
49             <div class="col-md-2">
50                 <label class="form-label">Rol</label>
51                 <select id="auRol" class="form-select" required>
52                     <option value="ADMIN">ADMIN</option>
53                     <option value="CONSERJE">CONSERJE</option>
54                     <option value="CLIENTE" selected>CLIENTE</option>
55                 </select>
56             </div>
57             <div class="col-md-2">
58                 <label class="form-label">Password</label>
59                 <input id="auPass" type="password" class="form-control"
60                     required>
61             </div>
62             <div class="col-md-2 d-grid">
63                 <button class="btn btn-primary" type="submit">Crear</
64                 button>
65             </div>
66         </form>
67
68         <hr>
69
70         <div class="table-responsive">
71             <table class="table table-hover align-middle">
72                 <thead><tr><th>Nombre</th><th>Email</th><th>Rol</th><th
73                     class="text-end">Acciones</th></tr></thead>
74                 <tbody id="tabAdminUsuarios"></tbody>
75             </table>
76         </div>
77     </div>
78 </div>
79 </div>
80 </div>
81
82 <!-- Instalaciones -->
83 <div class="col-12">
84     <div class="card shadow-sm">
85         <div class="card-header fw-semibold">ADMIN · Instalaciones</
86         div>
87         <div class="card-body">
88             <form id="formAdminInst" class="row g-2 align-items-end">
89                 <div class="col-md-4">

```

```

83         <label class="form-label">Nombre</label>
84         <input id="aiNombre" class="form-control" required>
85     </div>
86     <div class="col-md-4">
87         <label class="form-label">Dirección</label>
88         <input id="aiDir" class="form-control" required>
89     </div>
90     <div class="col-md-2">
91         <label class="form-label">Ciudad</label>
92         <input id="aiCiudad" class="form-control" required>
93     </div>
94     <div class="col-md-2 d-grid">
95         <button class="btn btn-primary" type="submit">Crear</
            button>
96     </div>
97 </form>
98
99 <hr>
100
101 <div class="table-responsive">
102     <table class="table table-hover align-middle">
103         <thead><tr><th>Nombre</th><th>Dirección</th><th>Ciudad<
            /th><th class="text-end">Acciones</th></tr></thead>
104         <tbody id="tabAdminInst"></tbody>
105     </table>
106 </div>
107 </div>
108 </div>
109 </div>
110
111 <!-- Horarios -->
112 <div class="col-12">
113     <div class="card shadow-sm">
114         <div class="card-header fw-semibold">ADMIN · Horarios
            disponibles</div>
115         <div class="card-body">
116             <form id="formAdminHorario" class="row g-2 align-items-end"
                >
117                 <div class="col-md-4">
118                     <label class="form-label">Instalación</label>
119                     <select id="ahInst" class="form-select" required></
                        select>
120                 </div>
121                 <div class="col-md-3">
122                     <label class="form-label">Día</label>
123                     <input id="ahDia" type="date" class="form-control"
                        required>
124                 </div>
125                 <div class="col-md-2">
126                     <label class="form-label">Inicio</label>
127                     <input id="ahIni" type="time" class="form-control"

```

```

128         required>
129     </div>
130     <div class="col-md-2">
131         <label class="form-label">Fin</label>
132         <input id="ahFin" type="time" class="form-control"
133             required>
134     </div>
135     <div class="col-md-1 d-grid">
136         <button class="btn btn-primary" type="submit">Crear</
137             button>
138     </div>
139 </form>
140 <hr>
141 <div class="table-responsive">
142     <table class="table table-hover align-middle">
143         <thead><tr><th>Instalación</th><th>Día</th><th>Inicio</
144             th><th>Fin</th><th class="text-end">Acciones</th></
145             tr></thead>
146         <tbody id="tabAdminHorarios"></tbody>
147     </table>
148 </div>
149 </div>
150 </div>
151 </section>
152
153 <!-- CONSERJE -->
154 <section id="secConserje" class="d-none">
155     <div class="card shadow-sm">
156         <div class="card-header fw-semibold">CONSERJE · Reservas de hoy</
157             div>
158         <div class="card-body">
159             <button class="btn btn-outline-secondary mb-3" id="
160                 btnRecargarHoy">Recargar</button>
161             <div class="table-responsive">
162                 <table class="table table-hover align-middle">
163                     <thead><tr><th>Instalación</th><th>Ciudad</th><th>Inicio</
164                         th><th>Fin</th></tr></thead>
165                     <tbody id="tabConserjeHoy"></tbody>
166                 </table>
167             </div>
168         </div>
169     </div>
170 </section>

```

```

171     <div class="row g-3">
172
173         <div class="col-12">
174             <div class="card shadow-sm">
175                 <div class="card-header fw-semibold">CLIENTE · Disponibilidad
176                 </div>
177                 <div class="card-body">
178                     <form id="formDisp" class="row g-2 align-items-end">
179                         <div class="col-md-3">
180                             <label class="form-label">Día</label>
181                             <input id="cdDia" type="date" class="form-control"
182                                 required>
183                         </div>
184                         <div class="col-md-2">
185                             <label class="form-label">Inicio</label>
186                             <input id="cdIni" type="time" class="form-control"
187                                 required>
188                         </div>
189                         <div class="col-md-2">
190                             <label class="form-label">Fin</label>
191                             <input id="cdFin" type="time" class="form-control"
192                                 required>
193                         </div>
194                         <div class="col-md-3 d-grid">
195                             <button class="btn btn-outline-primary" type="submit">
196                                 Ver pistas libres</button>
197                         </div>
198                     </form>
199
200                     <hr>
201
202                     <div class="table-responsive">
203                         <table class="table table-hover align-middle">
204                             <thead><tr><th>Instalación</th><th>Dirección</th><th>
205                                 Ciudad</th></tr></thead>
206                             <tbody id="tabDisp"></tbody>
207                         </table>
208                     </div>
209                 </div>
210             </div>
211         </div>
212
213         <div class="col-12">
214             <div class="card shadow-sm">
215                 <div class="card-header fw-semibold">CLIENTE · Mis reservas</
216                 div>
217                 <div class="card-body">
218                     <form id="formClienteReserva" class="row g-2 align-items-
219                         end">
220                         <div class="col-md-4">
221                             <label class="form-label">Instalación</label>

```

```

214         <select id="crInst" class="form-select" required></
           select>
215     </div>
216     <div class="col-md-3">
217         <label class="form-label">Día</label>
218         <input id="crDia" type="date" class="form-control"
           required>
219     </div>
220     <div class="col-md-2">
221         <label class="form-label">Inicio</label>
222         <input id="crIni" type="time" class="form-control"
           required>
223     </div>
224     <div class="col-md-2">
225         <label class="form-label">Fin</label>
226         <input id="crFin" type="time" class="form-control"
           required>
227     </div>
228     <div class="col-md-1 d-grid">
229         <button class="btn btn-success" type="submit">Crear</
           button>
230     </div>
231 </form>
232
233 <hr>
234
235 <div class="table-responsive">
236     <table class="table table-hover align-middle">
237         <thead><tr><th>Día</th><th>Horario</th><th>Instalación<
           /th><th class="text-end">Acciones</th></tr></thead>
238         <tbody id="tabMisReservas"></tbody>
239     </table>
240 </div>
241 <p class="text-muted mb-0">Solo podrás modificar o borrar
           reservas futuras.</p>
242 </div>
243 </div>
244 </div>
245
246 </div>
247 </section>
248
249 </main>
250
251 <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
252 <script src="js/app-secure.js"></script>
253 </body>
254 </html>

```

2.3 js/app-secure.js (jQuery + endpoints por rol)

Este fichero:

- comprueba sesión con `/api/auth/me`
- si 401, redirige a `/login.html`
- carga y opera según rol

```
1  /* global $ */
2
3  const API = {
4    me: "/api/auth/me",
5
6    admin: {
7      usuarios: "/api/admin/usuarios",
8      instalaciones: "/api/admin/instalaciones",
9      horarios: "/api/admin/horarios"
10   },
11
12   conserje: {
13     hoy: "/api/conserje/reservas-hoy"
14   },
15
16   cliente: {
17     reservas: "/api/cliente/reservas",
18     disponibilidad: "/api/cliente/disponibilidad"
19   }
20 };
21
22 let session = null;
23 let cacheInstalaciones = [];
24
25 $(document).ready(async function () {
26   try {
27     session = await getMe();
28   } catch (e) {
29     // no autenticado
30     window.location.href = "/login.html";
31     return;
32   }
33
34   $("#who").text(`${session.email} · ${session.rol}`);
35
36   if (session.rol === "ADMIN") initAdmin();
37   if (session.rol === "CONSERJE") initConserje();
38   if (session.rol === "CLIENTE") initCliente();
39 });
40
41 /* -----
42    Helpers
43 ----- */
44
```



```
45 function showAlert(type, msg) {
46     $("#alerta")
47         .removeClass("d-none alert-success alert-danger alert-warning alert
            -info")
48         .addClass("alert-" + type)
49         .text(msg);
50     setTimeout(() => $("#alerta").addClass("d-none"), 3500);
51 }
52
53 function parseApiError(xhr, fallback) {
54     const r = xhr.responseJSON;
55     if (!r) return fallback;
56     if (Array.isArray(r.details) && r.details.length) return `${r.message
        }: ${r.details.join(" | ")}`;
57     return r.message || fallback;
58 }
59
60 function escapeHtml(s) {
61     return String(s)
62         .replaceAll("&", "&amp;")
63         .replaceAll("<", "&lt;")
64         .replaceAll(">", "&gt;");
65 }
66
67 function ajaxJson(method, url, body) {
68     return $.ajax({
69         method,
70         url,
71         contentType: "application/json",
72         data: body ? JSON.stringify(body) : undefined
73     });
74 }
75
76 async function getMe() {
77     return $.getJSON(API.me);
78 }
79
80 function fillSelectInst($sel, instalaciones, placeholder) {
81     const opts = (instalaciones || []).map(i =>
82         `
```

```

92     $("#secAdmin").removeClass("d-none");
93
94     $("#formAdminUsuario").on("submit", crearAdminUsuario);
95     $("#formAdminInst").on("submit", crearAdminInstalacion);
96     $("#formAdminHorario").on("submit", crearAdminHorario);
97
98     recargarAdmin();
99 }
100
101 function recargarAdmin() {
102     $.when(
103         cargarAdminUsuarios(),
104         cargarAdminInstalaciones(),
105         cargarAdminHorarios()
106     ).fail(() => showAlert("danger", "Error cargando datos de
107         administración"));
108 }
109
110 function cargarAdminUsuarios() {
111     return $.getJSON(API.admin.usuarios)
112         .done(renderAdminUsuarios)
113         .fail(xhr => showAlert("danger", parseApiError(xhr, "Error cargando
114             usuarios")));
115 }
116
117 function renderAdminUsuarios(items) {
118     const rows = (items || []).map(u => `
119         <tr>
120             <td>${escapeHtml(u.nombre)}</td>
121             <td>${escapeHtml(u.email)}</td>
122             <td>${escapeHtml(u.rol)}</td>
123             <td class="text-end">
124                 <button class="btn btn-sm btn-outline-danger" data-del-user="${
125                     u.id}>Eliminar</button>
126             </td>
127         </tr>
128     `).join("");
129
130     $("#tabAdminUsuarios").html(rows || `<tr><td colspan="4" class="text-
131         center text-muted">Sin datos</td></tr>`);
132
133     $("#tabAdminUsuarios button[data-del-user]").off("click").on("click",
134         function () {
135             const id = $(this).data("del-user");
136             if (!confirm("? Eliminar usuario?")) return;
137             $.ajax({ method: "DELETE", url: `${API.admin.usuarios}/${id}` })
138                 .done(() => { showAlert("success", "Usuario eliminado");
139                     cargarAdminUsuarios(); })
140                 .fail(xhr => showAlert("danger", parseApiError(xhr, "Error
141                     eliminando usuario")));
142         });

```

```
136 }
137
138 function crearAdminUsuario(e) {
139   e.preventDefault();
140   const payload = {
141     nombre: $("#auNombre").val().trim(),
142     email: $("#auEmail").val().trim(),
143     rol: $("#auRol").val(),
144     password: $("#auPass").val()
145   };
146
147   ajaxJson("POST", API.admin.usuarios, payload)
148     .done(() => { showAlert("success", "Usuario creado"); $("#formAdminUsuario")[0].reset(); cargarAdminUsuarios(); })
149     .fail(xhr => showAlert("danger", parseApiError(xhr, "Error creando usuario")));
150 }
151
152 function cargarAdminInstalaciones() {
153   return $.getJSON(API.admin.instalaciones)
154     .done(data => {
155       cacheInstalaciones = data || [];
156       renderAdminInstalaciones(cacheInstalaciones);
157       fillSelectInst($("#ahInst"), cacheInstalaciones, "Seleccione instalación...");
158       fillSelectInst($("#crInst"), cacheInstalaciones, "Seleccione instalación...");
159     })
160     .fail(xhr => showAlert("danger", parseApiError(xhr, "Error cargando instalaciones")));
161 }
162
163 function renderAdminInstalaciones(items) {
164   const rows = (items || []).map(i => `
165     <tr>
166       <td>${escapeHtml(i.nombre)}</td>
167       <td>${escapeHtml(i.direccion)}</td>
168       <td>${escapeHtml(i.ciudad)}</td>
169       <td class="text-end">
170         <button class="btn btn-sm btn-outline-danger" data-del-inst="${i.id}>Eliminar</button>
171       </td>
172     </tr>
173   `).join("");
174
175   $("#tabAdminInst").html(rows || `<tr><td colspan="4" class="text-center text-muted">Sin datos</td></tr>`);
176
177   $("#tabAdminInst button[data-del-inst]").off("click").on("click",
178     function () {
179       const id = $(this).data("del-inst");
```

```

179     if (!confirm("?`Eliminar instalación?`")) return;
180     $.ajax({ method: "DELETE", url: `${API.admin.instalaciones}/${id}`
181         })
182         .done(() => { showAlert("success", "Instalación eliminada");
183             cargarAdminInstalaciones(); })
184         .fail(xhr => showAlert("danger", parseApiError(xhr, "Error
185             eliminando instalación")));
186     });
187 }
188
189 function crearAdminInstalacion(e) {
190     e.preventDefault();
191     const payload = {
192         nombre: $("#aiNombre").val().trim(),
193         direccion: $("#aiDir").val().trim(),
194         ciudad: $("#aiCiudad").val().trim()
195     };
196     ajaxJson("POST", API.admin.instalaciones, payload)
197     .done(() => { showAlert("success", "Instalación creada"); $("#
198         formAdminInst")[0].reset(); cargarAdminInstalaciones(); })
199     .fail(xhr => showAlert("danger", parseApiError(xhr, "Error creando
200         instalación")));
201 }
202
203 function cargarAdminHorarios() {
204     return $.getJSON(API.admin.horarios)
205     .done(renderAdminHorarios)
206     .fail(xhr => showAlert("danger", parseApiError(xhr, "Error cargando
207         horarios")));
208 }
209
210 function renderAdminHorarios(items) {
211     const instName = new Map((cacheInstalaciones || []).map(i => [i.id, `
212         ${i.nombre} (${i.ciudad})`]));
213
214     const rows = (items || []).map(h => `
215         <tr>
216             <td>${escapeHtml(instName.get(h.instalacionId) || h.instalacionId
217                 )}</td>
218             <td>${escapeHtml(h.dia)}</td>
219             <td>${escapeHtml(h.horaInicio)}</td>
220             <td>${escapeHtml(h.horaFin)}</td>
221             <td class="text-end">
222                 <button class="btn btn-sm btn-outline-danger" data-del-h="${h.
223                     id}">Eliminar</button>
224             </td>
225         </tr>
226     `).join("");
227     $("#tabAdminHorarios").html(rows || `<tr><td colspan="5" class="text-

```

```

        center text-muted">Sin datos</td></tr>`);
221
222     $("#tabAdminHorarios button[data-del-h]").off("click").on("click",
        function () {
223         const id = $(this).data("del-h");
224         if (!confirm("? Eliminar horario?")) return;
225         $.ajax({ method: "DELETE", url: `${API.admin.horarios}/${id}` })
226             .done(() => { showAlert("success", "Horario eliminado");
                cargarAdminHorarios(); })
227             .fail(xhr => showAlert("danger", parseApiError(xhr, "Error
                eliminando horario")));
228     });
229 }
230
231 function crearAdminHorario(e) {
232     e.preventDefault();
233     const payload = {
234         instalacionId: $("#ahInst").val(),
235         dia: $("#ahDia").val(),
236         horaInicio: $("#ahIni").val(),
237         horaFin: $("#ahFin").val()
238     };
239
240     ajaxJson("POST", API.admin.horarios, payload)
241         .done(() => { showAlert("success", "Horario creado"); $("#
            formAdminHorario")[0].reset(); cargarAdminHorarios(); })
242         .fail(xhr => showAlert("danger", parseApiError(xhr, "Error creando
            horario")));
243 }
244
245 /* -----
246     CONSERJE
247 ----- */
248
249 function initConserje() {
250     $("#secConserje").removeClass("d-none");
251     $("#btnRecargarHoy").on("click", cargarConserjeHoy);
252     cargarConserjeHoy();
253 }
254
255 function cargarConserjeHoy() {
256     $.getJSON(API.conserje.hoy)
257         .done(items => {
258             const rows = (items || []).map(x => `
259                 <tr>
260                     <td>${escapeHtml(x.instalacionNombre)}</td>
261                     <td>${escapeHtml(x.instalacionCiudad)}</td>
262                     <td>${escapeHtml(x.horaInicio)}</td>
263                     <td>${escapeHtml(x.horaFin)}</td>
264                 </tr>
265             `).join("");

```

```

266     $("#tabConserjeHoy").html(rows || `<tr><td colspan="4" class="
        text-center text-muted">Sin reservas</td></tr>`);
267     })
268     .fail(xhr => showAlert("danger", parseApiError(xhr, "Error cargando
        reservas de hoy")));
269 }
270
271 /* -----
272     CLIENTE
273 ----- */
274
275 function initCliente() {
276     $("#secCliente").removeClass("d-none");
277
278     // disponibilidad
279     $("#formDisp").on("submit", consultarDisponibilidad);
280
281     // reservas
282     $("#formClienteReserva").on("submit", crearReservaCliente);
283
284     // Necesitamos instalaciones para el select
285     // Si no es ADMIN, no tenemos cacheInstalaciones: las pedimos por
        admin endpoint? No.
286     // Para CLIENTE, lo correcto es exponer un endpoint público
        autenticado de instalaciones.
287     // Como aún no lo has creado, reutilizamos temporalmente /api/admin/
        instalaciones NO (está protegido).
288     // Solución: crear /api/instalaciones (autenticado) o /api/cliente/
        instalaciones.
289     // Aquí asumimos que ya mantienes /api/instalaciones autenticado (tu
        API antigua). Si no, dímelo y te doy el endpoint.
290     $.getJSON("/api/instalaciones")
291     .done(data => {
292         cacheInstalaciones = data || [];
293         fillSelectInst($("#crInst"), cacheInstalaciones, "Seleccione
            instalación...");
294     })
295     .fail(xhr => showAlert("danger", parseApiError(xhr, "Error cargando
        instalaciones")));
296
297     cargarMisReservas();
298 }
299
300 function consultarDisponibilidad(e) {
301     e.preventDefault();
302     const payload = {
303         dia: $("#cdDia").val(),
304         horaInicio: $("#cdIni").val(),
305         horaFin: $("#cdFin").val()
306     };
307

```

```

308     ajaxJson("POST", API.cliente.disponibilidad, payload)
309     .done(items => {
310         const rows = (items || []).map(i => `
311             <tr>
312                 <td>${escapeHtml(i.nombre)}</td>
313                 <td>${escapeHtml(i.direccion)}</td>
314                 <td>${escapeHtml(i.ciudad)}</td>
315             </tr>
316         `).join("");
317         $("#tabDisp").html(rows || `<tr><td colspan="3" class="text-
318             center text-muted">Sin disponibilidad</td></tr>`);
319     })
320     .fail(xhr => showAlert("danger", parseApiError(xhr, "Error
321         consultando disponibilidad")));
322 }
323
324 function cargarMisReservas() {
325     $.getJSON(API.cliente.reservas)
326     .done(renderMisReservas)
327     .fail(xhr => showAlert("danger", parseApiError(xhr, "Error cargando
328         mis reservas")));
329 }
330
331 function crearReservaCliente(e) {
332     e.preventDefault();
333     const payload = {
334         instalacionId: $("#crInst").val(),
335         dia: $("#crDia").val(),
336         horaInicio: $("#crIni").val(),
337         horaFin: $("#crFin").val()
338     };
339
340     ajaxJson("POST", API.cliente.reservas, payload)
341     .done(() => { showAlert("success", "Reserva creada"); $("#
342         formClienteReserva")[0].reset(); cargarMisReservas(); })
343     .fail(xhr => showAlert("danger", parseApiError(xhr, "Error creando
344         reserva")));
345 }
346
347 function renderMisReservas(items) {
348     const instName = new Map((cacheInstalaciones || []).map(i => [i.id, `
349         ${i.nombre} (${i.ciudad)}`]));
350
351     const rows = (items || []).map(r => {
352         const h = r.horario || {};
353         const snap = h.instalacionSnapshot || {};
354         const inst = instName.get(snap.instalacionId) || snap.nombre ||
355             snap.instalacionId || "";
356
357         return `
358             <tr>

```

```

352         <td>${escapeHtml(h.dia || "")}</td>
353         <td>${escapeHtml((h.horaInicio || "") + " - " + (h.horaFin || "
354         <td>${escapeHtml(inst)}</td>
355         <td class="text-end">
356             <button class="btn btn-sm btn-outline-danger" data-del-res="$
357                 {r.id}">Borrar</button>
358         </td>
359     </tr>
360     `).join("");
361
362     $("#tabMisReservas").html(rows || `<tr><td colspan="4" class="text-
363         center text-muted">Sin reservas</td></tr>`);
364
365     $("#tabMisReservas button[data-del-res]").off("click").on("click",
366         function () {
367         const id = $(this).data("del-res");
368         if (!confirm("?Borrar reserva? (solo se permitirá si es futura)"))
369             return;
370         $.ajax({ method: "DELETE", url: `${API.cliente.reservas}/${id}` })
371         .done(() => { showAlert("success", "Reserva borrada");
372             cargarMisReservas(); })
373         .fail(xhr => showAlert("danger", parseApiError(xhr, "Error
374             borrando reserva")));
375     });
376 }

```

1.26.19.3 3) Nota importante (para que CLIENTE cargue instalaciones sin ser ADMIN) En el JS del cliente he supuesto que existe `GET /api/instalaciones` accesible a usuarios autenticados (tu API “antigua”). Si actualmente has movido todo a `/api/admin/**`, crea un endpoint simple para instalaciones autenticadas, por ejemplo:

- `GET /api/instalaciones` (autenticado, cualquier rol)
- o `GET /api/cliente/instalaciones` (solo CLIENTE)

De esta manera el controlador quedaría así:

```

1  package com.dam.reservas.web;
2
3  import com.dam.reservas.model.Instalacion;
4  import com.dam.reservas.repo.InstalacionRepository;
5  import org.springframework.security.access.prepost.PreAuthorize;
6  import org.springframework.web.bind.annotation.*;
7
8  import java.util.List;
9
10 @RestController

```



```
11 @RequestMapping("/api/instalaciones")
12 @PreAuthorize("isAuthenticated()") // cualquier usuario autenticado (
    ADMIN/CONSERJE/CLIENTE)
13 public class InstalacionesController {
14
15     private final InstalacionRepository repo;
16
17     public InstalacionesController(InstalacionRepository repo) {
18         this.repo = repo;
19     }
20
21     /**
22      * Listado de instalaciones accesible para cualquier usuario
23      * autenticado.
24      * Filtros opcionales:
25      * - ?ciudad=Jaen
26      * - ?q=pista
27      */
28     @GetMapping
29     public List<Instalacion> listar(@RequestParam(required = false)
        String ciudad,
        @RequestParam(required = false)
        String q) {
30
31         if (ciudad != null && !ciudad.isBlank()) {
32             return repo.findByCiudadIgnoreCase(ciudad.trim());
33         }
34         if (q != null && !q.isBlank()) {
35             return repo.findByNombreContainingIgnoreCase(q.trim());
36         }
37         return repo.findAll();
38     }
39
40     @GetMapping("/{id}")
41     public Instalacion obtener(@PathVariable String id) {
42         return repo.findById(id).orElseThrow(() -> new NotFoundException("
        Instalación no encontrada: " + id));
43     }
44 }
```

**4) Cómo probar este paso*

1. Arrancar backend con `./mvnw spring-boot:run`
2. Crear un usuario ADMIN desde Mongo Express.
3. Ir a <http://localhost:8080/login.html>
4. Login con email/password
5. Ver:
 - ADMIN: CRUD en el dashboard

- CONSERJE: listado reservas de hoy
- CLIENTE: disponibilidad + reservas

A continuación dejo el **siguiente paso lógico**: ampliar la UI de **CLIENTE** para **editar reservas (PUT)** con un **modal Bootstrap** y mantener el borrado (DELETE). Incluye:

- Cambios en `index.html` (sección CLIENTE): añadir botón **Editar** y modal.
- Cambios en `js/app-secure.js`: abrir modal, precargar datos, enviar PUT a `/api/cliente/reservas/{id}`, refrescar tabla y gestionar errores (409/400/404).

Requisito backend: ya tienes `PUT /api/cliente/reservas/{id}` implementado en `ClienteReservasController` con la regla “no pasada”. Si la reserva es pasada o solapa, devolverá 409.

1.26.19.4 Modificaciones en el index.html y app.js 1) `index.html`: añadir botón Editar y el modal Bootstrap

1.1 Modificar la tabla “CLIENTE · Mis reservas”

Sustituye el `<thead>` y el `<tbody>` de `tabMisReservas` para que tenga dos acciones: **Editar** y **Borrar**.

En tu sección CLIENTE (tabla de mis reservas), usa esto:

```
1 <table class="table table-hover align-middle">
2   <thead>
3     <tr>
4       <th>Día</th>
5       <th>Horario</th>
6       <th>Instalación</th>
7       <th class="text-end">Acciones</th>
8     </tr>
9   </thead>
10  <tbody id="tabMisReservas"></tbody>
11 </table>
```

(La cabecera es igual, pero el render JS pondrá 2 botones.)

1.2 Añadir el modal de edición (al final del `<main>` o justo antes de `</main>`)

```
1 <!-- Modal: Editar reserva -->
2 <div class="modal fade" id="modalEditarReserva" tabindex="-1" aria-
  hidden="true">
3   <div class="modal-dialog">
4     <div class="modal-content">
5
6       <div class="modal-header">
```

```

7      <h5 class="modal-title">Editar reserva</h5>
8      <button type="button" class="btn btn-close" data-bs-dismiss="modal"
      aria-label="Cerrar"></button>
9  </div>
10
11  <div class="modal-body">
12    <div class="alert alert-danger d-none" id="editError" role="
      alert"></div>
13
14    <form id="formEditarReserva" class="row g-2">
15      <input type="hidden" id="editReservaId">
16
17      <div class="col-12">
18        <label class="form-label">Instalación</label>
19        <select id="editInst" class="form-select" required></select>
20      </div>
21
22      <div class="col-md-6">
23        <label class="form-label">Día</label>
24        <input id="editDia" type="date" class="form-control"
      required>
25      </div>
26
27      <div class="col-md-3">
28        <label class="form-label">Inicio</label>
29        <input id="editIni" type="time" class="form-control"
      required>
30      </div>
31
32      <div class="col-md-3">
33        <label class="form-label">Fin</label>
34        <input id="editFin" type="time" class="form-control"
      required>
35      </div>
36    </form>
37  </div>
38
39  <div class="modal-footer">
40    <button type="button" class="btn btn-outline-secondary" data-bs-
      -dismiss="modal">Cancelar</button>
41    <button type="button" class="btn btn-primary" id="
      btnGuardarEdicion">Guardar cambios</button>
42  </div>
43
44  </div>
45  </div>
46  </div>

```

1.3 Asegurar que Bootstrap JS está incluido

En tu `index.html` actual con jQuery, añade también Bootstrap bundle (si no lo tienes en esta versión). Debe ir antes de `app-secure.js`:

```
1 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/  
  bootstrap.bundle.min.js"></script>  
2 <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>  
3 <script src="js/app-secure.js"></script>
```

Importante: `bootstrap.bundle.min.js` incluye `Modal`.

2) `js/app-secure.js`: añadir edición (PUT) con modal

A continuación te dejo **solo las partes a añadir/modificar** dentro del fichero.

2.1 Estado para reservas y modal

Añade arriba, junto a `session` y `cacheInstalaciones`:

```
1 let cacheMisReservas = [];  
2 let modalEditar = null;
```

2.2 Inicializar modal y eventos en `initCliente()`

Dentro de `initCliente()` añade:

```
1 function initCliente() {  
2   $("#secCliente").removeClass("d-none");  
3  
4   // Bootstrap Modal instance  
5   modalEditar = new bootstrap.Modal(document.getElementById("  
    modalEditarReserva"));  
6  
7   // disponibilidad  
8   $("#formDisp").on("submit", consultarDisponibilidad);  
9  
10  // reservas  
11  $("#formClienteReserva").on("submit", crearReservaCliente);  
12  
13  // Edición  
14  $("#btnGuardarEdicion").on("click", guardarEdicionReserva);  
15  
16  // Cargar instalaciones (endpoint accesible a autenticados)  
17  $.getJSON("/api/instalaciones")  
18    .done(data => {  
19    cacheInstalaciones = data || [];  
20    fillSelectInst($("#crInst"), cacheInstalaciones, "Seleccione  
      instalación...");  
21    fillSelectInst($("#editInst"), cacheInstalaciones, "Seleccione  
      instalación...");  
22  })
```

```

23     .fail(xhr => showAlert("danger", parseApiError(xhr, "Error cargando
    instalaciones"))));
24
25     cargarMisReservas();
26 }

```

2.3 Modificar `cargarMisReservas()` para guardar cache

```

1 function cargarMisReservas() {
2     $.getJSON(API.cliente.reservas)
3     .done(items => {
4         cacheMisReservas = items || [];
5         renderMisReservas(cacheMisReservas);
6     })
7     .fail(xhr => showAlert("danger", parseApiError(xhr, "Error cargando
    mis reservas"))));
8 }

```

2.4 Modificar `renderMisReservas()` para incluir “Editar”

Sustituimos `renderMisReservas` por esta versión:

```

1 function renderMisReservas(items) {
2     const instName = new Map((cacheInstalaciones || []).map(i => [i.id, `
    ${i.nombre} (${i.ciudad})`]));
3
4     const rows = (items || []).map(r => {
5         const h = r.horario || {};
6         const snap = h.instalacionSnapshot || {};
7         const instId = snap.instalacionId;
8         const inst = instName.get(instId) || snap.nombre || instId || "";
9
10        return `
11        <tr>
12            <td>${escapeHtml(h.dia || "")}</td>
13            <td>${escapeHtml((h.horaInicio || "") + " - " + (h.horaFin || "
    "))}</td>
14            <td>${escapeHtml(inst)}</td>
15            <td class="text-end">
16                <button class="btn btn-sm btn-outline-primary me-2" data-edit-
    -res="${r.id}">Editar</button>
17                <button class="btn btn-sm btn-outline-danger" data-del-res="$
    {r.id}">Borrar</button>
18            </td>
19        </tr>
20        `;
21    }).join("");
22
23    $("#tabMisReservas").html(rows || `<tr><td colspan="4" class="text-
    center text-muted">Sin reservas</td></tr>`);
24

```

```
25 // Editar
26 $("#tabMisReservas button[data-edit-res]").off("click").on("click",
    function () {
27     const id = $(this).data("edit-res");
28     abrirModalEdicion(id);
29 });
30
31 // Borrar
32 $("#tabMisReservas button[data-del-res]").off("click").on("click",
    function () {
33     const id = $(this).data("del-res");
34     if (!confirm("? Borrar reserva? (solo se permitirá si es futura)"))
        return;
35     $.ajax({ method: "DELETE", url: `${API.cliente.reservas}/${id}` })
36         .done(() => { showAlert("success", "Reserva borrada");
37             cargarMisReservas(); })
38         .fail(xhr => showAlert("danger", parseApiError(xhr, "Error
39             borrando reserva"))));
40     });
41 }
```

2.5 Función: abrir modal con datos precargados

Añadimos estas funciones nuevas:

```
1 function abrirModalEdicion(reservaId) {
2     const r = (cacheMisReservas || []).find(x => x.id === reservaId);
3     if (!r) {
4         showAlert("danger", "No se encontró la reserva en memoria.
5             Recargando...");
6         return cargarMisReservas();
7     }
8
9     // reset de error
10    $("#editError").addClass("d-none").text("");
11
12    const h = r.horario || {};
13    const snap = h.instalacionSnapshot || {};
14
15    $("#editReservaId").val(r.id);
16    $("#editDia").val(h.dia || "");
17    $("#editIni").val(h.horaInicio || "");
18    $("#editFin").val(h.horaFin || "");
19
20    // Asegurar que el select está relleno
21    if (!cacheInstalaciones || cacheInstalaciones.length === 0) {
22        // si por lo que sea aún no están, intenta cargarlas
23        $.getJSON("/api/instalaciones").done(data => {
24            cacheInstalaciones = data || [];
25            fillSelectInst($("#editInst"), cacheInstalaciones, "Seleccione
26                instalación...");
27        });
28    }
29 }
```

```
25     $("#editInst").val(snap.instalacionId || "");
26     modalEditar.show();
27     }).fail(xhr => showAlert("danger", parseApiError(xhr, "Error
    cargando instalaciones")));
28     return;
29 }
30
31 // seleccionar instalación
32 $("#editInst").val(snap.instalacionId || "");
33
34 modalEditar.show();
35 }
```

2.6 Función: guardar cambios (PUT) y cerrar modal

```
1 function guardarEdicionReserva() {
2     const id = $("#editReservaId").val();
3     const payload = {
4         instalacionId: $("#editInst").val(),
5         dia: $("#editDia").val(),
6         horaInicio: $("#editIni").val(),
7         horaFin: $("#editFin").val()
8     };
9
10    // Validación mínima en cliente (la API valida igualmente)
11    if (!payload.instalacionId || !payload.dia || !payload.horaInicio ||
        !payload.horaFin) {
12        return mostrarErrorEdicion("Todos los campos son obligatorios.");
13    }
14
15    ajaxJson("PUT", `${API.cliente.reservas}/${id}`, payload)
16        .done(() => {
17            modalEditar.hide();
18            showAlert("success", "Reserva modificada");
19            cargarMisReservas();
20        })
21        .fail(xhr => {
22            const msg = parseApiError(xhr, "Error modificando reserva");
23            // Si es "reserva" pasada o solape, normalmente será 409; en
                ambos casos se muestra aquí
24            mostrarErrorEdicion(msg);
25        });
26 }
27
28 function mostrarErrorEdicion(msg) {
29     $("#editError").removeClass("d-none").text(msg);
30 }
```

3) Comportamiento esperado

- El cliente pulsa **Editar** en una reserva → se abre modal con:

- instalación actual seleccionada
- día e inicio/fin precargados
- Al guardar:
 - si la reserva es futura y no hay solape → 200 OK, se cierra modal y refresca tabla
 - si es pasada → 409 Conflict con mensaje “No se puede modificar una reserva pasada”
 - si solapa → 409 Conflict
 - si el rango es inválido → 400 Bad Request

4) Mejora para UX

- En la tabla de reservas, **deshabilitar** el botón Editar/Borrar si la reserva ya es pasada (usando la misma lógica en JS comparando con la fecha/hora actual). Esto no sustituye la seguridad del backend, pero evita frustración en el usuario.

```

1 function renderMisReservas(items) {
2   const instName = new Map((cacheInstalaciones || []).map(i => [i.id, `
   ${i.nombre} (${i.ciudad})`]));
3
4   const rows = (items || []).map(r => {
5     const h = r.horario || {};
6     const snap = h.instalacionSnapshot || {};
7     const instId = snap.instalacionId;
8     const inst = instName.get(instId) || snap.nombre || instId || "";
9
10    const pasada = reservaYaPasadaUI(r);
11    const disAttr = pasada ? "disabled" : "";
12    const titleAttr = pasada ? `title="Reserva pasada: no editable"` :
      "";
13
14    return `
15      <tr>
16        <td>${escapeHtml(h.dia || "")}</td>
17        <td>${escapeHtml((h.horaInicio || "") + " - " + (h.horaFin || "
      "))}</td>
18        <td>${escapeHtml(inst)}</td>
19        <td class="text-end">
20          <button class="btn btn-sm btn-outline-primary me-2"
21            data-edit-res="${r.id}" ${disAttr} ${titleAttr}>
22            Editar
23          </button>
24          <button class="btn btn-sm btn-outline-danger"
25            data-del-res="${r.id}" ${disAttr} ${titleAttr}>
26            Borrar
27          </button>
28        </td>
29      </tr>
30    `;

```



```
31  }).join("");
32
33  $("#tabMisReservas").html(rows || `|<td colspan="4" class="text-center text-muted">Sin reservas</td></tr>`);
34
35  // Editar
36  $("#tabMisReservas button[data-edit-res]").off("click").on("click",
37    function () {
38    if ($(this).is(":disabled")) return;
39    const id = $(this).data("edit-res");
40    abrirModalEdicion(id);
41  });
42
43  // Borrar
44  $("#tabMisReservas button[data-del-res]").off("click").on("click",
45    function () {
46    if ($(this).is(":disabled")) return;
47    const id = $(this).data("del-res");
48    if (!confirm("?Borrar reserva? (solo se permitirá si es futura)"))
49      return;
50    $.ajax({ method: "DELETE", url: `${API.cliente.reservas}/${id}` })
51      .done(() => { showAlert("success", "Reserva borrada");
52        cargarMisReservas(); })
53      .fail(xhr => showAlert("danger", parseApiError(xhr, "Error
54        borrando reserva")));
55  });
56 }
|  |

```

2 Bibliografía

- Documentación oficial de MongoDB: (<https://www.mongodb.com/docs/>).